

Analisando uma população de agentes submetidos a neuro-evolução em um jogo digital.

Guilherme Cunha Prada

Trabalho de Conclusão de Curso - MBA em Ciência de Dados
(CEMEAI)

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Analizando uma população de agentes
submetidos a neuro-evolução em um jogo
digital.

GUILHERME CUNHA PRADA

Analizando uma população de agentes submetidos a neuro-evolução em um jogo digital.

Trabalho de conclusão de curso apresentado ao Centro de Ciências Matemáticas Aplicadas à Indústria do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, como parte dos requisitos para conclusão do MBA em Ciência de Dados.

Área de concentração: Ciências de Dados

Orientador: Profa. Dra. Kamila Rios H. Rodrigues

USP - São Carlos

2023

Ficha catalográfica elaborada pela Biblioteca Prof. Achille Bassi
e Seção Técnica de Informática, ICMC/USP,
com os dados inseridos pelo(a) autor(a)

P896a Prada, Guilherme Cunha
Analisando uma população de agentes submetidos a
neuro-evolução em um jogo digital. / Guilherme Cunha
Prada; orientadora Kamila Rios da Hora Rodrigues. --
São Carlos, 2023.
112 p.

Trabalho de conclusão de curso (MBA em Ciência
de Dados) -- Instituto de Ciências Matemáticas e de
Computação, Universidade de São Paulo, 2023.

1. Inteligência Artificial. 2. Neuro-evolução. 3.
Aprendizado de Máquina. 4. Aprendizagem por Reforço.
5. Algoritmos Genéticos. I. Rodrigues, Kamila Rios
da Hora, orient. II. Título.

Bibliotecários responsáveis pela estrutura de catalogação da publicação de acordo com a AACR2:
Gláucia Maria Saia Cristianini - CRB - 8/4938
Juliana de Souza Moraes - CRB - 8/6176

FOLHA DE AVALIAÇÃO OU APROVAÇÃO

Elemento obrigatório. Deixe esta folha em branco, pois a folha de aprovação será entregue no dia da defesa.

RESUMO

PRADA, G. C. **Analisando uma população de agentes submetidos a neuro-evolução em um jogo digital**. 2023. 112 f. Trabalho de conclusão de curso (MBA em Ciência de Dados) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2020.

Algoritmos evolutivos têm o potencial para serem aplicados no controle de agentes em jogos digitais, mas como não são programados explicitamente e exibem comportamento variável ao longo das gerações, seu correto funcionamento e adequação não podem ser verificados caso a caso. Este trabalho teve como objetivo desenvolver uma infraestrutura neuro-evolutiva para controlar agentes de um jogo digital e elaborar uma metodologia para análise do desempenho desses agentes e do sistema neuro-evolutivo. A solução adotada consistiu em coletar dados dos eventos de jogo durante ensaios e sintetizá-los em relatórios gráficos e textuais para posterior análise. Aplicando essa metodologia, foi possível encontrar erros na infraestrutura desenvolvida e verificar a correção adotada, otimizar hiperparâmetros, validar as métricas de desempenho escolhidas, encontrar suas limitações, interdependências e aplicabilidade como base para cálculo do *fitness*, e finalmente foi possível avaliar como se comportam os agentes quando submetidos ao controlador desenvolvido.

Palavras-chave: Inteligência Artificial; Neuro-evolução; Aprendizado de Máquina; Aprendizagem por Reforço; Algoritmos Genéticos.

ABSTRACT

PRADA, G. C. **Analyzing a population of agents submitted to neuroevolution in a video game.** 2023. 112 f. Trabalho de conclusão de curso (MBA em Ciência de Dados) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2020.

Evolutionary algorithms have the potential to be applied in control of video game agents, but because they are not explicitly programmed and show variable behavior along generations, their correct operation and suitability can not be verified case-by-case. This research aimed to develop a neuroevolutionary infrastructure to control agents in a video game and create a methodology to analyze the performance of those agents and of the neuroevolutionary system. The adopted solution consisted of collecting data from gameplay events and synthesizing it into graphical and textual reports for later analysis. Using this methodology, it was possible to find errors in the developed infrastructure and check the adopted solution, to optimize hyperparameters, to validate the chosen performance metrics, find its limitations, interdependencies and applicability as the basis for fitness calculation, and finally it was possible to verify how those agents behave when submitted to the developed control system.

Keywords: Artificial Intelligence; Neuroevolution; Machine Learning; Reinforcement Learning; Genetic Algorithms.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxograma do algoritmo baseline_full_random.....	40
Figura 2 – Fluxograma do algoritmo baseline_valid_full_random.....	40
Figura 3 – Fluxograma do algoritmo baseline_collide_random.....	40
Figura 4 – Fluxograma do algoritmo baseline_random.....	41
Figura 5 – Fluxograma do algoritmo baseline_valid_random.....	41
Figura 6 – Fluxograma do algoritmo baseline.....	42
Figura 7 – Fluxograma do algoritmo baseline_pill.....	42
Figura 8 – Fluxograma do algoritmo baseline_pill_ghost.....	43
Figura 9 – Relatório para baseline_full_random, bateria 1.....	51
Figura 10 – Relatório para baseline_valid_full_random, bateria 1.....	52
Figura 11 – Relatório para baseline_collide_random, bateria 1.....	53
Figura 12 – Relatório para baseline_random, bateria 1.....	54
Figura 13 – Relatório para baseline_valid_random, bateria 1.....	55
Figura 14 – Relatório para baseline, bateria 1.....	56
Figura 15 – Relatório para baseline_pill, bateria 1.....	57
Figura 16 – Relatório para baseline_pill_ghost, bateria 1.....	58
Figura 17 – Relatório para baseline_pill_ghost_fixed, bateria 1.....	59
Figura 18 – Relatório para baseline_pill_ghost_fixed, bateria 1 – sistema novo.....	60
Figura 19 – b1 e movement_captures_hack_26 (1677445806).....	79
Figura 20 – b1_path_grading_hack e movement_captures_hack_26 (1677445825).....	80
Figura 21 – nb4 e movement_captures_hack_26 (1677445848)	81
Figura 22 – nb4_path_grading e movement_captures_hack_26 (1677445867)	82
Figura 23 – b1 e lifetime (1677445899)	83
Figura 24 – b1_path_grading_hack e lifetime (1677445913)	84
Figura 25 – nb4 e lifetime (1677445930)	85
Figura 26 – nb4_path_grading e lifetime (1677445953)	86
Figura 27 – b1 e movement_captures_hack_26 – NEAT(1677959359)	87
Figura 28 – b1_path_grading_hack e movement_captures_hack_26 – NEAT(1677959407)..	88
Figura 29 – nb4 e movement_captures_hack_26 – NEAT(1677959485)	89

Figura 30 – nb4_path_grading e movement_captures_hack_26 – NEAT (1677959539)	90
Figura 31 – b1 e lifetime – NEAT (1677959384)	91
Figura 32 – b1_path_grading_hack e lifetime – NEAT (1677959432).....	92
Figura 33 – nb4 e lifetime – NEAT (1677959510).....	93
Figura 34 – b4_path_grading e lifetime – NEAT (1677959569)	94
Figura 35 – b1, lifetime e duas conexões iniciais (1678204540).....	99
Figura 36 – b1_path_grading_hack, lifetime e duas conexões iniciais (1678204555).....	100
Figura 37 – nb4, lifetime e quatro conexões iniciais (1678205691).....	101
Figura 38 – b1, lifetime – nova especiação (1678278297)	104
Figura 39 – b1_path_grading_hack, lifetime e duas conexões iniciais (1678278313).....	105
Figura 40 – nb4, lifetime – nova especiação (1678278779))	106

LISTA DE TABELAS

Tabela 1 – Correlação entre updates/lifetime com lifetime e updates – baseline.....	62
Tabela 2 – Variação de updates/lifetime – baseline.....	62
Tabela 3 – Correlação para fps e métricas de tempo de vida – baseline_take_2.....	64
Tabela 4 – Variação de fps – baseline_take_2.....	64
Tabela 5 – Correlação fps e métricas derivadas de lifetime – baseline_take_2.....	64
Tabela 6 – Correlação updates/lifetime e grid_cell_changes/lifetime – baseline.....	65
Tabela 7 – Correlação entre fps e taxas de mudança de célula – baseline_take_2.....	65
Tabela 8 – Resultados baseline_full_random e baseline_valid_random.....	66
Tabela 9 – Resultados baseline_full_random e baseline_collide_random.....	67
Tabela 10 – Resultados baseline_collide_random e baseline_random.....	68
Tabela 11 – Resultados baseline_random e baseline_valid_random.....	69
Tabela 12 – Resultados baseline_valid_random e baseline.....	69
Tabela 13 – Resultados baseline e baseline_pill.....	70
Tabela 14 – Resultados baseline_pill e baseline_pill_ghost.....	71
Tabela 15 – Resultados baseline_pill e baseline_pill_ghost_fixed.....	71
Tabela 16 – Comparação de controle para mudanças no número de iterações.....	72
Tabela 17 – Capacidades e Métricas.....	73
Tabela 18 – Correlação internal_lifetime e métricas de captura – baseline_take_2.....	74
Tabela 19 – Correlação internal_lifetime e métricas de movimentação – baseline_take_2....	74
Tabela 20 – Correlação visited_count e métricas de captura – baseline_take_2.....	74

LISTA DE ABREVIATURAS E SIGLAS

ANN	–	<i>Artificial Neural Network</i> – Rede neural artificial
GA	–	<i>Genetic Algorithm</i> – Algoritmo genético
IA	–	Inteligência Artificial
NEAT	–	<i>Neuroevolution of Augmenting Topologies</i>
NPC	–	<i>Non-player character</i> – Personagem não jogador
TWEANN	–	<i>Topology and Weight Evolving Artificial Neural Network</i>
FPS	–	<i>Frames Per Second</i> – Quadros por segundo

SUMÁRIO

1 INTRODUÇÃO.....	31
1.1 Justificativa.....	31
1.2 Objetivo Geral.....	32
1.3 Objetivo Específico.....	32
1.4 Estrutura do Documento.....	32
2 REVISÃO BIBLIOGRÁFICA.....	33
2.1 Algoritmos Genéticos.....	33
2.2 Redes Neurais Artificiais.....	34
2.3 Neuro-evolução.....	34
2.3.1 Neuro-evolução de topologia variável (TWEANN).....	35
2.3.2 NEAT.....	35
3 METODOLOGIA.....	37
3.1 Ferramentas.....	38
3.2 Baseline.....	39
3.3 Indicadores de desempenho.....	43
3.4 Modos a serem testados.....	44
3.5 Funções Fitness.....	46
3.6 Ensaios.....	46
3.6.1 Ensaios baseline (bateria 1).....	46
3.6.2 Ensaios Neuro-evolutivos de topologia fixa iniciais (bateria 2).....	46
3.6.3 Ensaios NEAT e Neuro-evolutivos de topologia fixa finais (bateria 3).....	47
3.7 Resultados.....	48
3.7.1 Resultados baseline (bateria 1).....	50
3.7.2 Resultados Neuro-evolutivos de topologia fixa (bateria 2).....	75
3.7.3 Resultados NEAT e Neuro-evolutivos de topologia fixa finais (bateria 3).....	76
4 CONCLUSÕES.....	107
4.1 Conclusões baseline (bateria 1).....	107
4.2 Conclusões Neuro-evolutivo de topologia fixa (bateria 2).....	107
4.3 Conclusões NEAT e Neuro-evolutivo de topologia fixa (bateria 3).....	108
4.4 Considerações finais.....	109
4.5 Melhorias.....	110

REFERÊNCIAS.....	112
-------------------------	------------

1 INTRODUÇÃO

Técnicas neuro-evolutivas podem ser aplicadas a sistemas de inteligência artificial usados para controle de agentes em uma grande variedade de jogos digitais. Neste Trabalho desenvolveram-se algoritmos de controle neuro-evolutivos para um ambiente de jogo com regras similares ao PACMAN(Extinção 5). O comportamento desses foi analisado com uma metodologia própria, elaborada visando verificar aplicabilidade, orientar o processo de desenvolvimento e embasar a escolha do algoritmo final.

Este trabalho é continuidade do trabalho de graduação Algoritmo Genético aplicado ao comportamento de uma população de NPCs em um jogo digital, realizado pelo autor. Neste trabalho inicial, foi explorada a utilização de um algoritmo genético para otimizar os fantasminhas de uma reimplementação do jogo PACMAN(Extinção 3), que manteve as regras de comportamento para os personagens e usou, para controle, um algoritmo similar ao original.

Nesta sequência, reaproveitou-se o projeto Extinção 3 e deste melhorou-se a coleta e análise dos dados e desta vez o enfoque foi no desenvolvimento de um algoritmo de controle para o jogador, usando neuro-evolução. O personagem jogador tem regras de comportamento mais flexíveis, aumentando o espaço de busca para o algoritmo e essa alteração também possibilitou que a execução de ensaios ocorra de forma automática.

1.1 Justificativa

O uso de algoritmos evolutivos em sistemas de controle pode conferir a agentes de jogos digitais algumas características desejáveis, tais como eficácia progressiva e adaptável, variabilidade e imprevisibilidade de comportamento. Eles trazem um aspecto mais natural para a Inteligência Artificial, e possibilitam a criação de sistemas que agem de maneira mais similar a um humano e podem diminuir o tempo de desenvolvimento e teste, já que não possuem uma abundância de casos especiais(BUCKLAND, 2002, prefácio). De acordo com Mikkulainen (2006, p. 6 – tradução nossa) “aprendizado de máquina pode tornar jogos digitais mais interessantes e reduzir custos de produção”.

Os algoritmos evolutivos têm como característica intrínseca não serem programados explicitamente com o comportamento desejado, e embora isso seja uma motivação para sua aplicação, também impossibilita sua avaliação de forma usual (testando as possíveis casos) e

sua operação é mais bem observada como um fenômeno, de forma empírica. Por isso é desejável o uso de uma metodologia para a coleta de dados (produzidos em ensaios de teste, que podem ser automatizados) e análise.

1.2 Objetivo Geral

O objetivo desse trabalho foi construir um sistema neuro-evolutivo e aplicar técnicas de Ciência de Dados para analisar a progressão e as características de uma população de agentes de um jogo digital quando submetidos a esse sistema.

1.3 Objetivo Específico

- Estudar técnicas de neuro-evolução, em particular o algoritmo NEAT.
- Estudar métricas de desempenho de agentes de jogos digitais.
- Estudar técnicas para qualificação do comportamento de agentes de jogos digitais.
- Estudar técnicas para análise e interpretação de dados.

1.4 Estrutura do Documento

Este documento foi organizado nos capítulos:

- Revisão bibliográfica – Contém a base teórica das técnicas utilizadas no desenvolvimento do sistema neuro-evolutivo.
- Metodologia – Contém a descrição da forma como o sistema foi construído, suas particularidades de operação, ferramentas utilizadas e a descrição dos ensaios que foram realizados, como foram organizados e sua motivação.
- Resultados – Contém o resultado dos ensaios e análises.
- Conclusões – Contém uma análise dos resultados obtidos, da forma como foram obtidos, alinhamento com expectativas iniciais e sugestões de melhorias.

2 REVISÃO BIBLIOGRÁFICA

Neste trabalho aplicaram-se técnicas de neuro-evolução, a qual é a utilização de um algoritmo genético para evoluir uma população de redes neurais ou, sob a ótica de um algoritmo genético, a aplicação de uma rede neural para modelar soluções.

2.1 Algoritmos Genéticos

Algoritmos Genéticos, ou GA, são uma técnica de otimização bioinspirada que simula a seleção natural numa população. Eles são um processo iterativo que atua em um grupo onde cada indivíduo (fenótipo) tem suas características representadas por um genótipo. Durante a iteração é associado um *fitness* a cada fenótipo, que será usado pelo operador de seleção para escolher indivíduos que participarão da reprodução, realizada pelos operadores de *crossover* e mutação. O produto da reprodução são os indivíduos da próxima geração (PRADA, 2019, p. 8-9).

A implementação de toda infraestrutura usada pelo GA (os genes, função *fitness*, operador de seleção, operador de *crossover* e operador de mutação) são particulares de cada aplicação e para ocorrer a otimização devem ser combinados para possibilitar a melhora gradual do *fitness* médio da população a cada iteração.

O *fitness* de cada indivíduo é um valor que representa o sucesso do genótipo (BUCKLAND, 2002, p. 92). A consequência da operação do GA é evoluir a população para maximizar o *fitness* (PRADA, 2019, p. 9) e é, portanto, “essencial que esteja alinhado com o problema ao qual o algoritmo genético será aplicado” (PRADA, 2019, p. 12).

A seleção é quem determina quais indivíduos participarão da fase de reprodução. Um exemplo é o algoritmo *Roulette Wheel*, no qual cada indivíduo da população terá uma chance de ser escolhido que é proporcional ao seu *fitness* (BUCKLAND, 2002, p. 99).

O *crossover* combina dois genótipos em um. O resultado pode ser um genótipo com bons genes que resulta em um *fitness* melhor que o dos genes originais, mas também pode ser pior, diminuindo a sua chance de ser escolhido para reprodução (BUCKLAND, 2002, p. 92).

A mutação modifica aleatoriamente alguns genes do novo indivíduo, a sua função é permitir que a população escape de um máximo local e possa exibir genes com valores que não estão presentes na população (PRADA, 2019, p. 13).

Para construir o GA é necessário ser conhecida uma forma de codificar genes que representem possíveis soluções e que para estes se estabeleça um operador de *crossover* que realize o cruzamento e produza um novo gene que seja não somente válido mas também possa combinar as boas características do par de genes original. Para isto pode ser usado uma rede neural, pois essas podem ser representadas por um genótipo e existem algumas opções de *crossover*(*single-point*, *two-point*, *multiple-point* e *uniform*, por exemplo). Essa técnica é conhecida por neuro-evolução.

2.2 Redes Neurais Artificiais

As redes neurais artificiais, ou ANNs, são uma técnica de aprendizado de máquina bioinspirada que procura simular o funcionamento do cérebro biológico e sua rede interconectada de neurônios. Cada um destes, de forma simplificada, recebe sinais eletroquímicos dos outros membros da rede e de alguma maneira decide se esse sinal será propagado ou não. A intensidade do sinal propagado é sempre a mesma, a atuação do neurônio está na frequência na qual são disparados (BUCKLAND, 2002, p. 235-237).

Uma rede neural artificial é formada por neurônios artificiais, os quais são simplificações simuladas computacionalmente do neurônio biológico (BUCKLAND, 2002, p. 238). Um neurônio artificial é uma abstração, que possui conexões ponderadas de entrada e uma função de ativação. O valor de saída do neurônio é calculado aplicando a função de ativação a soma dos valores ponderados de suas conexões de entrada (BUCKLAND, 2002, p. 239).

A forma mais usual de estruturar uma ANN é conhecida por *feedforward network*. Nesta estrutura os neurônios são agrupados em camadas, sendo ao menos uma camada de saída e opcionalmente algumas camadas ocultas. Cada camada recebe como entrada os sinais dos neurônios da camada anterior até chegar na camada de entrada (BUCKLAND, 2002, p. 242). Se todos os neurônios de cada camada estão conectadas a todos os neurônios da camada anterior, a rede é chamada de *fully-connected*, ou totalmente conectada.

2.3 Neuro-evolução

A aplicação de algoritmos genéticos na otimização de redes neurais para desempenhar uma determinada tarefa é uma técnica conhecida como neuro-evolução.(STANLEY, 2002, p. 99). Em sua abordagem tradicional, a técnica consiste em escolher, inicialmente, uma

topologia totalmente conectada para a rede neural de cada indivíduo. “A evolução realiza a busca no espaço de pesos das conexões dessa topologia totalmente conectada ao permitir que indivíduos de alto desempenho se reproduzam” (STANLEY, 2002, p.100 – tradução nossa). A reprodução ocorre através da aplicação de *crossover* e mutação aos genes, que correspondem aos pesos da rede neural. “O objetivo da neuro-evolução de topologia fixa é otimizar os pesos que determinam a funcionalidade da rede” (STANLEY, 2002, p.100 – tradução nossa).

2.3.1 Neuro-evolução de topologia variável(TWEANN)

É possível estender a neuro-evolução de forma que a topologia de rede seja otimizada junto aos pesos das conexões durante o processo evolutivo. Essas técnicas são conhecidas pelo acrônimo em inglês TWEANN, que significa redes neurais artificiais que evoluem topologia e pesos. Ainda não existe consenso sobre a vantagem dessa extensão, em particular, não é claro se a topologia da rede tem influência na velocidade de treinamento quando o *backpropagation* não é utilizado (STANLEY, 2002, p.100). A melhor motivação para aplicar neuro-evolução à topologia é a economia de tempo, que seria desperdiçado por humanos tentando encontrar a topologia ideal por tentativa e erro (GRUAU *et al.*, 1996 *apud* STANLEY, 2002, p. 100).

Em sistemas neuro-evolutivos, a mesma solução pode ser representada por genótipos diferentes, este problema, conhecido como *Competing Conventions Problem*, pode levar a operação de *crossover* a produzir genótipos defeituosos. Em um sistema com topologia variável esse problema é exacerbado, pois a mesma solução pode ser representada por uma rede neural com estrutura diferente (STANLEY, 2002, p.103).

Um sistema TWEANN faz a procura por soluções com diferentes topologias adicionando novas estruturas, quando essas mutações ocorrem é normal que o fitness desses indivíduos sofra uma diminuição (STANLEY, 2002, p.104), o que pode dificultar a consolidação dessas novas estruturas na população.

2.3.2 NEAT

Neuroevolution of Augmenting Topologies, ou neuro-evolução de topologias que aumentam é uma técnica desenvolvida por Kenneth Stanley Owen e Risto Miikulainen na Universidade do Texas. Essa técnica progressivamente aumenta a complexidade dos

indivíduos, como ocorre na natureza. O aumento ocorre adicionando neurônios e conexões (BUCKLAND, 2002, p.358). Desenvolvida originalmente para aprendizado de estratégias de comportamento, a rede neural controla agentes através da seleção de ações de saída em resposta a entrada sensorial e é particularmente apropriada para uso em videogames (MIKKULAINEN, *Creating Intelligent Agents in Games*, 2006, p.7).

Essa técnica introduz dois artifícios para seu funcionamento, o cadastro de inovações e a especiação. O cadastro de inovações é necessário para o funcionamento correto do operador *crossover*. Sua função é registrar cada novo neurônio, ou conexão que seja descoberto, de forma que, caso seja redescoberto, exista uma referência única e seja possível realizar um alinhamento. A especiação, ou seja, a separação da população em subgrupos, procura possibilitar que novas espécies tenham um desenvolvimento inicial protegido. O *fitness* é calculado por espécie, criando nichos evolutivos. Isso é feito para que inovações não sejam extintas rapidamente, possibilitando que evoluam (BUCKLAND, 2002, p. 387).

A população inicial utilizada por um sistema NEAT é composta por indivíduos com topologia mínima e por isso a busca por melhores soluções ocorre sempre na menor dimensionalidade possível, resultando na diminuição do espaço de busca e leva a ganhos em desempenho (STANLEY, 2002, p.106). Por iniciar com topologia mínima e somente incorporar estrutura quando beneficiar a população, a técnica tende a convergir para soluções minimalistas (STANLEY, 2002, p.106).

Neste trabalho utilizaram-se as técnicas de neuro-evolução de topologia fixa e NEAT para evoluir sistemas de controle para o personagem jogador de uma reimplementação simplificada do jogo digital PACMAN.

3 METODOLOGIA

Este trabalho foi dividido em duas fases de coleta e análise de dados. Na primeira etapa realizaram-se ensaios mais curtos com modelos neuro-evolutivos de topologia fixa (bateria 2) e especialistas (bateria 1). Usou-se os dados produzidos pelos modelos especialistas para análise comparativa (baseline) e com isso se validou as métricas escolhidas e observaram-se como diferentes capacidades as influenciam.

A bateria 2 teve a finalidade de desenvolver e exercitar a infraestrutura utilizada e os dados produzidos foram aplicados na seleção e ajuste dos modos de operação participantes da segunda etapa, que consistiu da bateria 3 e incluiu modelos baseados na técnica NEAT de Kenneth Stanley Owen e Risto Miikulainen e neuro-evolutivos de topologia fixa, produziu os dados que geraram os relatórios para a análise final.

A técnica NEAT foi escolhida pela forma como resolve o *Competing Conventions Problem*, por incluir conexões recorrentes, por minimizar o espaço de busca e pela tendência a produzir soluções minimalistas, o que é desejável para otimizar o custo computacional.

O algoritmo NEAT utilizado foi desenvolvido para este trabalho, portanto, não é a versão original e também não é uma reimplementação funcional. Alguns aspectos são divergentes, em particular o mecanismo de nicho é diferente. O funcionamento da especiação e o cadastro de inovações serão mantidos. A implementação criada foi baseada nas listagens de código presentes no livro de Buckland com adaptações. Os resultados obtidos foram considerados pelo que são e não necessariamente podem ser estendidos para a implementação original, também vale lembrar que a finalidade dessa análise é observar a progressão da população e suas características e não julgar o mérito de cada técnica.

O mecanismo de nicho, no algoritmo original, funciona calculando quantos indivíduos de cada espécie serão incluídos na próxima geração. O algoritmo que o jogo utilizou não é baseado em gerações, sendo que um novo indivíduo é produzido quando necessário a partir de genes selecionados por *roulette wheel* de um histórico dos melhores indivíduos (por *fitness*) da população.

A operação do mecanismo de nicho implementado, utiliza uma fila, de forma que quando uma nova espécie é encontrada, são adicionadas ao final desta, uma quantidade (tamanho do nicho) de entradas correspondentes. Quando um novo indivíduo for necessário e houver elementos nessa fila, o sistema os retirará, um por vez e criará um indivíduo da espécie correspondente. Essa fila garante a cada nova espécie a oportunidade de um

desenvolvimento protegido, antes que seja retomada a operação normal (seleção do gene mãe por *roulette wheel* do histórico geral, e do gene pai por *roulette wheel* do histórico da espécie da mãe). Quando não houver mais indivíduos da espécie no histórico geral, esta é extinta.

Os dados coletados são compostos por informações de eventos de criação e destruição dos atores do jogo que ocorreram durante a execução de cada ensaio. Estes foram consolidados em arquivos-texto, onde cada linha representa um evento e os atributos são separados por vírgulas. Algumas contagens são totalizadas durante a execução do jogo, e outras são obtidas processando esses eventos.

Quando o ensaio é iniciado, o sistema cria dentro da pasta *logs*, um arquivo com o nome da semente usada para inicializar o gerador de números randômicos e a extensão *conf*. Este arquivo contém as configurações usadas para a execução do ensaio, caso a semente já tenha sido usada, e os arquivos ainda estiverem na pasta, esses serão sobrescritos.

Os eventos são salvos com o mesmo nome, em arquivos com a extensão *data-**. O asterisco é um número, que iniciando do 0, é a ordem temporal em que os arquivos foram preenchidos. Caso uma semente seja reutilizada, os arquivos *data-** também podem ser sobrescritos, com o agravante de não serem previamente apagados, podendo eventualmente produzir inconsistências.

Cada ensaio consistiu em uma execução automatizada do jogo. Esta foi interrompida quando a quantidade de NPCs de interesse atingir a marca pré-determinada de 35.000 para os modelos especialistas ou, no caso dos modelos neuro-evolutivos, quando foi possível determinar a viabilidade (ou inviabilidade) do modelo.

3.1 Ferramentas

O jogo utilizado no ensaio foi baseado no código produzido no trabalho Algoritmo Genético aplicado ao comportamento de uma população de NPCs em um jogo digital (PRADA, [Extinção 3](#)), que foi desenvolvido utilizando o *framework* para jogos [Love2d](#), em linguagem [Lua](#). As principais modificações foram a alteração do número de agentes, a diminuição da área do jogo, a adição dos algoritmos de controle neuro-evolutivos e a alteração do sistema de coleta de dados.

A análise dos dados foi feita no ambiente [Jupyter Notebook](#), em linguagem [Python](#) e utilizando as bibliotecas [Pandas](#), [Matplotlib](#) e [SciPy](#).

Os arquivos de código, incluindo os *Notebooks* usados na análise e processamento dos dados e os dados consolidados, foram disponibilizados em [Extinção 5](#).

Os ensaios foram realizados em uma estação de trabalho com sistema operacional Windows 11. A máquina, equipada com um processador AMD Ryzen 3800X, possui 32 Gb de memória RAM instalada e um acelerador gráfico AMD Radeon RX 6800XT com 16 Gb de VRAM. A máquina é de uso pessoal e sua capacidade total foi compartilhada com outras tarefas durante a realização dos ensaios.

3.2 Baseline

Os algoritmos especialistas foram elaborados para exibir algumas capacidades específicas, visando estabelecer patamares evolutivos.

Os algoritmos em ordem incremental de capacidades são:

- *baseline_full_random*, um algoritmo especialista que, assume outra rota aleatoriamente. Tem comportamento aleatório (Figura 1).

- *baseline_valid_full_random*, um algoritmo especialista que, assume outra rota válida aleatoriamente. Tem a capacidade de selecionar um caminho válido (Figura 2).

- *baseline_collide_random*, um algoritmo especialista que caso tenha ocorrido uma colisão, assume outra rota aleatoriamente. Tem a capacidade de detectar um caminho inválido (Figura 3).

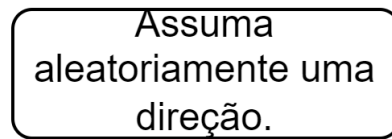
- *baseline_random*, um algoritmo especialista que foge quando vulnerável e ataca quando oportuno, caso a rota atual seja ruim, assume uma rota aleatoriamente. Tem a capacidade de ataque oportuno e evasão (Figura 4).

- *baseline_valid_random*, um algoritmo especialista que foge quando vulnerável e ataca quando oportuno, caso a rota atual seja ruim, assume outra rota válida aleatoriamente. Tem a capacidade de detectar um caminho inválido e de selecionar um caminho válido (Figura 5).

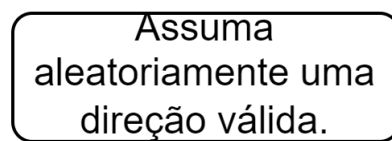
- *baseline*, um algoritmo especialista que foge quando vulnerável e ataca quando oportuno, mas não sai da rota para procurar pílulas ou fantasmas. Tem a capacidade de ataque oportuno e evasão (Figura 6).

- *baseline_pill*, um algoritmo especialista que foge quando vulnerável e ataca quando oportuno, desvia da rota para capturar pílulas. Tem a capacidade de ataque oportuno, evasão e de procurar pílulas (Figura 7).

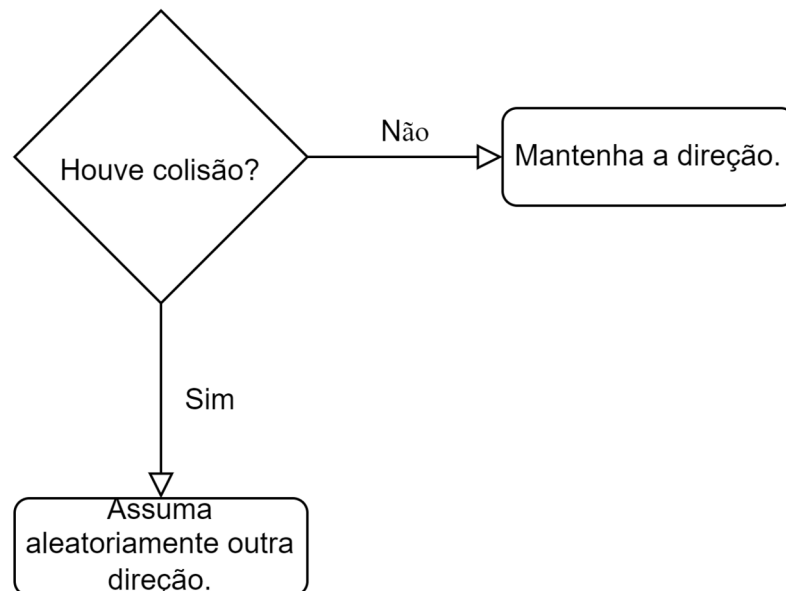
- *baseline_pill_ghost*, um algoritmo especialista que foge quando vulnerável e ataca quando oportuno, desvia da rota para capturar pílulas e fantasmas. Tem a capacidade de ataque, evasão e de procurar pílulas e fantasmas (Figura 8).

Figura 1: Fluxograma do algoritmo *baseline_full_random*

Fonte: próprio autor.

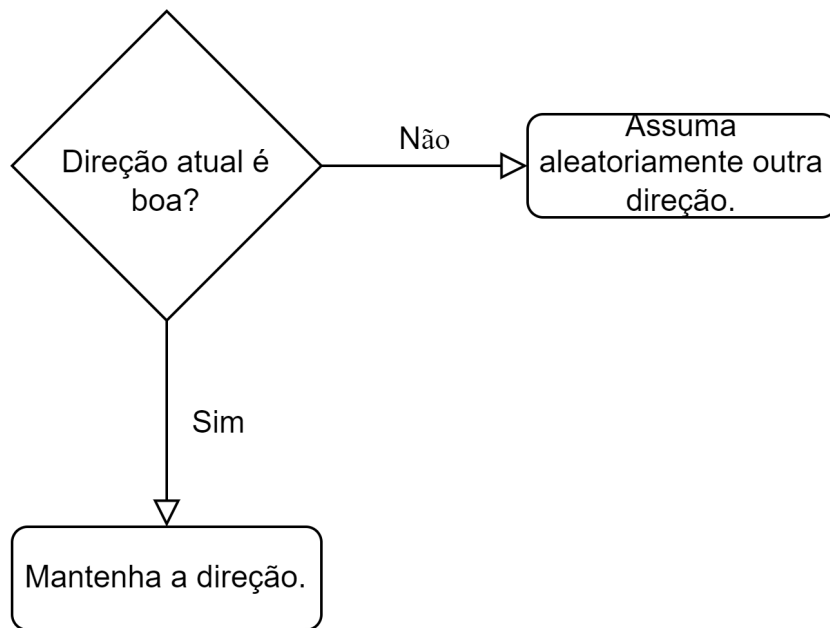
Figura 2: Fluxograma do algoritmo *baseline_valid_full_random*

Fonte: próprio autor.

Figura 3: Fluxograma do algoritmo *baseline_collide_random*

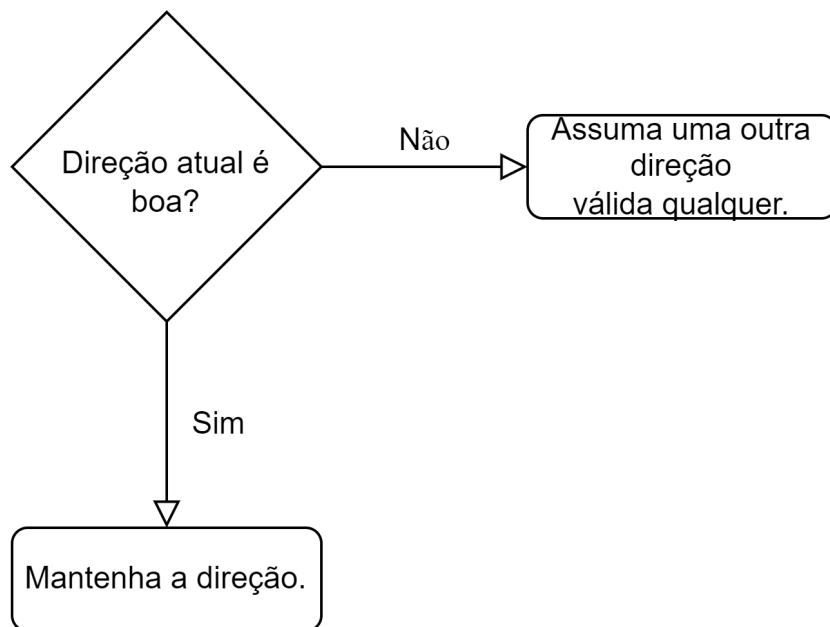
Fonte: próprio autor.

Figura 4: Fluxograma do algoritmo *baseline_random*

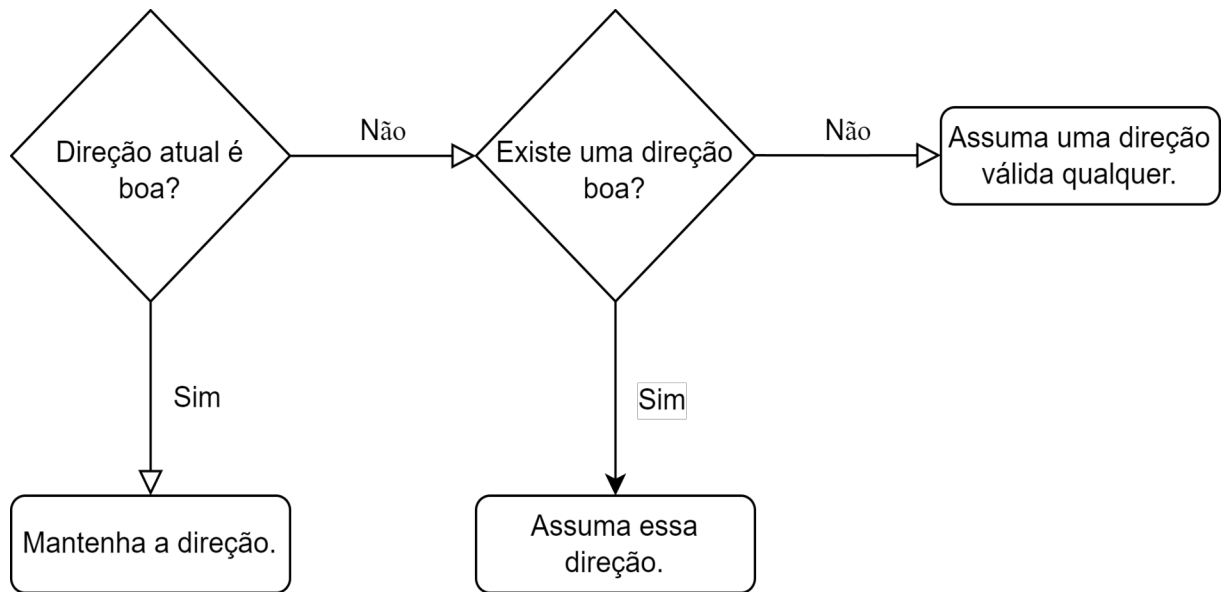


Fonte: próprio autor.

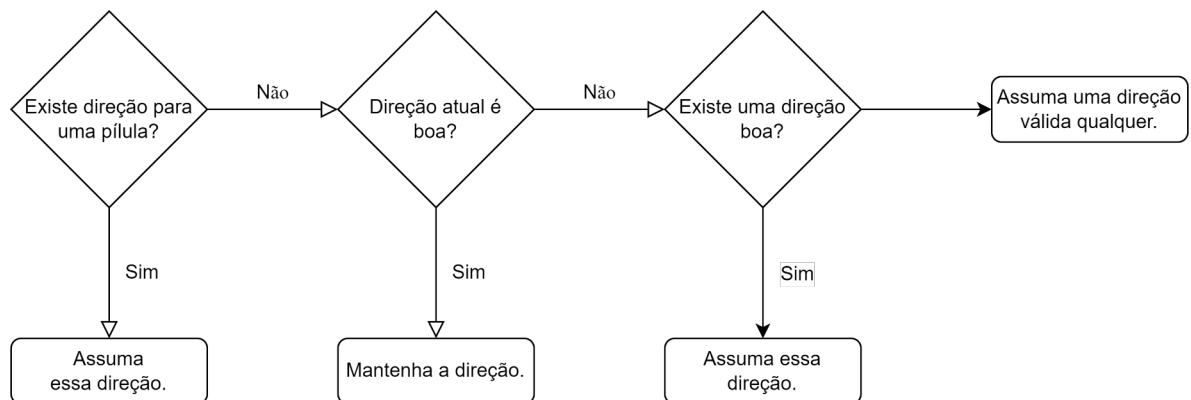
Figura 5: Fluxograma do algoritmo *baseline_valid_random*



Fonte: próprio autor.

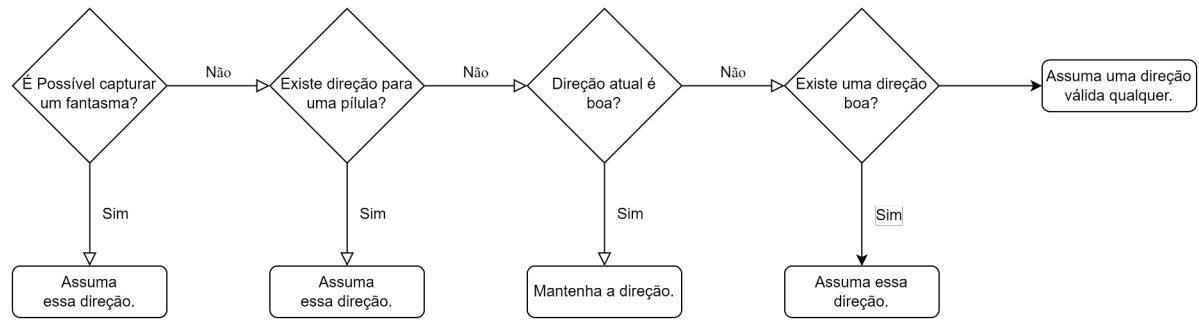
Figura 6: Fluxograma do algoritmo *baseline*

Fonte: próprio autor.

Figura 7: Fluxograma do algoritmo *baseline_pill*

Fonte: próprio autor.

Figura 8: Fluxograma do algoritmo *baseline_pill_ghost*



Fonte: próprio autor.

3.3 Indicadores de desempenho

A análise do desempenho do jogador foi feita utilizando como base as seguintes métricas:

- *grid_cell_changes* – Contador para a quantidade de vezes que o personagem mudou de célula, foi usado como um indicador de mobilidade, também referido como mudanças de célula do grid.

- *visited_count* – Contador para a quantidade de células distintas que o NPC visitou, foi usado como um indicador da efetividade da mobilidade, também referido como cobertura do grid.

- *collision_count* – Contador para a quantidade de vezes que o personagem colidiu com a parede, também referido como contador de colisões sendo utilizado como indicador de mobilidade.

- *ghosts_captured* – Calculado a partir dos eventos de destruição de fantasmas, foi usado como um indicador da capacidade de capturá-los, também referido como fantasmas capturados.

- *pills_captured* – Calculado a partir dos eventos de destruição de pílulas, foi usado como um indicador da capacidade de capturá-las, também referido como pílulas capturadas.

- *lifetime* – Calculado a partir da diferença entre o *timestamp* dos eventos de criação e destruição de jogadores, fantasmas e pílulas. A resolução desse indicador é de 1 segundo. Varia com a taxa de atualização. No texto aparece como *lifetime* para diferenciar do tempo de vida como qualidade externa ao sistema.

- *updates* – Contador para quantidade de vezes que a função de atualização do jogador foi chamada. Varia com a taxa de atualização. Foi usado como indicador do tempo de vida, também referido como contador de atualizações.

- *internal lifetime* (sistema novo) – Totalizado internamente a partir do intervalo de tempo decorrido entre atualizações do jogo. Foi adicionado na bateria 3 para ser utilizado como tempo de vida. Não varia com a taxa de atualização.

- *fps* (sistema novo) – Taxa de atualização instantânea do jogo no momento do evento. *FPS*, do inglês, frames per second, é a taxa de atualização da tela do jogo, que nesse sistema é a mesma taxa de atualização do jogo. Foi adicionado na bateria 3 para ser utilizado para monitorar a estabilidade na execução da simulação. Referido como *fps*, para diferenciar da razão *lifetime/updates* que também é tratada como taxa de atualização.

3.4 Modos a serem testados

Para o desenvolvimento dos modelos de redes neurais foram aplicadas três formas básicas.

A primeira foi elaborada para ser independente da orientação do agente. O seu resultado é uma ação, ou combinação de ações, esse tipo de sistema precisa aprender as regras de movimentação. As possibilidades são:

- Manter a orientação (*keep*).
- Rotacionar a esquerda (*rotate left*).
- Rotacionar a direita (*rotate right*).
- Inverter (*flip*).

Os modos dessa forma são:

- B1 – Uma saída binária, codifica a ação de virar a esquerda, ou não (*keep*). Não consegue assumir todas as direções em um único ciclo de atualização.

- B2 – Duas saídas binárias, codificam a ação de *flip* (meia volta) e a ação de virar a esquerda. As duas ações podem ser combinadas de forma que todas as quatro direções são possíveis.

- B3 – Três saídas binárias, codificam a ação de *flip* (meia volta), a ação de virar a esquerda e a ação de virar a direita. As três ações podem ser combinadas de forma que todas as quatro direções são possíveis.

- NB4 – Quatro saídas em ponto flutuante. Cada saída corresponde a uma nota para cada ação (manter, inverter, rotação esquerda e rotação direita). É escolhida a ação com melhor nota.

A segunda forma usa a rede neural para associar uma nota a cada uma das direções que o NPC possa assumir. Essa versão tem a vantagem de não precisar aprender as regras de movimentação do jogo e sempre produz uma saída válida (não é uma colisão).

O modo dessa forma é:

- *NB4-path-grading* – Uma saída em ponto flutuante representando uma nota, a rede neural é usada para cada um das quatro possíveis direções que sejam válidas. A saída é a direção com maior nota, com preferência para a atual, em caso de empate com a rota atual, ou com a prioridade inversa da ordem de consulta, ou seja, direita – esquerda – para baixo – para cima.

A terceira forma usa uma rede neural para classificar a rota atual em boa ou ruim. Caso seja considerada ruim, o algoritmo procura uma rota que considere boa. Esse sistema não precisa aprender as regras de movimentação, só é necessário aprender a identificar uma rota com colisão como ruim.

Os modos dessa forma são:

- *B1-path-grading-simple* – Uma saída binária representando a adequação da rota. Caso a rota seja ruim, o algoritmo vira o NPC a esquerda e até que a rota seja boa ou sejam esgotadas as quatro direções.

- *B1-path-grading-hack* – Uma saída binária representando a adequação da rota. Caso a rota seja ruim, o algoritmo testa a direção à esquerda, caso a rota ainda seja ruim o algoritmo testa a direção à direita e caso ainda seja ruim, o algoritmo retorna a ação de *flip*.

O sistema foi implementado para que cada modo possa ser testado com diferentes topologias, variando a quantidade e dimensão das camadas ocultas e a função de ativação. As configurações são feitas pelo arquivo *extinction.conf*, existente na pasta conf. Alguns exemplos, incluindo configurações utilizadas nos testes, podem ser encontradas na pasta raiz do pacote do jogo, nos arquivos, *tests_1_baseline*, *tests_3_ne* e *tests_3_neat*.

3.5 Funções *Fitness*

A métrica que se queria otimizar é o tempo de vida do jogador. Para alcançar esse objetivo os ensaios foram realizados com algumas funções de *fitness* diferentes:

- *updates* (baterias 1 e 2) – Essa função tem como valor de retorno a contagem de ciclos de *update* aos quais o NPC foi submetido. Possui uma correlação com o tempo de vida e por isso foi utilizada como base. Embora seja afetada pela taxa de atualização do jogo durante a simulação, foi escolhida, pois tem uma resolução maior que o tempo de vida.

- *lifetime* (bateria 3) – Essa função tem como valor de retorno o de tempo de vida interno (*internal_lifetime*). Foi adicionada na bateria 3 para substituir a *updates*, pois não sofre influência da taxa de atualização.

- *movement_captures_hack_26* (bateria 3) – Essa função tem como valor de retorno o resultado do cálculo (cobertura do *grid*) + (pílulas capturadas) + (fantasmas capturados) quando a contagem *cell_visited* for maior que 26 e caso contrário, retorna somente a cobertura do *grid*. Foi elaborada para privilegiar a capacidade de movimentação e captura. Não é afetada pela taxa de atualização do jogo.

3.6 Ensaios

O trabalho foi composto de 3 baterias de ensaios. A primeira para os algoritmos de *baseline*. A segunda para os testes iniciais do sistema com a neuro-evolução de topologia fixa e seleção da estrutura e modo de operação para serem testados na bateria 3. Essa, a última, conteve ensaios com a técnica NEAT e neuro-evolução de topologia fixa.

3.6.1 Ensaios *baseline* (bateria 1)

Nesta bateria foram realizados ensaios para todos os algoritmos *baseline*. O objetivo foi avaliar as métricas escolhidas e determinar a influência de cada capacidade nos indicadores de desempenho.

3.6.2 Ensaios Neuro-evolutivos de topologia fixa iniciais (bateria 2)

A configuração usada para os ensaios foi elaborada para possibilitar comparações entre o desempenho de modelos com mais camadas ocultas, com camadas ocultas com mais

neurônios e com diferentes funções de ativação para subsidiar a escolha dos modos e arquiteturas para a fase final. Todos utilizaram a função *updates* para calcular o *fitness*.

Foram utilizadas duas funções de ativação, as funções de ativação recebem o valor da soma ponderada dos sinais de um neurônio e retornam um valor que será usado como o sinal de saída. A função *Relu*, ou *rectified linear unit*, retorna o próprio valor recebido caso este seja positivo, ou 0 no caso contrário. A função *Tanh* retorna a tangente hiperbólica do valor recebido.

Para cada algoritmo foram testados modelos com as seguintes configurações:

- Nenhuma camada oculta.
- Uma camada oculta com um neurônio e ativação *Relu*.
- Uma camada oculta com três neurônios e ativação *Relu*.
- Uma camada oculta com três neurônios e ativação *Tanh*.
- Uma camada oculta com cinco neurônios e ativação *Relu*.
- Duas camadas ocultas com três neurônios e ativação *Relu*.
- Duas camadas ocultas com três neurônios e ativação *Tanh*.

3.6.3 Ensaios NEAT e Neuro-evolutivos de topologia fixa finais (bateria 3)

Esta bateria foi composta de ensaios selecionados com base no resultado da bateria 2. Essas configurações foram aplicadas a neuro-evolução de topologia fixa(modos de operação e topologia) e em seus equivalentes NEAT(modos de operação). As opções não relacionadas a ativação do NEAT foram mantidas, exceto o tamanho do histórico de indivíduos, pois tem comportamento diferente nos dois sistemas.

Algumas alterações foram feitas no sistema. Para poder executar ensaios mais livremente, sem preocupação com evitar problemas de variação na taxa de atualização, o sistema foi estendido para totalizar o tempo de vida internamente baseado no intervalo de tempo entre atualizações, eliminando a interferência da taxa de atualização nessa métrica e melhora a sua resolução. O *log* de eventos passou a trazer duas novas informações, o *internal_lifetime* e o *fps*.

O sistema foi configurado para inicializar os jogadores sem uma direção, essa alteração foi feita para que jogadores que comecem em posições melhores não sejam privilegiados e aumentar a pressão seletiva na movimentação. Devido a essa nova configuração os fantasmas tiveram duas opções ativadas, uma que troca progressivamente a

célula-alvo toda vez que ele for iniciar o retorno para a sua casa e a outra que randomiza a ordem de tentativas. Elas são necessárias para evitar que algumas células fiquem inalcançáveis, o que é causado quando um fantasma repete a mesma rota, e que pode estragar o ensaio.

Foram adicionadas duas novas funções de *fitness*, a *lifetime* e a *movement_captures_hack_26*. A primeira é uma substituição direta da função *updates*, com a vantagem de usar o *internal_lifetime* e por isso não ser afetada pela taxa de atualização. A segunda foi elaborada a partir da análise das baterias 1 e 2 o que motivou a privilegiar a movimentação no início do desenvolvimento e posteriormente a capacidade de captura.

O número 26, usado como limite de corte, é o comprimento em células de *grid* do corredor mais longo presente no jogo. Pode ser observado nos relatórios da bateria 2 como um degrau evolutivo de *visited_count*, ao ultrapassá-lo, pode-se entender que a população apresenta a capacidade de movimentação. A intenção é evitar que a capacidade de captura interfira no cálculo do *fitness* antes que a capacidade de movimentação esteja desenvolvida.

3.7 Resultados

Os resultados de cada ensaio foram sintetizados em um relatório gráfico e um arquivo de texto contendo um resumo de estatísticas. Essa síntese foi idealizada para dar visibilidade a progressão de algumas características de interesse na população durante o ensaio.

Os gráficos, na ordem que aparecem no relatório, são:

- *updates/lifetime x player iteration* – Mostra a distribuição ao longo do ensaio do valor médio de ciclos de *update* por segundo durante a vida de cada jogador. Esse gráfico permite analisar a estabilidade da taxa de atualização do jogo durante o ensaio.

- *updates x player iteration* – Mostra a distribuição ao longo do ensaio do total, para cada jogador, de ciclos de *update*.

- *Capture heatmap for player* – Mostra o *heatmap* para as posições de captura dos jogadores.

- *lifetime x player iteration* – Mostra a distribuição ao longo do ensaio do tempo de vida, por jogador.

- *Capture heatmap for ghosts* – Mostra o *heatmap* para as posições de captura dos fantasmas.

- *lifetime x ghost iteration* – Mostra a distribuição ao longo do ensaio do tempo de vida de fantasmas.

- *Capture heatmap for pills* – Mostra o *heatmap* para as posições de captura das pílulas (e criação, já que são imóveis).

- *lifetime x pill iteration* – Mostra a distribuição ao longo do ensaio do tempo de vida de pílulas.

- *pills_captured/lifetime x player iteration* – Mostra a distribuição ao longo do ensaio da taxa de pílulas capturadas, por jogador.

- *ghosts_captured/lifetime x player iteration* – Mostra a distribuição ao longo do ensaio da taxa de captura de fantasmas, por jogador.

- *ghosts_captured/pills_captured x player iteration* (eficiência de captura) – Mostra a distribuição ao longo do ensaio da quantidade de fantasmas capturados por pílula capturada para cada jogador.

- *visited_count x player iteration* – Mostra a distribuição ao longo do ensaio da quantidade de células de *grid* diferentes visitadas por jogador. Este indicador é relacionado com a capacidade de movimentação.

- *grid_cell_changes/lifetime x player iteration* – Mostra a distribuição ao longo do ensaio da taxa de mudança de células de *grid*, para cada jogador.

- *collision_count/lifetime x player iteration* – Mostra a distribuição ao longo do ensaio da taxa de colisão contra a parede sofrida por jogador. Esse indicador é relacionado com a capacidade de movimentação, mas alguns algoritmos usam a colisão intencionalmente.

A análise textual contém um conjunto de informações resultantes de cada ensaio, foi separada em três seções, uma inicial com dados básicos sobre a contagem de entradas de cada *dataframe*, uma seção com resumo estatístico de cada conjunto de dados usado e por último os resultados para testes de correlação.

Devido ao volume de dados produzido, não foi possível disponibilizar os arquivos com os *logs* de eventos em um servidor público, mas os relatórios e os *dataframes* estão disponíveis com o pacote do projeto em [Extinção 5](#), dentro da pasta *reports*. Cada ensaio possui 4 arquivos, eles são:

- {código_do_ensaio}.conf – Configurações usadas para o ensaio.

- {código_do_ensaio}.pd – Arquivo do *dataframe* pandas com os dados processados.

- {código_do_ensaio}_plots.png – Relatório gráfico.

- {código_do_ensaio}_analysis.txt – Relatório textual.

Os *notebooks* Jupyter para processar os dados estão disponíveis no pacote, dentro da pasta *data_analysis*. Para analisar um *dataframe* e produzir os relatórios, usa-se o notebook

data_analysis.ipynb já o processamento dos dados de eventos é feita com o *data_analysis_generate_pd.ipynb*.

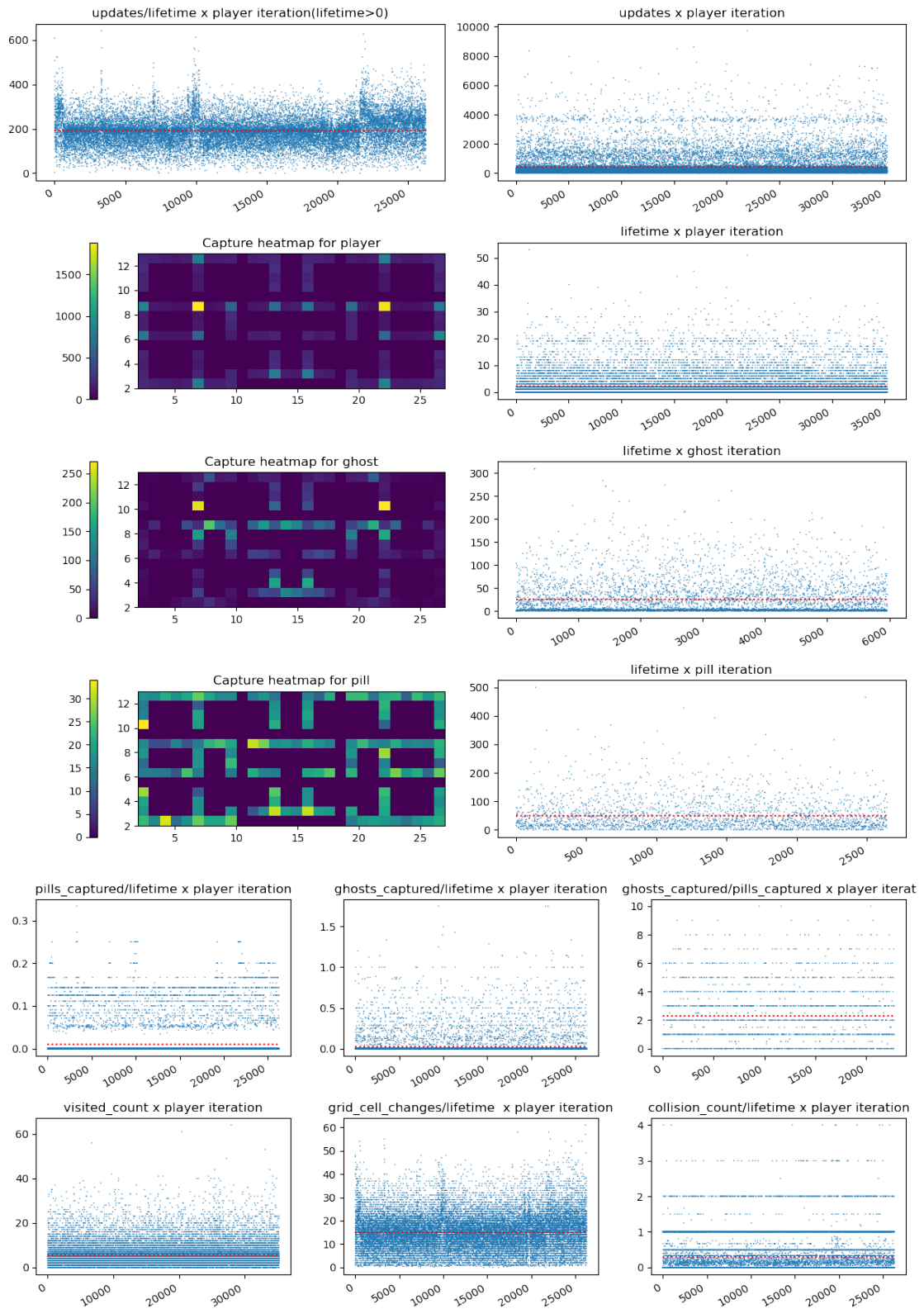
3.7.1 Resultados baseline (bateria 1)

Os resultados gráficos dessa bateria de ensaios podem ser vistos nas Figuras de 9 a 18. O conjunto desses ensaios será referido como ensaios *baseline*.

Os ensaios são:

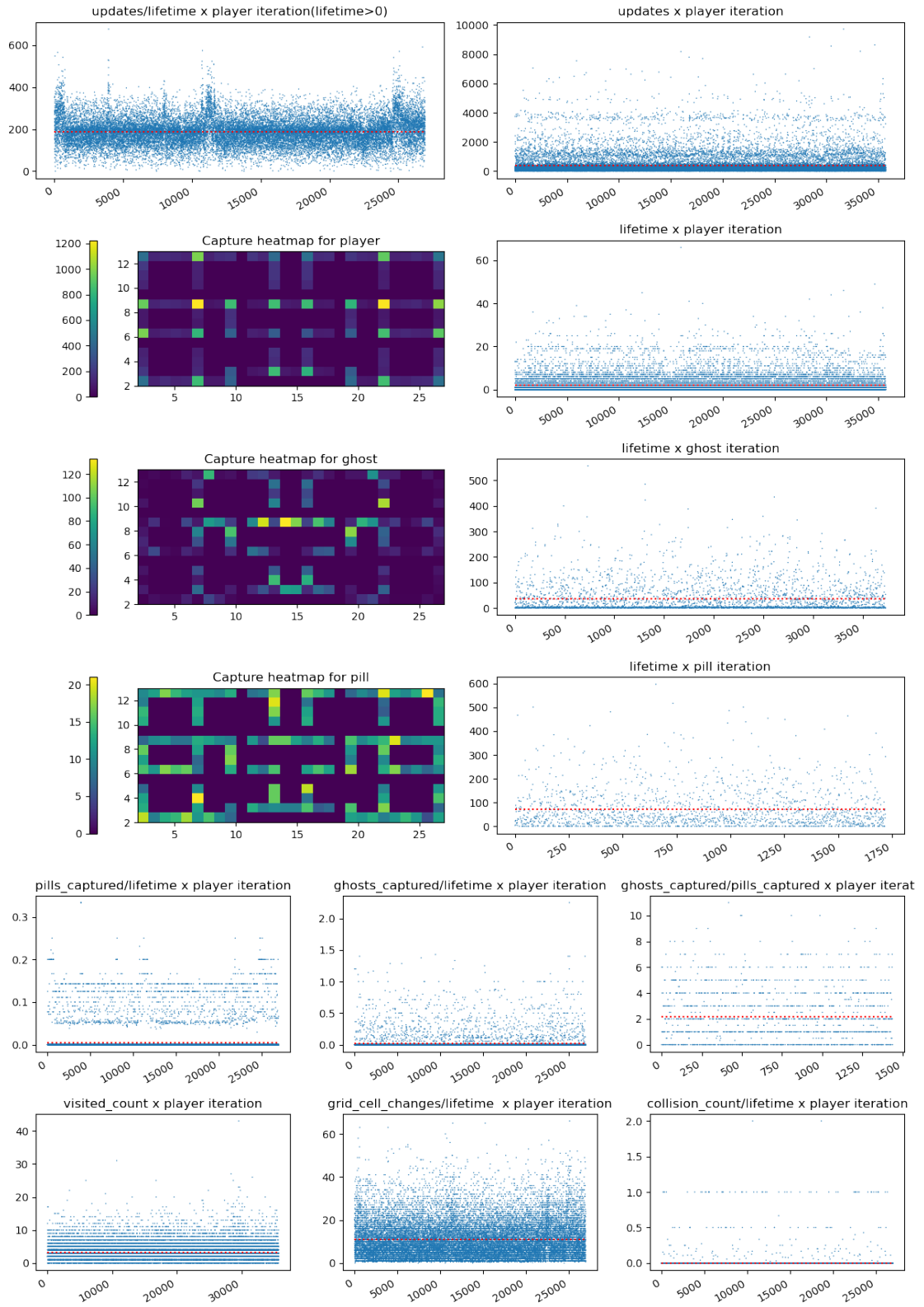
- 1675110923, utilizou o algoritmo *baseline_full_random*, Figura 9.
- 1675110956, utilizou o algoritmo *baseline_valid_full_random*, Figura 10.
- 1675110905, utilizou o algoritmo *baseline_collide_random*, Figura 11.
- 1675110970, utilizou o algoritmo *baseline_random*, Figura 12.
- 1675110942, utilizou o algoritmo *baseline_valid_random*, Figura 13.
- 1675110985, utilizou o algoritmo *baseline*, Figura 14.
- 1675111001, utilizou o algoritmo *baseline_pill*, Figura 15.
- 1675111017, utilizou o algoritmo *baseline_pill_ghost*, Figura 16.
- 1665852850, utilizou o algoritmo *baseline_pill_ghost_fixed*, Figuras 17 e 18.

Figura 9: Relatório para *baseline_full_random*, bateria 1(ensaio 1675110923)



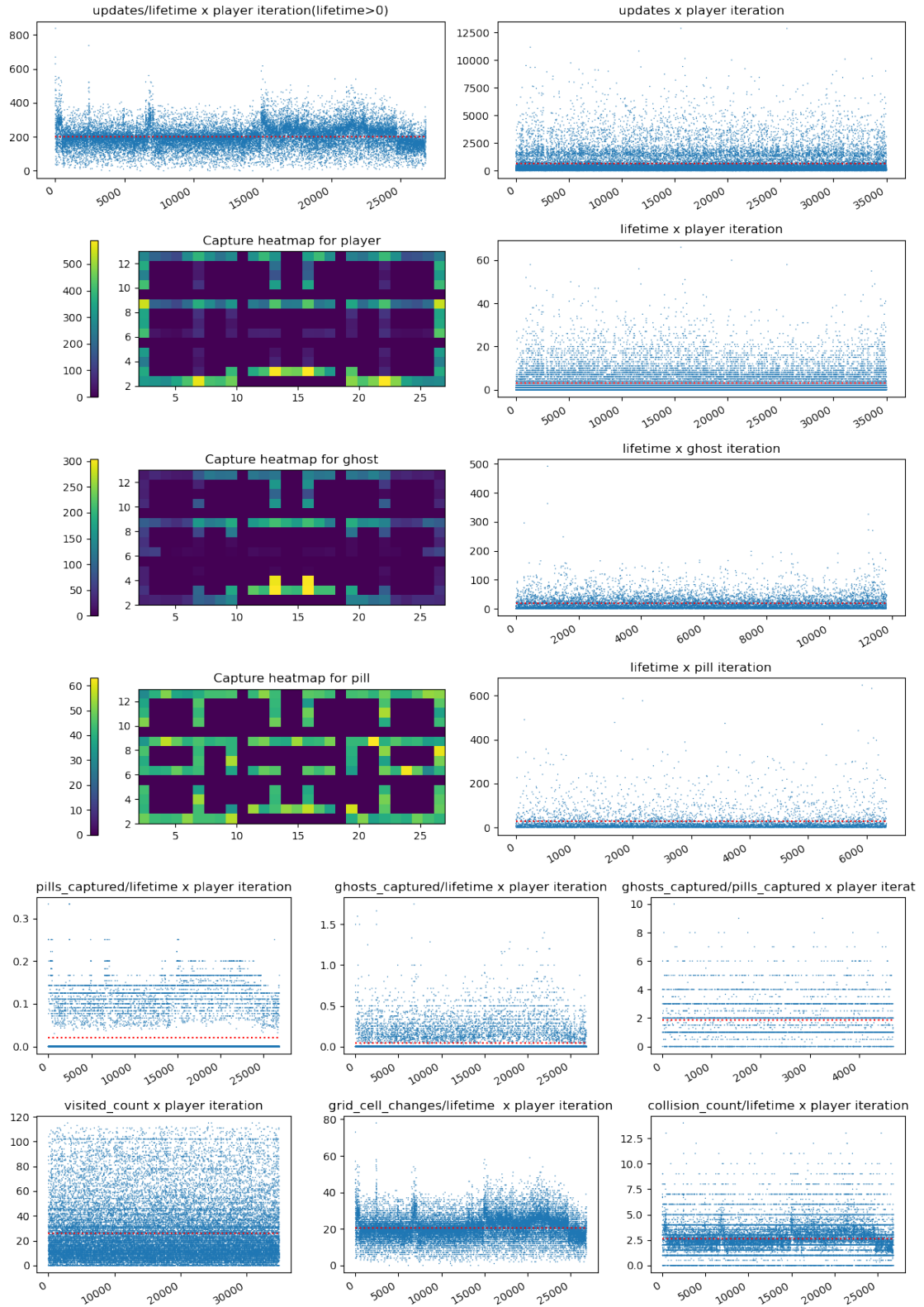
Fonte: próprio autor.

Figura 10: Relatório para *baseline_valid_full_random*, bateria 1(ensaio 1675110956)



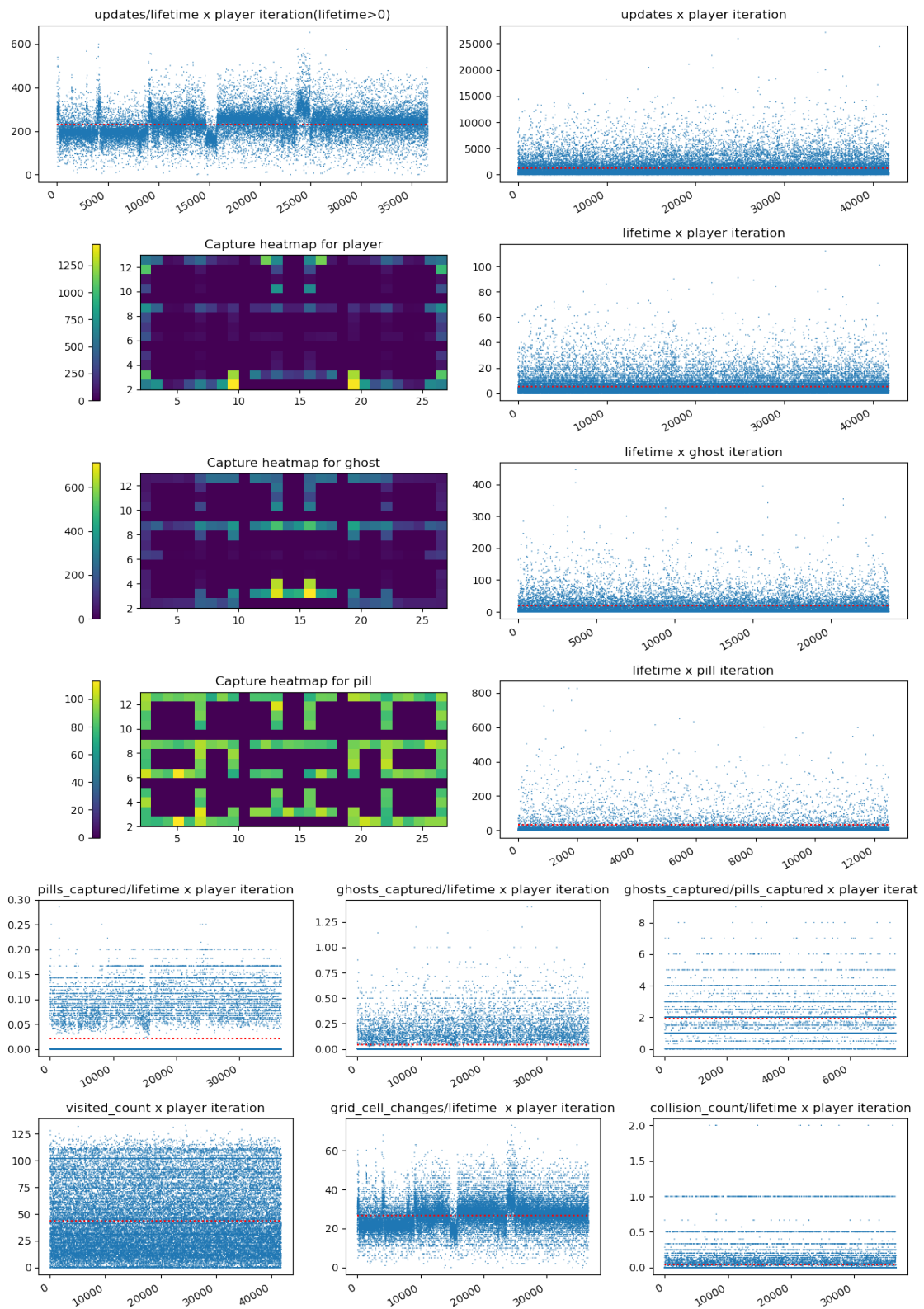
Fonte: próprio autor.

Figura 11: Relatório para *baseline_collide_random*, bateria 1(ensaio 1675110905)



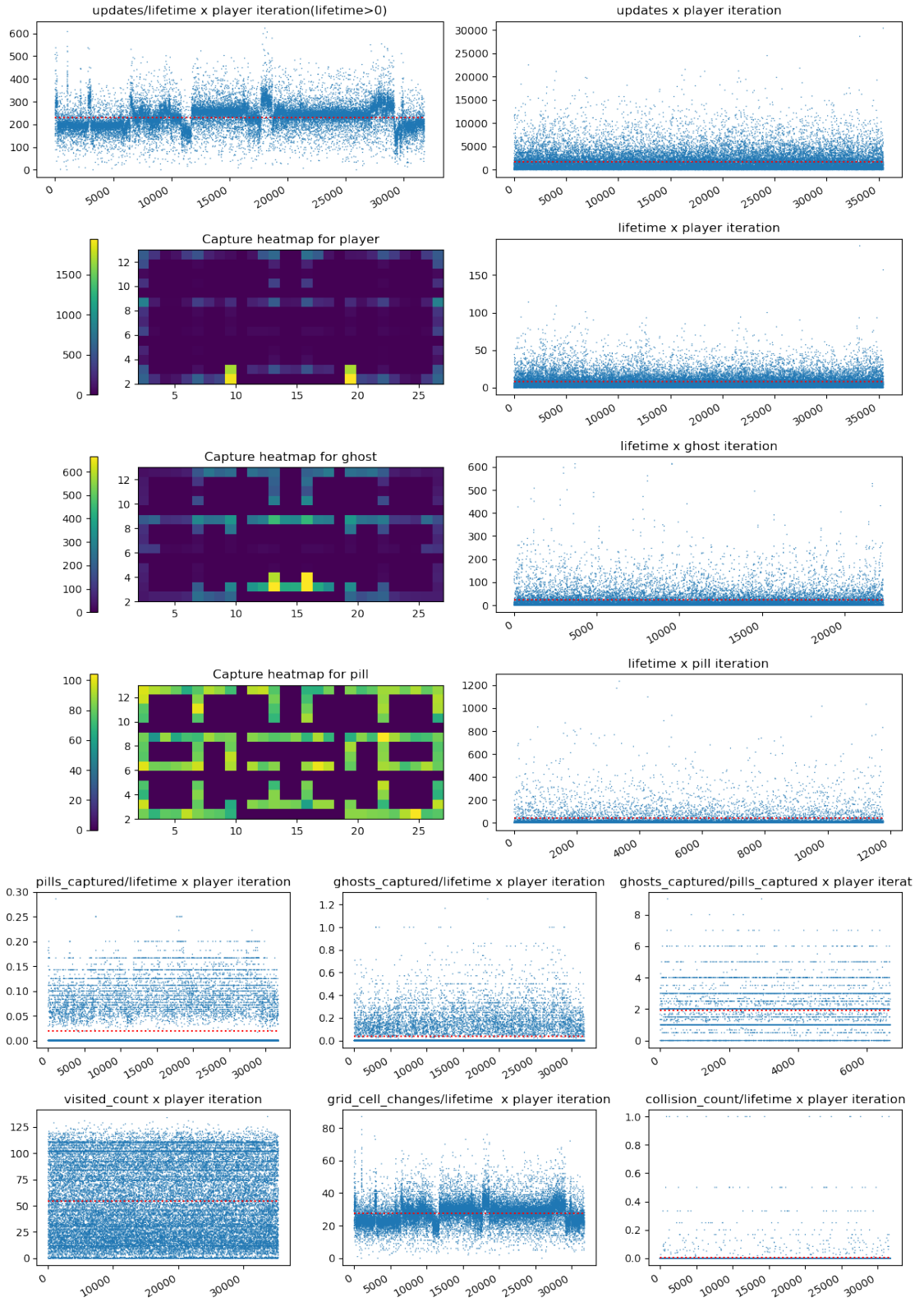
Fonte: próprio autor.

Figura 12: Relatório para *baseline_random*, bateria 1(ensaio 1675110970)



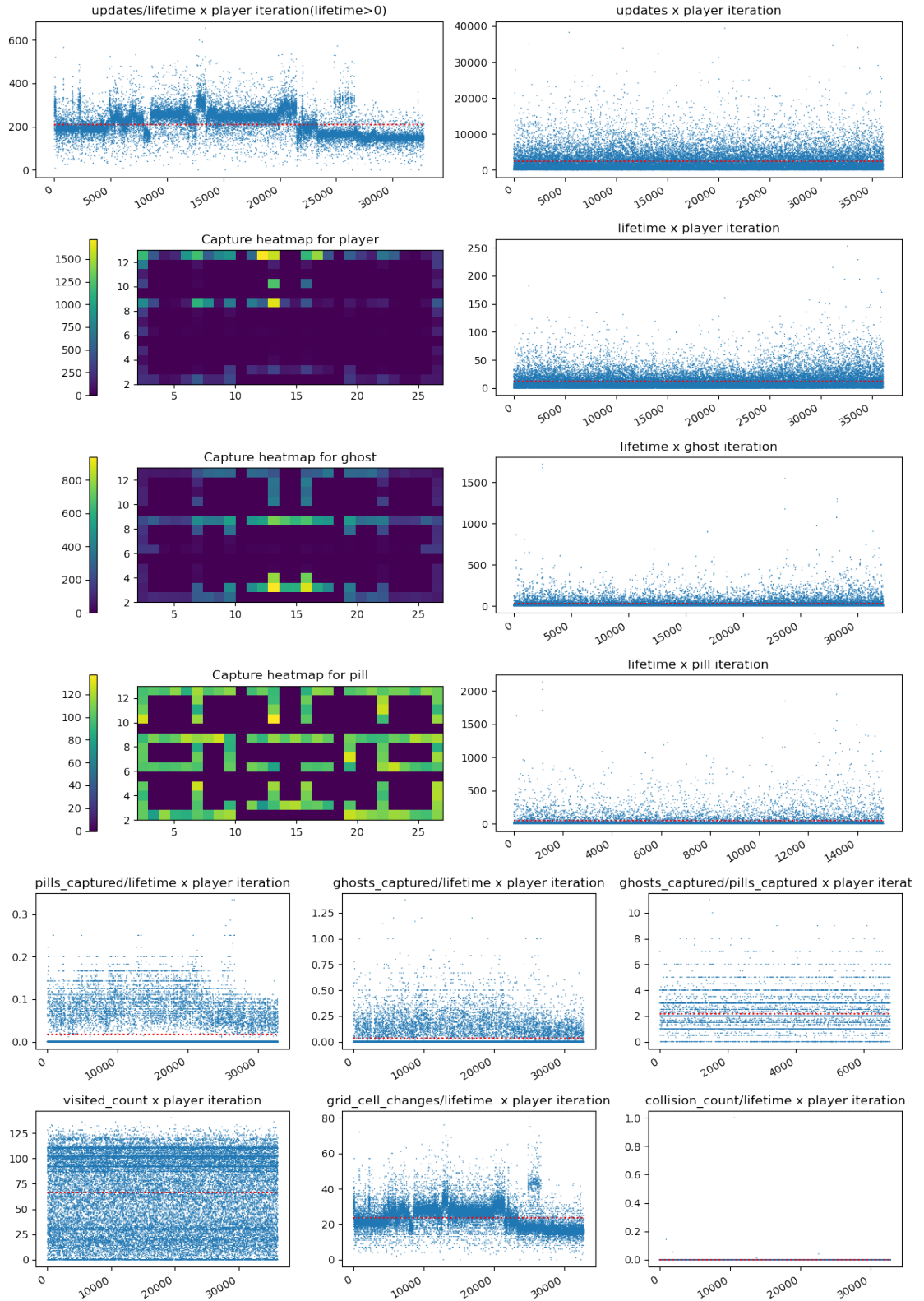
Fonte: próprio autor.

Figura 13: Relatório para *baseline_valid_random*, bateria 1(ensaio 1675110942)



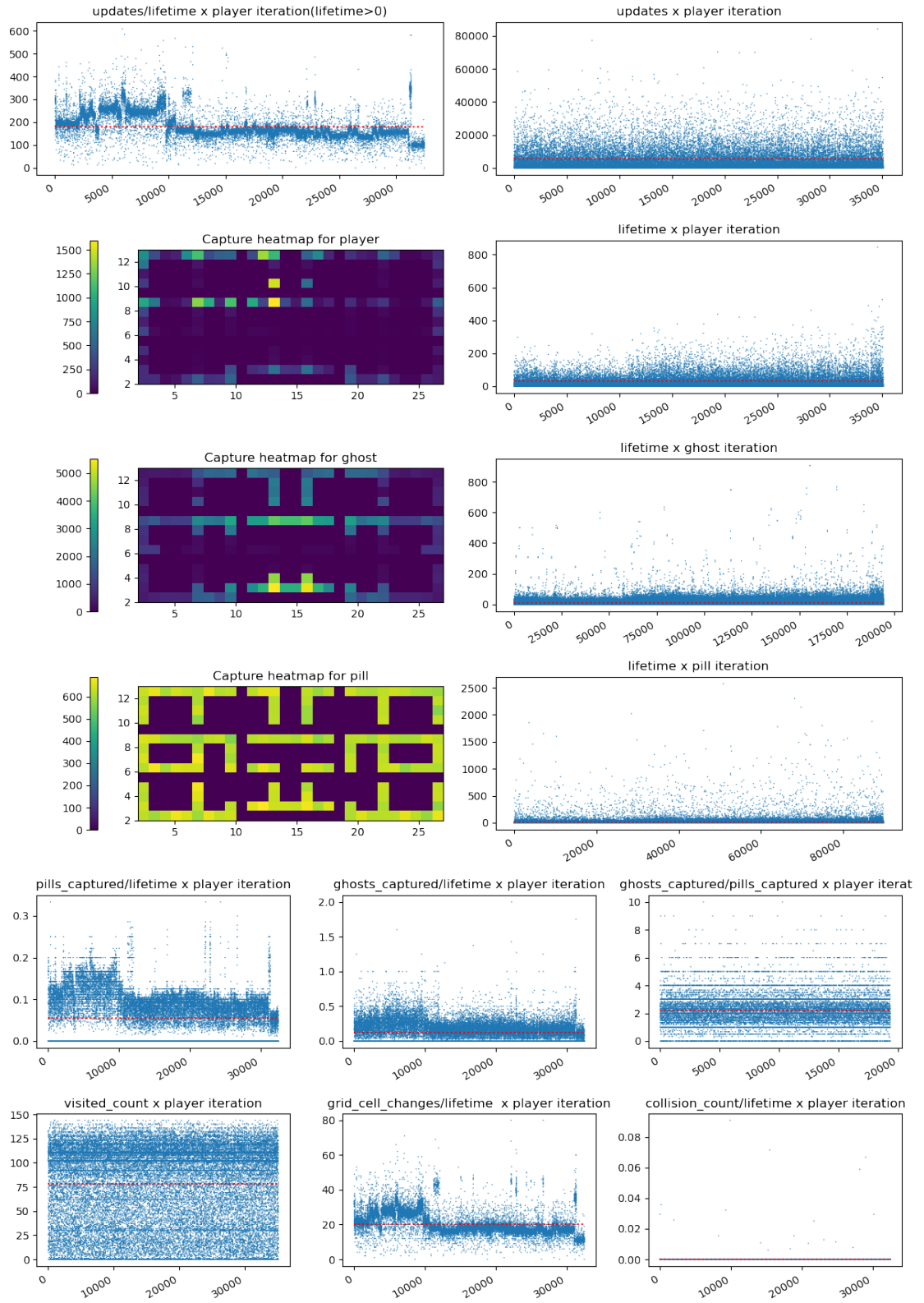
Fonte: próprio autor.

Figura 14: Relatório para *baseline*, bateria 1(ensaio 1675110985)



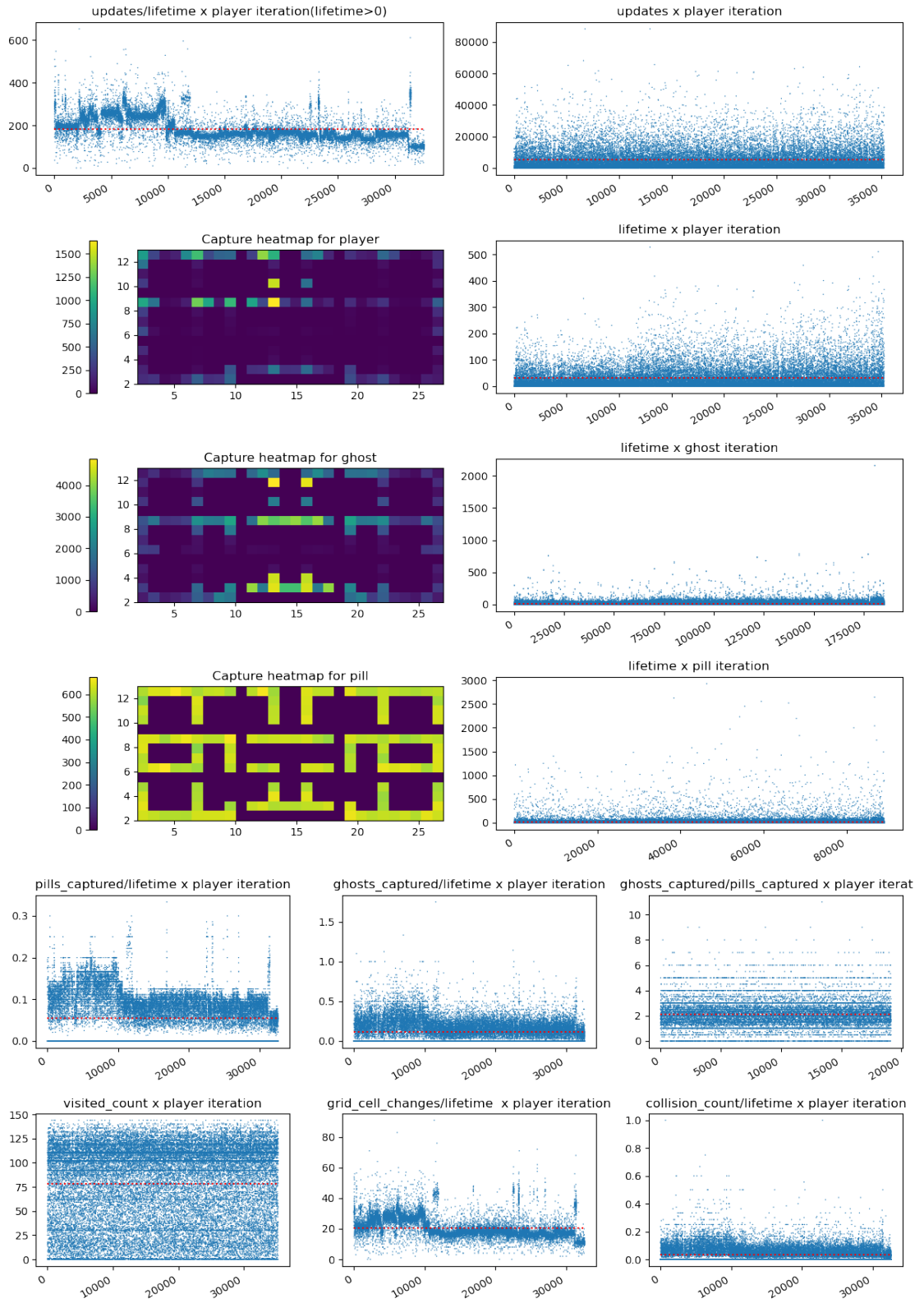
Fonte: próprio autor.

Figura 15: Relatório para *baseline_pill*, bateria 1(ensaio 1675111001)



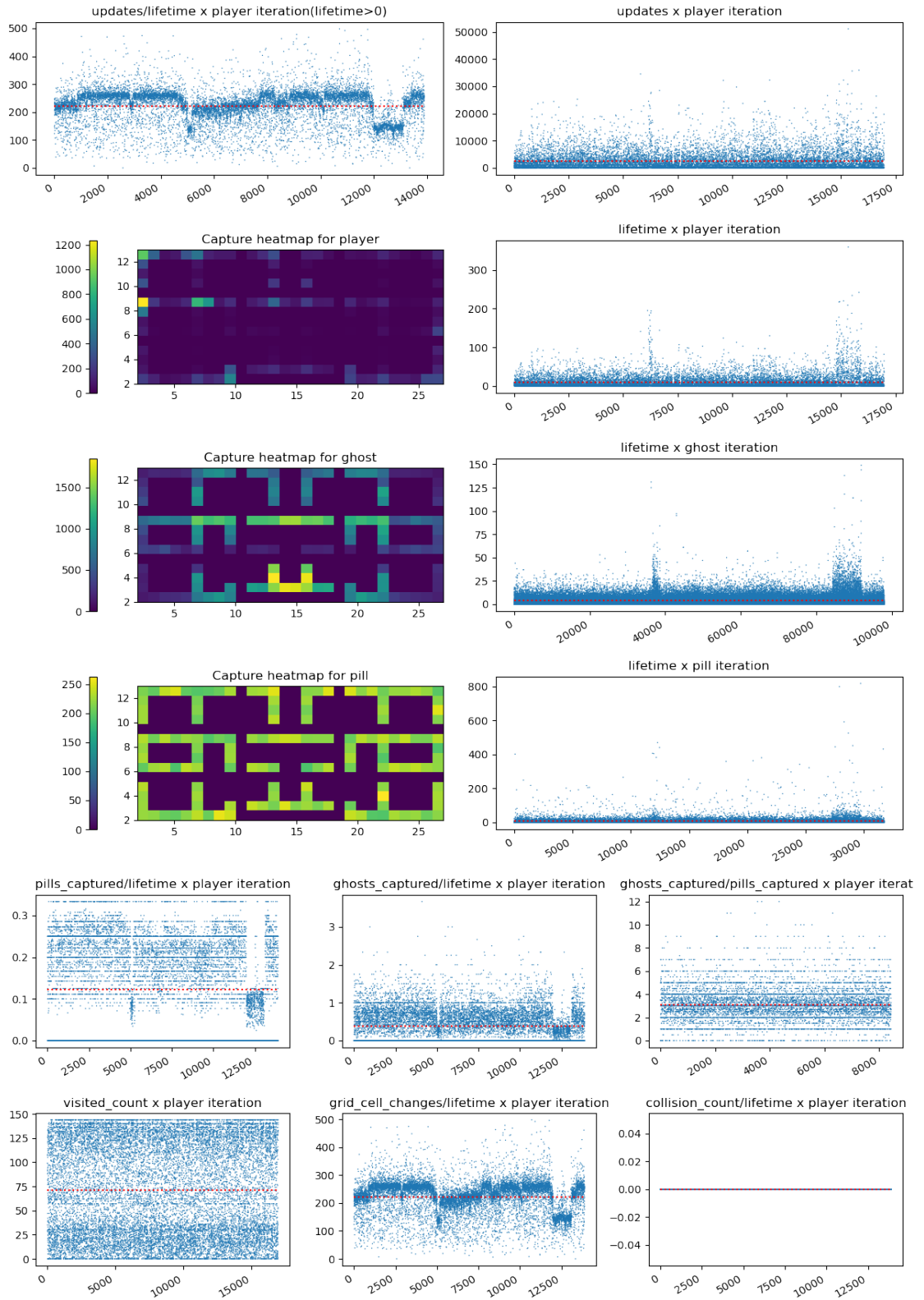
Fonte: próprio autor.

Figura 16: Relatório para *baseline_pill_ghost*, bateria 1(ensaio 1675111017)



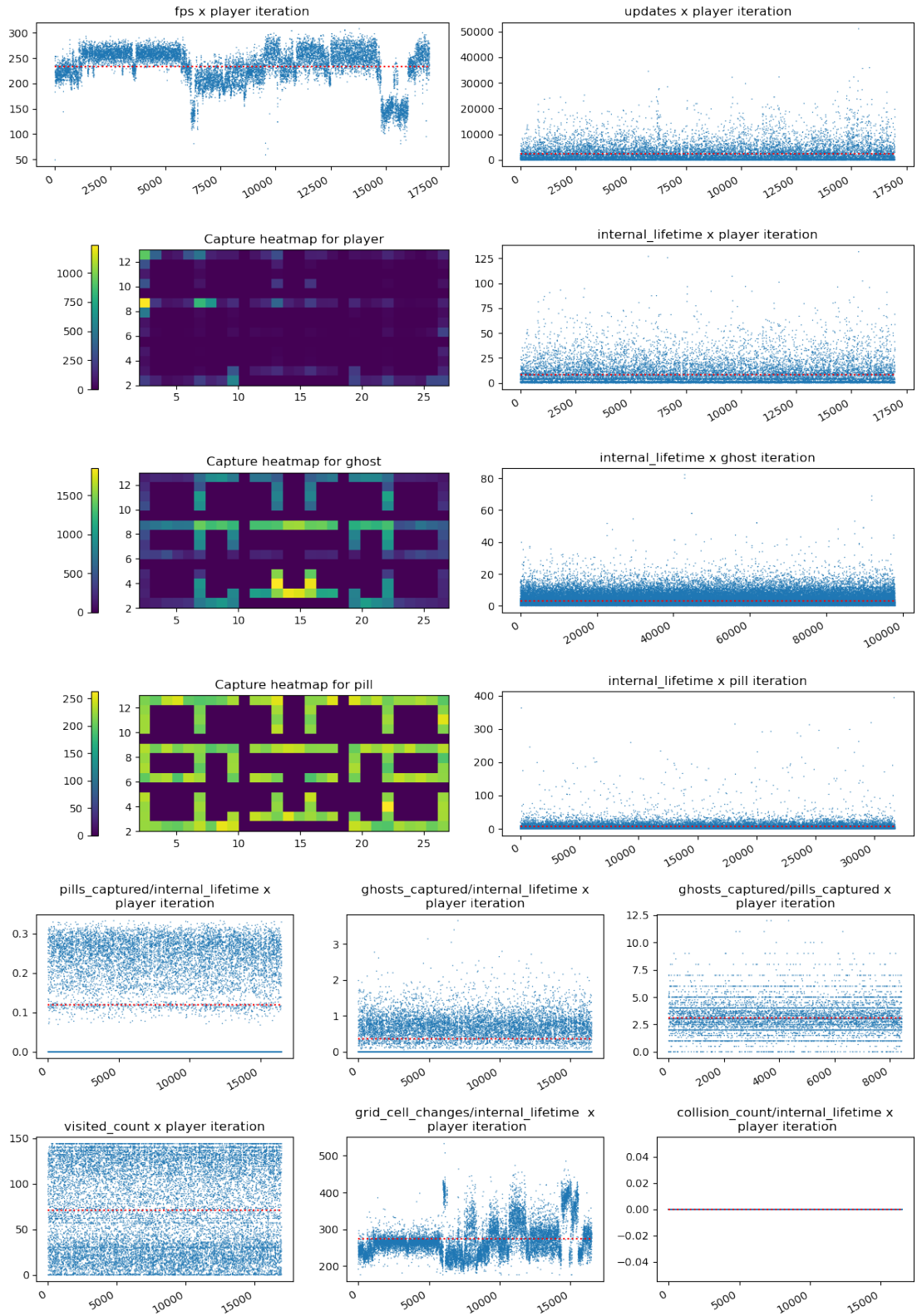
Fonte: próprio autor.

Figura 17: Relatório para *baseline_pill_ghost_fixed*, bateria 1(ensaio 1665852850)



Fonte: próprio autor.

Figura 18: Relatório para *baseline_pill_ghost_fixed*, bateria 1 – sistema novo(ensaio 1665852850)



Fonte: próprio autor.

Analisando de forma geral os relatórios gráficos resultantes dos ensaios (Figuras 9-18) pode-se observar que existe uma grande variação das métricas entre indivíduos para todos os algoritmos especialistas, tanto nos que usam alguma aleatoriedade na tomada de decisão (Figuras 9, 10, 11, 12, 13) quanto nos outros (Figuras 14, 15 e 16). Como o funcionamento de cada indivíduo foi o mesmo durante cada ensaio, conclui-se que o desempenho não depende somente do controle do indivíduo, o que pode dificultar que o GA identifique de boas soluções.

O comportamento dos jogadores permanece constante durante cada ensaio desta bateria e por isso se assume que o tempo de vida tem distribuição uniforme, sendo esperado que o *lifetime* também apresente uma distribuição uniforme, já que é uma métrica do tempo de vida. Isso não é observado nos relatórios das Figuras 14, 15 e 16. Nestes mesmos ensaio pode-se observar que existe uma aparente correlação inversa entre o *lifetime* e o *updates/lifetime*.

Observa-se ainda uma correlação aparente entre as taxas que usam a métrica *lifetime* em seu cálculo com a taxa de atualização representada por *updates/lifetime*, ou seja, *grid_cell_changes/lifetime*, *collision_count/lifetime*, *pill_captured/lifetime*, *ghosts_captured/lifetime* e *collision_count/lifetime* são afetadas. Isso pode ser explicado pelo uso do *lifetime* no cálculo dessas métricas e a correlação entre essa e *updates/lifetime*.

O sistema foi elaborado para manter o *lifetime* independente de variações na taxa de atualização, mas é possível observar no gráfico que os resultados são afetados e que a correlação aparente também estende-se para o *lifetime* de fantasmas e pílulas (note que as iterações têm contagem independente, e o eixo das abscissas não são sincronizados no tempo para tipos de agentes distintos, mas tendem a se aproximar, para agentes com comportamento constante, já que o tempo de vida tende a uma distribuição uniforme).

O *update/lifetime* é um indicador da taxa de atualização do jogo e a correlação indica que esta afeta os tempos de vida dos agentes do jogo. Também é notável que não seja possível observar alterações na distribuição da contagem de updates, pois esta deveria ser alterada já que é a contagem dos *updates*.

Existem mais mecanismos que podem estar relacionados com essa observação, como a diminuição da eficiência do algoritmo dos agentes quando a taxa de atualização diminui e a baixa resolução usada na aferição dos *timestamps* utilizados para calcular o *lifetime*.

Visando analisar a influência da taxa de atualização do jogo nos resultados obtidos, foi feita uma análise usando a correlação de Spearman entre os pares de métricas *updates/lifetime*

– *lifetime* e *updates/lifetime* – *updates*. Os valores obtidos estão na Tabela 1. Os ensaios nos quais se observou a correlação inversa, estão marcados com um asterisco.

Tabela 1 – Correlação entre *updates/lifetime* com *lifetime* e *updates* – *baseline*

Ensaio	Correlação de Spearman updates/lifetime e lifetime	Correlação de Spearman updates/lifetime e updates
baseline_full_random	0,07707568169548283	0,5682367498090364
baseline_valid_full_random	0,026663389225462696	0,5579836921499078
baseline_collide_random	0,10544569193623475	0,46480596427389187
baseline_random	0,024213386065655863	0,25120806234689336
baseline_valid_random	0,02901008881143561	0,21280476956721053
baseline*	-0,047789363729892835	0,1550853745053004
baseline_pill*	-0,06696614170108994	0,10502578367964012
baseline_pill_ghost*	-0,0830502996742866	0,08690282196384501
baseline_pill_ghost_fixed*	-0,27210346738437524	-0,13241049299742655

Fonte: próprio autor.

Apesar de a análise gráfica sugerir uma correlação inversa entre *updates/lifetime* e *lifetime* os dados da Tabela 1 indicam que esta é bem fraca. O mais notável é que a correlação só se apresenta negativa nos três ensaios em é graficamente aparente e também há variação na correlação entre *updates/lifetime* e *updates*, o que não deveria ocorrer e que sugere que em alguns ensaios essas métricas perderam a proporcionalidade.

Comparando os dados das Tabelas 1 e 2 nota-se que variações na taxa de atualização acompanham alterações na forma como essas métricas se relacionam.

Tabela 2 – Variação de *updates/lifetime* – *baseline*

Ensaio	Média	Desvio padrão
baseline_full_random	190,391732	66,761938
baseline_valid_full_random	188,140596	65,768339
baseline_collide_random	198,766864	67,795082
baseline_random	230,325488	62,607303
baseline_valid_random	230,004556	58,101294
baseline*	211,101025	61,337245
baseline_pill*	180,004096	55,503697
baseline_pill_ghost*	180,504786	55,094620
baseline_pill_ghost_fixed*	220,400696	58,519508

Fonte: próprio autor.

O jogo limita a tempo decorrido entre atualizações artificialmente caso a distância percorrida pelo agente for maior que $\frac{1}{4}$ de célula de *grid*. Isso é um mecanismo de proteção para que a física do jogo funcione corretamente e desacopla o tempo da máquina (externo) do tempo percebido pelos agentes e faz com que esses ganhem a diferença como tempo de vida extra. A possibilidade desse problema era prevista, mas foi inicialmente considerada pouco

importante, mas como pode ser observado, acabou afetando outras métricas que usam o *lifetime* no seu cálculo.

Essa interferência causada pela variação na taxa de atualização terá que ser considerada na análise gráfica dos ensaios de *baseline* e da bateria 2. Neste tipo de análise é possível identificar faixas afetadas e considerar a variação na interpretação, porém no caso do *baseline* o resultado gráfico será comparado ao relatório textual, que contem análises numéricas e embora seja possível identificar graficamente a faixa afetada e eliminá-la dos resultados, ou fazer algum tipo de correção, optou-se por usar uma segunda bateria de testes, que seja isenta dessa interferência, como controle.

A bateria de controle (*baseline_take_2*) foi composta dos mesmos testes do *baseline* e estes tiveram pelo menos 1000 iterações. Foram executados com o sistema desenvolvido para a bateria 3, o qual, não tem as métricas de interesse afetadas pela taxa de atualização, já que totaliza o mesmo intervalo de tempo usado para calcular a distância percorrida pelo jogador, o que também torna os resultados entre baterias comparáveis, mesmo que tenham sido executadas com diferentes taxas de atualização.

No novo sistema a métrica para medição da taxa de atualização é o *fps* e a do tempo de vida é o *internal_lifetime*. Estes resultados aparecem entre parênteses após seus análogos de *baseline* nas tabelas pertinentes. As escalas são as mesmas do sistema antigo, mas como são calculadas de outra forma, apresentam alguma diferença de escala, porém (desconsiderando o efeito da variação da taxa de atualização e variações amostrais entre ensaios) mantém uma proporção.

Os *dataframes* com os resultados destes testes e os relatórios podem ser encontrados no pacote do projeto, nas pastas *reports/battery_1_baseline_take_2*.

Na última análise comparativa entre capacidades foi identificado um erro na implementação do algoritmo *baseline_pill_ghost* e os testes foram refeitos usando o sistema novo e por pouco mais de 15000 iterações. Esse ensaio possibilitou realizar uma comparação direta entre o sistema de métricas novo e o antigo em um ensaio mais extenso.

Comparando as Figuras 17 e 18 é possível observar que as métricas *internal_lifetime*, *ghosts_captured_lifetime* e *pills_captured_lifetime* não possuem correlação aparente com o *fps* no novo sistema como se observa no antigo. Este segundo ensaio teve variações significativas de taxa de atualização e a análise gráfica corrobora os dados das Tabelas 3, 4 e 5.

Tabela 3 – Correlação para *fps* e métricas de tempo de vida– *baseline_take_2*

Ensaio	Correlação de Spearman fps e internal lifetime	Correlação de Spearman fps e updates
baseline_full_random	0.001624045859067844	0.001506541327979438
baseline_valid_full_random	0.026853137753150236	0.0977217348182223
baseline_collide_random	-0.06458876542196358	-0.20174317871500944
baseline_random	0.06833112056984754	0.12172925095501623
baseline_valid_random	0.01687236536259582	0.012111720758983955
baseline	0.0016316897041042265	0.06138095609703784
baseline_pill	-0.006090562877869611	0.06507518744615777
baseline_pill_ghost	-0.011634776799484669	0.024246948103169068
baseline_pill_ghost_fixed	0.01230071543698303	0.025488276851246404

Fonte: próprio autor.

Tabela 4 – Variação de *fps* – *baseline_take_2*

Ensaio	Média	Desvio padrão
baseline_full_random	139.804348	15.926703
baseline_valid_full_random	202.697378	16.777129
baseline_collide_random	180.921393	39.158285
baseline_random	203.413163	16.812601
baseline_valid_random	196.750000	28.997808
baseline	207.981354	18.269126
baseline_pill	213.560621	20.619153
baseline_pill_ghost	224.110128	17.919375
baseline_pill_ghost_fixed	234.056769	35.845217

Fonte: próprio autor.

Tabela 5 – Correlação *fps* e métricas derivadas de *lifetime* – *baseline_take_2*

Ensaio	Correlação de Spearman pills_captured/ internal lifetime	Correlação de Spearman ghosts_captured/ internal lifetime	Correlação de Spearman collision_count/ internal lifetime
baseline_full_random	-0.0104265853657289	-0.0195645762187801	0.04132391062443848
baseline_valid_full_random	0.02187956597418419	0.03699598813793709	NaN
baseline_collide_random	-0.0956390599023680	-0.0967340792305936	-0.0190506056586611
baseline_random	0.03495327486290665	0.02751175613655255	-0.0174011878416826
baseline_valid_random	0.00544765668935789	0.00265749707696768	NaN
baseline	-0.0074938839341223	-0.0076836702869472	NaN
baseline_pill	-0.0305642514507802	-0.0249708804926025	NaN
baseline_pill_ghost_fixed	0.02040946385278938	0.01549155290062675	NaN

Fonte: próprio autor.

Na Tabela 5 é possível observar que o *collision_count/internal_lifetime* também não é afetado no sistema novo, o que não ocorreu com *grid_cell_changes/internal_lifetime* (Tabelas 6 e 7). Conclui-se que a modificação do sistema atinge o efeito desejado de eliminar a influência da taxa de atualização, exceto para *grid_cell_changes/internal_lifetime*.

Tabela 6 – Correlação *updates/lifetime* e *grid_cell_changes/lifetime* – *baseline*

Ensaio	Correlação de Spearman <i>updates/lifetime</i> e <i>grid_cell_changes/lifetime</i>
baseline_full_random	0.5596552486714614
baseline_valid_full_random	0.3397862715315719
baseline_collide_random	0.9830396921965107
baseline_random	0.9632918762069108
baseline_valid_random	0.9331097156243796
baseline	0.9674129844688635
baseline_pill	0.9593051551134758
baseline_pill_ghost	0.9622916151618717

Fonte: próprio autor.

Tabela 7 – Correlação *fps* e taxas de mudança de célula – *baseline_take_2*

Ensaio	Correlação de Spearman <i>grid_cell_changes/lifetime</i>	Correlação de Spearman <i>grid_cell_changes/internal lifetime</i>
baseline_full_random	0.13409196756112968	0.1826001373777052
baseline_valid_full_random	0.1347485508776208	0.6338545787562494
baseline_collide_random	0.32210498229499707	-0.43423938041922744
baseline_random	0.20272267408674222	0.6376897215499308
baseline_valid_random	0.38101634030662457	0.32177212640675007
baseline	0.17705301674568819	0.622683527725999
baseline_pill	0.4322661395706399	0.6988328016131626
baseline_pill_ghost_fixed	0.5565021219597373	0.29272315808123767

Fonte: próprio autor.

Nota-se que as métricas de taxa de mudança de célula estão correlacionadas e de forma bastante variada com a taxa de atualização, tanto no sistema novo (métricas novas e antigas – Tabela 7) como no antigo (Tabela 6). Por esse efeito se manter com o sistema novo, acredita-se que esteja relacionado com a variação na resolução da verificação de mudança de célula, o que faz com que a distância percorrida entre cada verificação seja variável.

A influência da distância percorrida por atualização deve ser corrigida executando os ensaios utilizando uma distância fixa por *update*, o que pode ser habilitado por configuração, mas optou-se por desconsiderar as métricas de taxa de *grid_cell_changes* nas análises numéricas e observar a razão entre *visited_count* e o *lifetime* quando se desejar isolar melhoras de movimentação das do tempo de vida. Note que *visited_count* é uma contagem sendo limitada pelo número de células que podem ser acessadas pelo jogador. Acredita-se que a taxa *grid_cell_changes* tenha valor para a análise gráfica, onde podem ser consideradas as variações de taxa de atualização em sua interpretação, e será mantida nos relatórios.

Analisando os *heatmaps* de todos os ensaios é possível identificar que de forma geral, com a adição de capacidades, as capturas ficam progressivamente mais distribuídas pelo mapa. Foram feitas as seguintes observações:

- *heatmap* de captura dos jogadores – Apresenta aumento na distribuição enquanto este ganha melhor capacidade de movimentação (Figuras 9 à 11) o que é revertido quando ganham a capacidade de evasão (Figura 12) o que é explicado pela diminuição de aleatoriedade nas capturas, já que o jogador precisa ser cercado, ou não estar de frente para ser capturado.

- *heatmap* de captura dos fantasmas – Apresenta aumento na distribuição com a melhoria na movimentação dos jogadores (Figuras 9 à 11). Não apresenta mudança quando o jogador ganha a capacidade de ataque oportuno (Figuras 11, comparada à 12, 13, 14 e 15) nem a capacidade para capturar fantasmas deliberadamente (Figura 17).

- *heatmap* de captura de pílulas – Apresenta aumento na distribuição e intensidade conforme o jogador ganha capacidades (Figuras 9 à 15) até estabilizar quando o jogador ganha a capacidade de desviar da rota para capturar pílulas (Figuras 15 e 17).

Para verificar como cada capacidade adquirida afeta as métricas, realizou-se uma análise comparativa entre os resultados dos algoritmos com e sem essa capacidade.

A primeira comparação é entre os resultados dos algoritmos *baseline_full_random* (Figura 9) e *baseline_valid_full_random* (Figura 10). O que os diferencia é a capacidade de escolher um caminho válido, evitando colisões e afetando as métricas de movimentação, como pode ser observado na Tabela 8.

Tabela 8 – Resultados *baseline_full_random* e *baseline_valid_full_random*

Resultado	<i>baseline_full_random</i>	<i>baseline_valid_full_random</i>
Média <i>lifetime</i>	2.120473 (0.992421)	1.890852 (0.869822)
Desvio padrão <i>lifetime</i>	3.642241 (1.778318)	3.256114 (1.587019)
Média <i>visited_count</i>	5,547602 (5.616168)	3,276818 (4.760300)
Desvio padrão <i>visited_count</i>	4,543592 (4.816139)	2,050012 (4.190488)
Média <i>collision_count/lifetime</i>	0.296972 (4.031150)	0.003410 (0)
Desvio padrão <i>collision_count/lifetime</i>	0.474309 (16.914371)	0.052358 (0)

Fonte: próprio autor.

Nota-se que o *lifetime* não foi afetado significativamente. A principal alteração foi em *collision_count/lifetime*, e disso conclui-se que a capacidade de identificar um caminho válido praticamente elimina as colisões.

Também pode ser observado que a contagem *visited_count* diminuiu, o que é inesperado, pois a mobilidade de ambos deveriam ser igualmente ruins. Isso pode ser

atribuído a diminuição da aleatoriedade devido ao menor espaço de decisões, já que as rotas válidas são um subconjunto das rotas.

O algoritmo *baseline_collide_random* (Figura 11) tem comportamento similar ao *baseline_full_random* (Figura 9) mas adiciona a capacidade de detectar que o jogador está em uma rota inválida e trocar para uma rota aleatória, o que pode evitar uma colisão, mas principalmente confere a habilidade de manter-se em uma rota, o que deve melhorar as métricas de movimentação.

Ao comparar relatórios da Figura 9 e 11 e a Tabela 9 verifica-se que houve um aumento significativo na contagem *visited_count* (mesmo quando ajustado para o tempo de vida) o que indica que esses indivíduos se movem de maneira mais efetiva.

Tabela 9 – Resultados *baseline_full_random* e *baseline_collide_random*

Resultado	<i>baseline_full_random</i>	<i>baseline_collide_random</i>
Média <i>lifetime</i>	2.120473 (0.992421)	3.016078 (1.261200)
Desvio padrão <i>lifetime</i>	3.642241 (1.778318)	4.925978 (1.950525)
Média <i>visited_count</i>	5,547602 (5.616168)	26.032585 (24.907463)
Desvio padrão <i>visited_count</i>	4,543592 (4.816139)	23.685879 (23.043532)
Média <i>collision_count/lifetime</i>	0.296972 (4.031150)	2.663081 (5.833328)
Desvio padrão <i>collision_count/lifetime</i>	0.474309 (16.914371)	1.384473 (3.824151)

Fonte: próprio autor.

Nota-se que houve aumento do *lifetime* e disso infere-se que a melhora da mobilidade melhora o tempo de vida. A taxa *collision_count/lifetime* aumentou, o que pode ser explicado pela melhora na movimentação e como o algoritmo funciona. Conclui-se que a capacidade de manter uma rota melhora a mobilidade e que isto melhora o tempo de vida.

O ataque oportuno e a evasão foram capacidades adicionadas no algoritmo *baseline_random* (Figura 12). Vamos usar para comparação o algoritmo *baseline_collide_random* (Figura 11) e a Tabela 10. Essa comparação não é da capacidade de evasão de forma isolada, já que os algoritmos diferem na forma como encontram uma nova rota, o que pode acarretar mudanças na mobilidade.

Pode-se observar aumento da média e da distribuição do *lifetime* jogador. A média do *lifetime* dos fantasmas permaneceu praticamente inalterada, mas a apresenta maior variação. As métricas de captura, representadas pelas taxas de *pills_captured/lifetime*, *ghosts_captured/lifetime* e *ghosts_captured/pills_captured* permaneceram praticamente inalteradas.

Tabela 10 – Resultados *baseline_collide_random* e *baseline_random*

Resultado	<i>baseline_collide_random</i>	<i>baseline_random</i>
Média <i>lifetime</i>	3.016078 (1.261200)	5.437643 (1.880410)
Desvio padrão <i>lifetime</i>	4.925978 (1.950525)	7.843447 (2.877371)
Média <i>visited_count</i>	26.032585 (24.907463)	43.687691 (31.308958)
Desvio padrão <i>visited_count</i>	23.685879 (23.043532)	31.558303 (26.175800)
Média <i>lifetime fantasmas</i>	17.844366 (6.377696)	19.193014 (6.767003)
Desvio padrão <i>lifetime fantasmas</i>	24.859661 (8.254263)	29.827258 (9.255677)
Média <i>pills_captured/lifetime</i>	0.020171 (0.032865)	0.020654 (0.038130)
Desvio padrão <i>pills_captured/lifetime</i>	0.046440 (0.085557)	0.043958 (0.086040)
Média <i>ghosts_captured/lifetime</i>	0.038491 (0.077826)	0.039364 (0.083880)
Desvio padrão <i>ghosts_captured/lifetime</i>	0.116360 (0.244908)	0.104894 (0.229583)
Média <i>ghosts_captured/pills_captured</i>	1.882534 (2.362179)	1.912545 (2.180901)
Desvio padrão <i>ghosts_captured/pills_captured</i>	1.415129 (1.480883)	1.341063 (1.431897)
Média <i>collision_count/lifetime</i>	2.663081 (5.833328)	0.036677 (1.794869)
Desvio padrão <i>collision_count/lifetime</i>	1.384473 (3.824151)	0.135076 (2.216859)

Fonte: próprio autor.

Houve uma diminuição na taxa *collision_count/lifetime*, que pode ser explicada pela forma como os algoritmos operam. O aumento na contagem *visited_count* pode ser explicada pelo aumento do tempo de vida, mas nota-se que a proporção *visited_count/lifetime* diminuiu. Esse resultado sugere que a capacidade de evasão e captura oportuna melhora o tempo de vida dos agentes.

O relatório do algoritmo *baseline_valid_random* (Figura 13) é comparável com o relatório do algoritmo *baseline_random* (Figura 12), eles se diferenciam pela forma como encontram um caminho válido, já que o *baseline_random* consegue encontrá-lo, mas o faz aleatoriamente, por tentativa e erro.

A influência dessa capacidade nas métricas já foi analisada na comparação dos relatórios *baseline_full_random* (Figura 9) e *baseline_valid_random* (Figura 10), mas será feita novamente em algoritmos que não trocam de rota desnecessariamente.

Nesta análise se verifica a influência causada pela melhora na eficiência em achar uma rota válida. Pode-se observar pelos gráficos e pela Tabela 11 que não ocorreram mudanças significativas no desempenho dos algoritmos, mas houve melhora na média do *lifetime* dos jogadores acompanhada de aumento na média da contagem *visited_count* e mantendo uma proporcionalidade.

Isso sugere que aumento na eficiência para escolher rotas válidas aumenta o tempo de vida dos jogadores, mas note que a melhora foi sutil que as métricas de captura acabaram piorando.

Tabela 11 – Resultados *baseline_random* e *baseline_valid_random*

Resultado	<i>baseline_random</i>	<i>baseline_valid_random</i>
Média lifetime	5.437643 (1.880410)	7.497152 (2.090577)
Desvio padrão lifetime	7.843447 (2.877371)	9.772873 (3.467381)
Média visited_count	43.687691 (31.308958)	54.468780 (32.801587)
Desvio padrão visited_count	31.558303 (26.175800)	35.579954 (27.304352)
Média lifetime fantasmas	19.193014 (6.767003)	23.767272 (7.947020)
Desvio padrão lifetime fantasma	29.827258 (9.255677)	42.624808 (12.869246)
Média pills_captured/lifetime	0.043958 (0.086040)	0.019654 (0.028557)
Desvio padrão pills_captured/lifetime	0.039364 (0.083880)	0.041716 (0.072228)
Média ghosts_captured/lifetime	0.104894 (0.229583)	0.037368 (0.059982)
Desvio padrão ghosts_captured/lifetime	1.912545 (2.180901)	0.099099 (0.174826)
Média ghosts_captured/pills_captured	1.912545 (2.180901)	1.897277 (2.171181)
Desvio padrão ghosts_captured/pills_captured	1.341063 (1.431897)	1.310949 (1.332378)

Fonte: próprio autor.

A Capacidade de escolher uma boa rota diretamente foi adicionada no algoritmo *baseline* e para analisar as consequências causadas comparar-se-á o seu relatório (Figura 14) com o do algoritmo *baseline_valid_random* (Figura 13) e os dados da Tabela 12.

Tabela 12 – Resultados *baseline_valid_random* e *baseline*

Resultado	<i>baseline_valid_random</i>	<i>baseline</i>
Média lifetime	7.497152 (2.090577)	11.575185 (2.323688)
Desvio padrão lifetime	9.772873 (3.467381)	15.439541 (4.129051)
Média visited_count	54.468780 (32.801587)	65.889569 (40.815505)
Desvio padrão visited_count	35.579954 (27.304352)	36.634172 (30.841144)
Média lifetime fantasmas	23.767272 (7.947020)	25.920817 (6.941866)
Desvio padrão lifetime fantasma	42.624808 (12.869246)	59.115156 (11.338907)
Média pills_captured/lifetime	0.019654 (0.028557)	0.017482 (0.028328)
Desvio padrão pills_captured/lifetime	0.041716 (0.072228)	0.038725 (0.073228)
Média ghosts_captured/lifetime	0.037368 (0.059982)	0.037259 (0.068875)
Desvio padrão ghosts_captured/lifetime	0.099099 (0.174826)	0.098022 (0.211330)
Média ghosts_captured/pills_captured	1.897277 (2.171181)	2.151605 (2.427156)
Desvio padrão ghosts_captured/pills_captured	1.310949 (1.332378)	1.328892 (1.432112)

Fonte: próprio autor.

O algoritmo *baseline_valid_random* consegue encontrar uma boa rota, mas o faz por tentativa e erro, e por isso, essa análise é novamente sobre a eficiência em encontrar boas rotas.

A análise gráfica mostra que o desempenho dos algoritmos não sofreu mudanças drásticas nas distribuições. Observando os dados da análise textual, verifica-se que houve um aumento no *lifetime* médio.

Observa-se também um aumento na taxa *ghosts_captured/pills_captured* e com isso conclui-se que o aumento na eficiência para escolher boas rotas aumenta o tempo de vida dos jogadores e a eficácia na captura de fantasmas.

Capturar uma pílula deliberadamente, ou seja, desviar de uma rota para realizar a captura, foi a capacidade adicionada no algoritmo *baseline_pill*. Para entender como as métricas de desempenho são afetadas, este será comparado com o algoritmo *baseline*. Nos relatórios das Figuras 14 e 15 e na Tabela 13.

Tabela 13 – Resultados *baseline* e *baseline_pill*

Resultado	baseline	baseline_pill
Média lifetime	11.575185 (2.323688)	31.732253 (8.579658)
Desvio padrão lifetime	15.439541 (4.129051)	42.636980 (12.224337)
Média visited_count	65.889569 (40.815505)	78.614716 (63.649855)
Desvio padrão visited_count	36.634172 (30.841144)	40.201784 (42.994588)
Média lifetime fantasmas	25.920817 (6.941866)	11.466015 (3.729739)
Desvio padrão lifetime fantasma	59.115156 (11.338907)	19.658614 (4.625352)
Média lifetime pílulas	49.312106 (14.613352)	17.405312 (6.076113)
Desvio padrão lifetime pílulas	122.391835 (36.668465)	59.986197 (13.269045)
Média pills_captured/lifetime	0.017482 (0.028328)	0.055452 (0.115734)
Desvio padrão pills_captured/lifetime	0.038725 (0.073228)	0.053756 (0.123234)
Média ghosts_captured/lifetime	0.037259 (0.068875)	0.120560 (0.278327)
Desvio padrão ghosts_captured/lifetime	0.098022 (0.211330)	0.142202 (0.344289)
Média ghosts_captured/pills_captured	2.151605 (2.427156)	2.173264 (2.437723)
Desvio padrão ghosts_captured/pills_captured	1.328892 (1.432112)	1.095832 (1.143494)

Fonte: próprio autor.

Verifica-se que ocorreram mudanças significativas no *lifetime*, e este é o primeiro ensaio em que o jogador tem *lifetime* maior que fantasmas e pílulas. A taxa *pills_captured/lifetime* teve aumento significativo, indicando melhora na capacidade de capturar pílulas, mas a taxa *ghosts_captured/pills_captured* permaneceu praticamente inalterada. Conclui-se que a capacidade de capturar pílulas deliberadamente melhora o tempo de vida e a captura de pílulas e acarreta diminuição no tempo médio de vida de fantasmas e pílulas.

A última análise dessa bateria que será feita é a da capacidade de desviar da rota para capturar fantasmas, ou seja, capturá-los deliberadamente.

Esta capacidade foi adicionada no algoritmo *baseline_ghost_pill*. Para analisar as alterações que ela causou nas métricas de desempenho analisar-se-á as Figuras 15 e 16 e a Tabela 14. Nota-se uma alteração significativa na taxa *collision_count/updates*. Esta análise é um contrassenso e devido a ela identificou-se um erro na implementação do algoritmo *baseline_pill_ghost* conforme mencionado anteriormente.

Tabela 14 – Resultados *baseline_pill* e *baseline_pill_ghost*

Resultado	<i>baseline_pill</i>	<i>baseline_pill_ghost</i>
Média <i>collision_count/updates</i>	9.939039e-08	0.000183
Desvio padrão <i>collision_count/updates</i>	5.511021e-06	0.000269

Fonte: próprio autor.

Este teste foi feito com 15000 iterações, usando o sistema novo, e chamado de *baseline_pill_ghost_fixed*, o relatório gráfico pode ser observado nas Figuras 17 e 18, usando as métricas novas e antigas, respectivamente. Os resultados numéricos estão na Tabela 15.

A diferença mais notável é a da escala de tempo entre os ensaios, que é aproximadamente 3 vezes maior para o ensaio *baseline_pill* como pode ser visto nas métricas de *lifetime*, e propagado para as taxas de *lifetime*, onde se nota uma mudança proporcional. Esta diferença pode ser atribuída a mudança na percepção do tempo para os agentes, causada por variação na taxa de atualização que existe no sistema antigo, já que o mesmo efeito não é observado nas métricas do sistema novo. Para esta comparação se utilizou somente as métricas novas.

Tabela 15 – Resultados *baseline_pill* e *baseline_pill_ghost_fixed*

Resultado	<i>baseline_pill</i>	<i>baseline_pill_ghost_fixed</i>
Média <i>lifetime</i>	31.732253 (8.579658)	10.594665 (8.521430)
Desvio padrão <i>lifetime</i>	42.636980 (12.224337)	17.067515 (11.996945)
Média <i>visited_count</i>	78.614716 (63.649855)	70.842471 (70.842471)
Desvio padrão <i>visited_count</i>	40.201784 (42.994588)	48.393639 (48.393639)
Média <i>lifetime fantasmas</i>	11.466015 (3.729739)	3.666765 (2.950298)
Desvio padrão <i>lifetime fantasmas</i>	19.658614 (4.625352)	5.258799 (3.770631)
Média <i>lifetime pílulas</i>	17.405312 (6.076113)	7.599622 (6.119774)
Desvio padrão <i>lifetime pílulas</i>	59.986197 (13.269045)	18.677552 (13.377343)
Média <i>pills_captured/lifetime</i>	0.055452 (0.115734)	0.123272 (0.118595)
Desvio padrão <i>pills_captured/lifetime</i>	0.053756 (0.123234)	0.111014 (0.122760)
Média <i>ghosts_captured/lifetime</i>	0.120560 (0.278327)	0.383403 (0.368334)
Desvio padrão <i>ghosts_captured/lifetime</i>	0.142202 (0.344289)	0.412898 (0.445136)
Média <i>ghosts_captured/pills_captured</i>	2.173264 (2.437723)	3.109689 (3.109689)
Desvio padrão <i>ghosts_captured/pills_captured</i>	1.095832 (1.143494)	1.400886 (1.400886)
Média <i>collision_count/lifetime</i>	9.939039e-08 (0)	0 (0)
Desvio padrão <i>collision_count/lifetime</i>	5.511021e-06 (0)	0 (0)
Média <i>updates/lifetime</i>	180.004096 (213.560621)	220.400696 (234.056769)
Desvio padrão <i>updates/lifetime</i>	55.503697 (20.619153)	58.519508 (35.845217)

Fonte: próprio autor.

O tempo de vida do jogador e das pílulas praticamente não sofreram alterações como pode ser visto na métrica *internal_lifetime* (linha *lifetime*, entre parênteses), mas o tempo de vida de fantasmas caiu consideravelmente. Note que a capacidade de capturar fantasmas

deliberadamente não impacta significativamente a vida do jogador, mas diminui o tempo de vida dos fantasmas.

Como o tempo de vida dos jogadores não é alterado significativamente, os ensaios que usam exclusivamente o tempo de vida para calcular o *fitness* não têm pressão seletiva para adquirir a capacidade de capturar fantasmas deliberadamente, pelo menos no caso de já terem adquirido a capacidade de evasão e de capturar pílulas deliberadamente, mas existe a possibilidade da progressão de capacidades ocorrer de outra forma.

Analisando as métricas de captura podemos notar que os jogadores controlados por *baseline_pill_ghost* capturam mais fantasmas e com maior eficiência. A captura de pílulas permanece inalterada, o que é esperado.

Nota-se também uma pequena melhora na movimentação, como pode ser visto pelo aumento da contagem *visited_count*. Conclui-se que a capacidade de capturar fantasmas deliberadamente melhora a movimentação de jogadores sem impactar o seu tempo de vida e aumenta a eficiência na captura de fantasmas e estes vivem por menos tempo.

Devido à realização do teste *baseline_ghost_pill_fixed* foi possível realizar uma comparação de controle para o impacto causado pela diferença de iterações e de ambiente de execução nas métricas. Os dados estão na Tabela 16 e os resultados são similares e nota-se que as métricas do sistema novo estão, em geral, mais próximas. As divergências mais notáveis entre os ensaios são as métricas de tempo de vida e o *visited_count*.

Tabela 16 – Comparação de controle para mudanças no número de iterações

Resultado	<i>baseline_pill_ghost</i> <i>baseline_take_2</i> 1017 iterações	<i>baseline_pill_ghost_fixed</i> 16981 iterações
Média lifetime	7.725664 (7.028454)	10.594665 (8.521430)
Desvio padrão lifetime	11.933397 (10.793323)	17.067515 (11.996945)
Média visited_count	63.482793 (63.482793)	70.842471 (70.842471)
Desvio padrão visited_count	46.917480 (46.917480)	48.393639 (48.393639)
Média lifetime fantasmas	3.258385 (2.966141)	3.666765 (2.950298)
Desvio padrão lifetime fantasma	4.934307 (4.450538)	5.258799 (3.770631)
Média lifetime pílulas	6.788732 (6.182109)	7.599622 (6.119774)
Desvio padrão lifetime pílulas	14.876564 (13.482384)	18.677552 (13.377343)
Média pills_captured/lifetime	0.114062 (0.099939)	0.123272 (0.118595)
Desvio padrão pills_captured/lifetime	0.114977 (0.121441)	0.111014 (0.122760)
Média ghosts_captured/lifetime	0.359332 (0.313357)	0.383403 (0.368334)
Desvio padrão ghosts_captured/lifetime	0.435861 (0.442507)	0.412898 (0.445136)
Média ghosts_captured/pills_captured	3.116054 (3.116054)	3.109689 (3.109689)
Desvio padrão ghosts_captured/pills_captured	1.374744 (1.374744)	1.400886 (1.400886)
Média collision_count/lifetime	0 (0)	0 (0)
Desvio padrão collision_count/lifetime	0 (0)	0 (0)
Média fps	207.930288	234.056769
Desvio padrão fps	51.977758	35.845217

Fonte: próprio autor.

As capacidades que serão analisadas na população são a de movimentação, evasão, captura de pílulas e fantasmas. Com base no resultado das comparações (desprezando influências cruzadas) foi elaborada a Tabela 17 que contém um resumo das alterações que se pode esperar com a progressão das capacidades de interesse.

Tabela 17 – Capacidades e Métricas

Resultado	movimentação	evasão	captura de pílulas	captura de fantasma
lifetime	aumenta	aumenta	aumenta	---
lifetime pílulas	---	---	diminui	---
lifetime fantasmas	---	---	diminui	diminui
visited_count	aumenta	aumenta	aumenta	aumenta
collision_count	diminui	---	---	---
pills_captured/lifetime	---	---	aumenta	aumenta
ghosts_captured/lifetime	---	---	---	aumenta
ghosts_captured/pills_captured	---	---	---	aumenta

Fonte: próprio autor.

Analisando os resultados dessa bateria, pode-se observar que o tempo de vida do jogador é influenciado positivamente pelas capacidades de capturar pílulas deliberadamente, escolher rotas boas e válidas de forma eficiente, a capacidade de evasão e manter-se em uma rota e por isso é um bom candidato a ser usado diretamente como função de *fitness*, com o benefício de estar alinhado com a métrica que se pretende otimizar, que é ele mesmo. Notou-se, porém, que para jogadores que já tem a capacidade movimentação, evasão e de capturar pílulas deliberadamente, usar o tempo de vida como *fitness* exclusivamente não cria pressão seletiva para evoluir a captura deliberada de fantasmas.

A correlação entre o *internal_lifetime* e as outras métricas de interesse, como pode ser observado nas Tabelas 18 e 19, apresenta-se bastante alta em geral, sendo a eficiência de captura a única exceção, indicando novamente que funções *fitness* baseadas exclusivamente em tempo de vida talvez não criem pressão seletiva para a capacidade de capturar fantasmas.

Pode ser notado também que esta se apresenta menor para os ensaios *baseline_full_random* e *baseline_valid_random* que não conferem aos jogadores boa capacidade de movimentação, isto se estende para as Tabelas 18, 19 e 20 e indica que a capacidade de movimentação aumenta a correlação entre o tempo de vida e as outras métricas, o que sustenta a hipótese de que o início da evolução dos jogadores ocorre na capacidade de movimentação, já que sem ela todas as outras capacidades não conseguem exercer seu potencial, ou ainda, não surtem efeito. Essa é uma das motivações da elaboração da função de *fitness movement_captures_hack_26*.

Tabela 18 – Correlação *internal_lifetime* e métricas de captura – *baseline_take_2*

Ensaio	Correlação de Spearman <i>internal_lifetime</i> e <i>pills_captured</i>	Correlação de Spearman <i>internal_lifetime</i> e <i>ghosts_captured</i>	Correlação de Spearman <i>internal_lifetime</i> e <i>pills_captured/ghosts_captured</i>
baseline_full_random	0.3914583396246274	0.4329227446714141	-0.0841120136111849
baseline_valid_full_random	0.3450634156918524	0.3221901604443303	0.02542114851235921
baseline_collide_random	0.5801152390363242	0.5519864799117642	0.006651682333376426
baseline_random	0.6445949887486111	0.6075687695125525	0.04856224387073505
baseline_valid_random	0.6009753676670254	0.5722426956829563	0.2403598232558879
baseline	0.5892183498837358	0.5679121273888286	0.18973942106115527
baseline_pill	0.9064897247695021	0.888019749887194	0.05511874727046364
baseline_pill_ghost_fixed	0.9117095225785019	0.8930231193408955	0.04332302507573646

Fonte: próprio autor.

Tabela 19 – Correlação *internal_lifetime* e métricas de movimentação – *baseline_take_2*

Ensaio	Correlação de Spearman <i>internal_lifetime</i> e <i>visited_count</i>	Correlação de Spearman <i>internal_lifetime</i> e <i>collision_count</i>
baseline_full_random	0.8407365557140285	0.5034397120549118
baseline_valid_full_random	0.8203472277640385	NaN
baseline_collide_random	0.9495848219815823	0.9357402311670212
baseline_random	0.9384903274691926	0.7838666106573386
baseline_valid_random	0.9414950886251815	NaN
baseline	0.966871546811761	NaN
baseline_pill	0.9743901685699444	NaN
baseline_pill_ghost	0.9764832963763974	NaN

Fonte: próprio autor.

Tabela 20 – Correlação *visited_count* e métricas de captura – *baseline_take_2*

Ensaio	Correlação de Spearman <i>internal_lifetime</i> e <i>pills_captured</i>	Correlação de Spearman <i>internal_lifetime</i> e <i>ghosts_captured</i>	Correlação de Spearman <i>internal_lifetime</i> e <i>pills_captured/ghosts_captured</i>
baseline_full_random	0.42450595122687523	0.3838790792913599	-0.04713298035791586
baseline_valid_full_random	0.3466825426248602	0.3212222204829110	0.10060759660410448
baseline_collide_random	0.5612406819699777	0.5362804024868922	0.13061316800616535
baseline_random	0.6155157229678652	0.579356188593894	0.1202251106164757
baseline_valid_random	0.5780534266274021	0.5510659870347135	0.16876664439424552
baseline	0.558283117338076	0.5415779191306305	0.12045959453528782
baseline_pill	0.8845299452164712	0.8686331916077388	0.0460750681344088
baseline_pill_ghost_fixed	0.9034884092517598	0.8901796006596537	0.1004720962399488

Fonte: próprio autor.

Apesar de o *collision_count* não ser exatamente relevante nas comparações de correlação, já que alguns algoritmos especialistas usam a colisão em seu funcionamento, o resumo da Tabela 17 sugere que pode ser benéfico desenvolver uma função *fitness* usando-o como base.

3.7.2 Resultados Neuro-evolutivos de topologia fixa (bateria 2)

Nesta bateria foram realizados 7 ensaios com cada um dos modos (7 no total) usando as funções de ativação *Relu* e *Tanh*, totalizando 98 ensaios.

Essa etapa acumula as funções de testar a infraestrutura implementada, corrigir erros e determinar os testes da bateria 3, ou seja, quais modos, e para cada modo qual arquitetura.

Foi escolhido ao menos um modo de cada forma básica e como a segunda forma só possui um modo, o *NB4-path-grading*, este já estava selecionado e restou apenas determinar a arquitetura do teste final.

A primeira forma pode ser dividida entre os modos com saída binária ou ponto flutuante de forma que dois modos serão selecionados, como o NB4 é o único com saída em ponto flutuante, este também já estava incluído, restando escolher um modo com saída binária.

Após essas considerações a tarefa desta etapa reduziu-se a escolher um entre os modos B1, B2 e B3 e outro entre *B1-path-grading-simple* e *B1-path-grading-hack* e dentre os modos escolhidos determinar uma arquitetura e com isso listar os testes para a terceira bateria.

Para realizar a escolha dentre os modos, foram comparados os relatórios gráficos entre os concorrentes buscando por indícios de melhora ou estagnação e não necessariamente valores em absoluto, analisando a progressão das métricas em geral e do tempo de vida em particular para selecionar o de melhor desempenho geral. Os modos escolhidos foram NB4, *NB4-path-grading*, B1 e *B1-path-grading-hack*.

A comparação entre os ensaios de *B1-path-grading-simple* e *B1-path-grading-hack* mostrou desempenho bastante semelhante. Em geral, foi possível observar que o primeiro teve desempenho melhor para as contagens *visited_count* e as métricas *ghosts_captured*. A taxa *ghosts_captured/pills_captured* atingiu níveis equivalentes, embora tenha levado mais iterações.

Entre os ensaios B1, B2 e B3 foi escolhido o modo B1, pois este se apresentou melhor no lifetime e na contagem *visited_count* (no ensaio com uma camada de oculta, 3 neurônios por camada e função de ativação tanh)

Para decidir a arquitetura foram comparados os relatórios gráficos de cada um dos modos entre si, a arquitetura escolhida foi a mais complexa que mostrou indícios de progressão analisando comparativamente como a adição de camadas e neurônios por camada e função de ativação alterou o resultado.

As arquiteturas escolhidas foram:

- 1x3(tanh) – Modo B1
- 1x3(tanh) – Modo *B1-path-grading-hack*
- 1x12(relu) – Modo NB4
- 3x3(relu) – Modo *NB4-path-grading*

Nesta bateria não foram realizados testes com 12 neurônios por camada. Como este ensaio seria para a etapa final e considerando a complexidade do modo NB4 e a melhora no desempenho quando comparados os ensaios 1x3(relu) e 1x5(relu), o que sugere que talvez seja benéfico adicionar mais neurônios a camada oculta, optou-se por testar essa configuração, que se mostrou promissora em testes preliminares.

A função *fitness* utilizada em todos esses ensaios foi a *updates*. Essa função foi escolhida por ser um *proxy* do *lifetime* com melhor resolução. Após analisar o resultado da bateria 1, e observar que esta métrica é afetada pelo desempenho da simulação, entende-se que essa talvez não tenha sido uma boa escolha, porém os testes da bateria 1 e 2 foram realizados em paralelo, de forma que a análise da bateria 1 só foi concluída após a realização destes ensaios. Não foram observados problemas causados por variação na taxa de atualização do jogo nesta bateria, o que poderia ter invalidado os resultados, já que os impactos causados têm ordem de magnitude dentro da escala da maior parte das melhorias observadas.

Nesses ensaios não foi observada a capacidade de captura e evasão no mesmo indivíduo. Os indivíduos que evoluíram a captura geralmente mostravam a estratégia de sempre seguir os fantasmas e com o passeio pelo mapa capturar eventualmente uma pílula e então o fantasma. Alguns indivíduos adquiriram a capacidade de capturar pílulas e fantasmas, mas sem apresentar evasão. Essa é outra das motivações da criação da função *fitness movement_captures_hack_26*.

3.7.3 Resultados NEAT e Neuro-evolutivos de topologia fixa finais (bateria 3)

Após a execução desses ensaios notou-se que a configuração para reiniciar os fantasmas comprometeu os dados coletados para o tempo de vida destes, já que eram criados fantasmas novos. Isso acabou motivando alterações no estilo dos relatórios gráficos, modificado para dar mais visibilidade a progressão da média móvel e auxiliar a identificar vícios de movimentação.

Nos novos relatórios a progressão da média móvel em verde corresponde a divisão em 100 blocos e em azul 30 blocos. A média global aparece em vermelho.

A análise para os ensaios NEAT acompanha algumas observações obtidas durante a execução do ensaio, embora isso vá contra o propósito de eliminar a supervisão do ensaio. Isso acabou ocorrendo devido à curiosidade sobre a progressão exibida por esse tipo de sistema e, portanto alguns comentários serão inseridos quando pertinente.

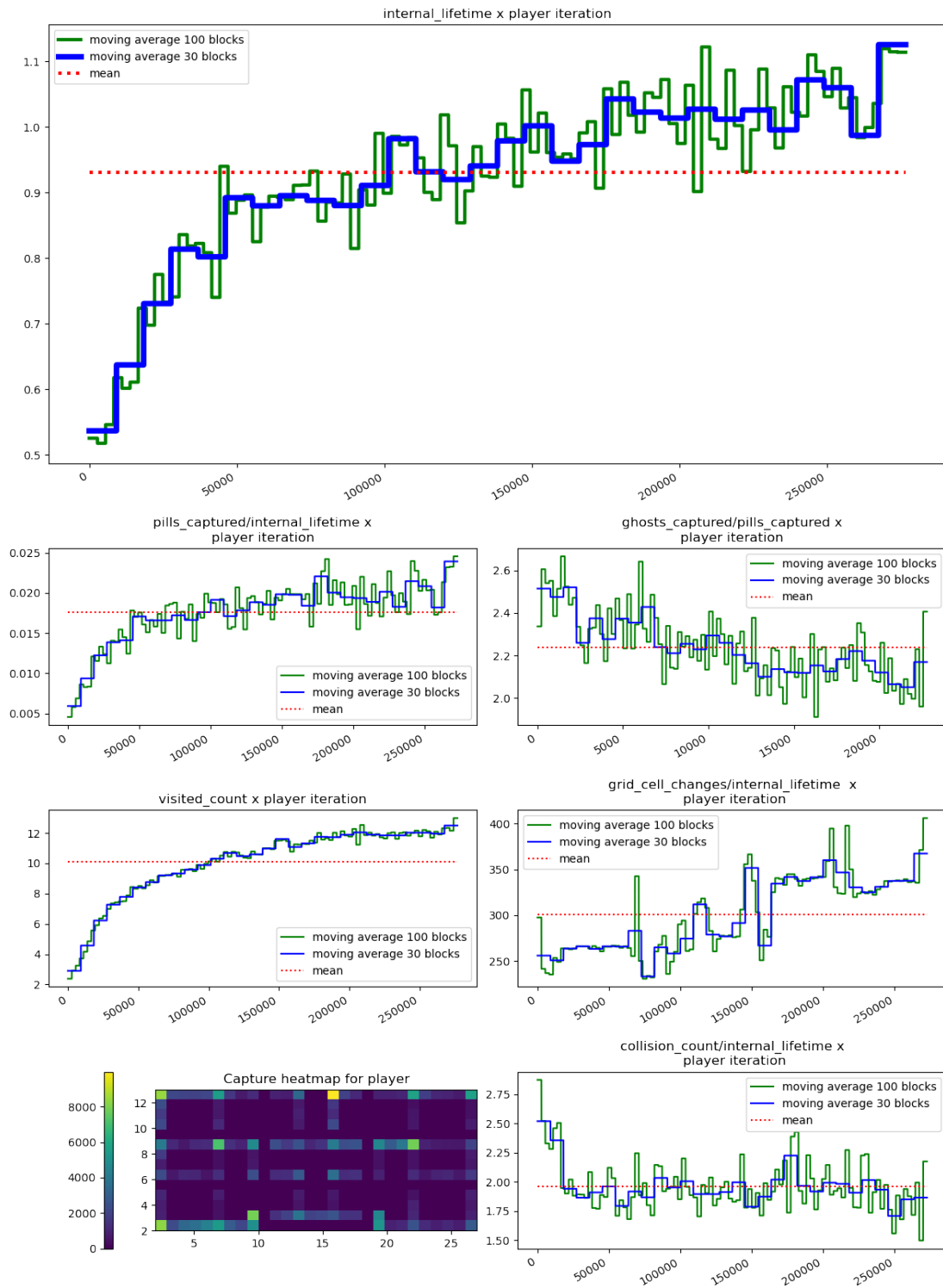
A metodologia evoluiu para analisar sistemas de topologia fixa e os sistemas NEAT acabaram evidenciando a necessidade de extensões que acomodem a análise de seu funcionamento interno, ou seja, coletar eventos relacionados a descoberta de inovações e espécies e intervalos onde o nicho estava operando. O relatório foi modificado para esses sistemas e a métrica *grid_cell_changes* foi substituída pela distribuição das espécies, já que estes dados estavam disponíveis nos logs e deles podemos inferir alguns desses aspectos.

Para produzir esses relatórios foi usado o *notebook* Jupyter *data_analysis_generate_pd_battery_3*, que pode ser encontrado no pacote [Extinção 5](#), na pasta *data_analysis*.

Os resultados gráficos dessa bateria de ensaios podem ser vistos nas Figuras de 18 a 34. O conjunto desses ensaios será referido como ensaios bateria 3. Os ensaios são:

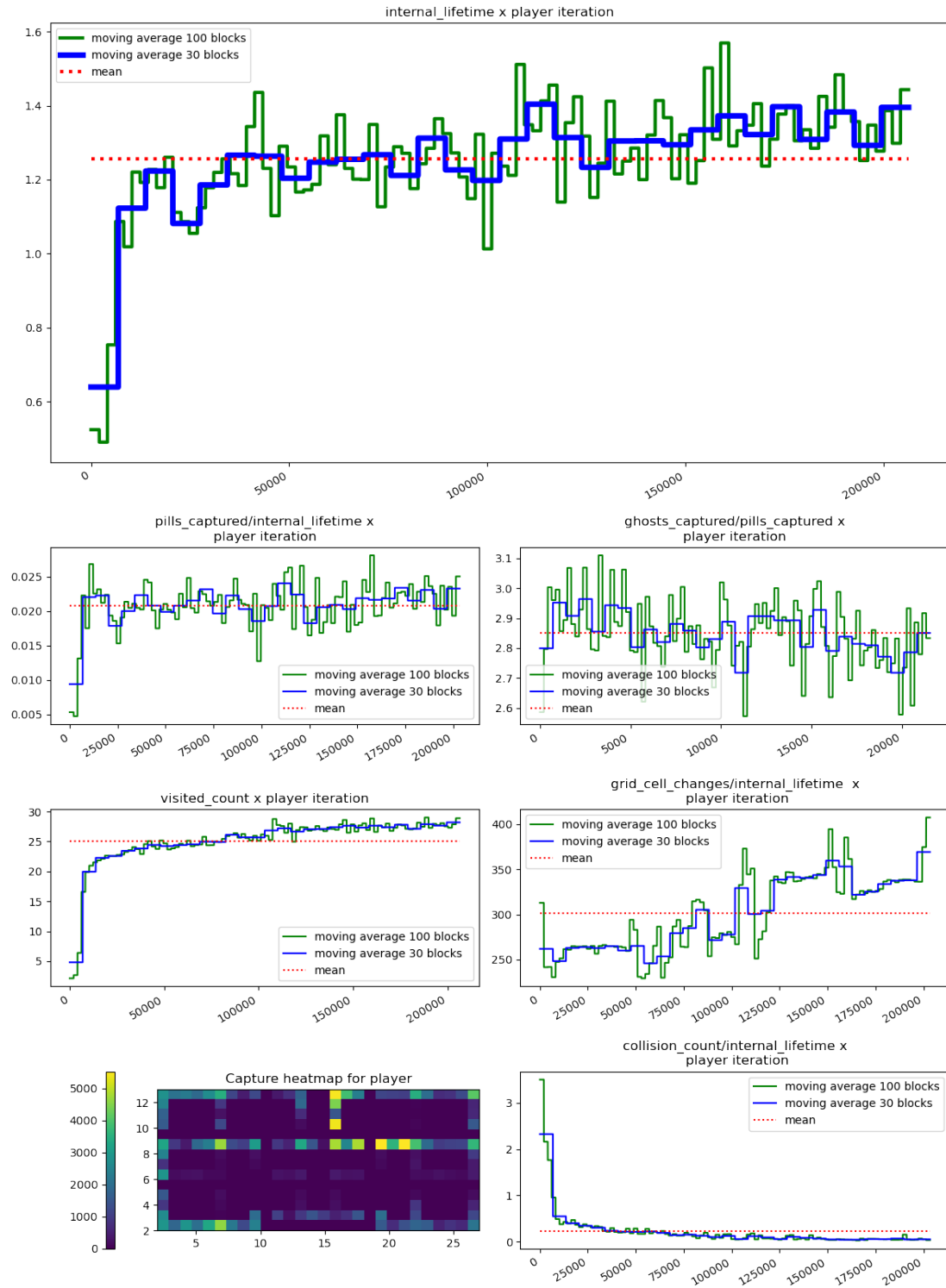
- 1677445806 b1 e *movement_captures_hack_26*, Figura 19.
- 1677445825 *b1_path_grading_hack* e *movement_captures_hack_26*, Figura 20
- 1677445848 nb4 e *movement_captures_hack_26*, Figura 21.
- 1677445867 *nb4_path_grading* e *movement_captures_hack_26*, Figura 22.
- 1677445899 b1 e *lifetime*, Figura 23.
- 1677445913 *b1_path_grading_hack* e *lifetime*, Figura 24.
- 1677445930 nb4 e *lifetime*, Figura 25.
- 1677445953 *nb4_path_grading* e *lifetime*, Figura 26.
- 1677959359 NEAT b1 e *movement_captures_hack_26*, Figura 27.
- 1677959407 NEAT *b1_path_grading_hack* e *movement_captures_hack_26*, Figura 28.
- 1677959485 NEAT nb4 e *movement_captures_hack_26*, Figura 29.
- 1677959539 NEAT *nb4_path_grading* e *movement_captures_hack_26*, Figura 30.
- 1677959384 NEAT b1 e *lifetime*, Figura 31.
- 1677959432 NEAT *b1_path_grading_hack* e *lifetime*, Figura 32.
- 1677959510 NEAT nb4 e *lifetime*, Figura 33.
- 1677959569 NEAT *nb4_path_grading* e *lifetime*, Figura 34.

Figura 19: b1 e movement_captures_hack_26 (1677445806)



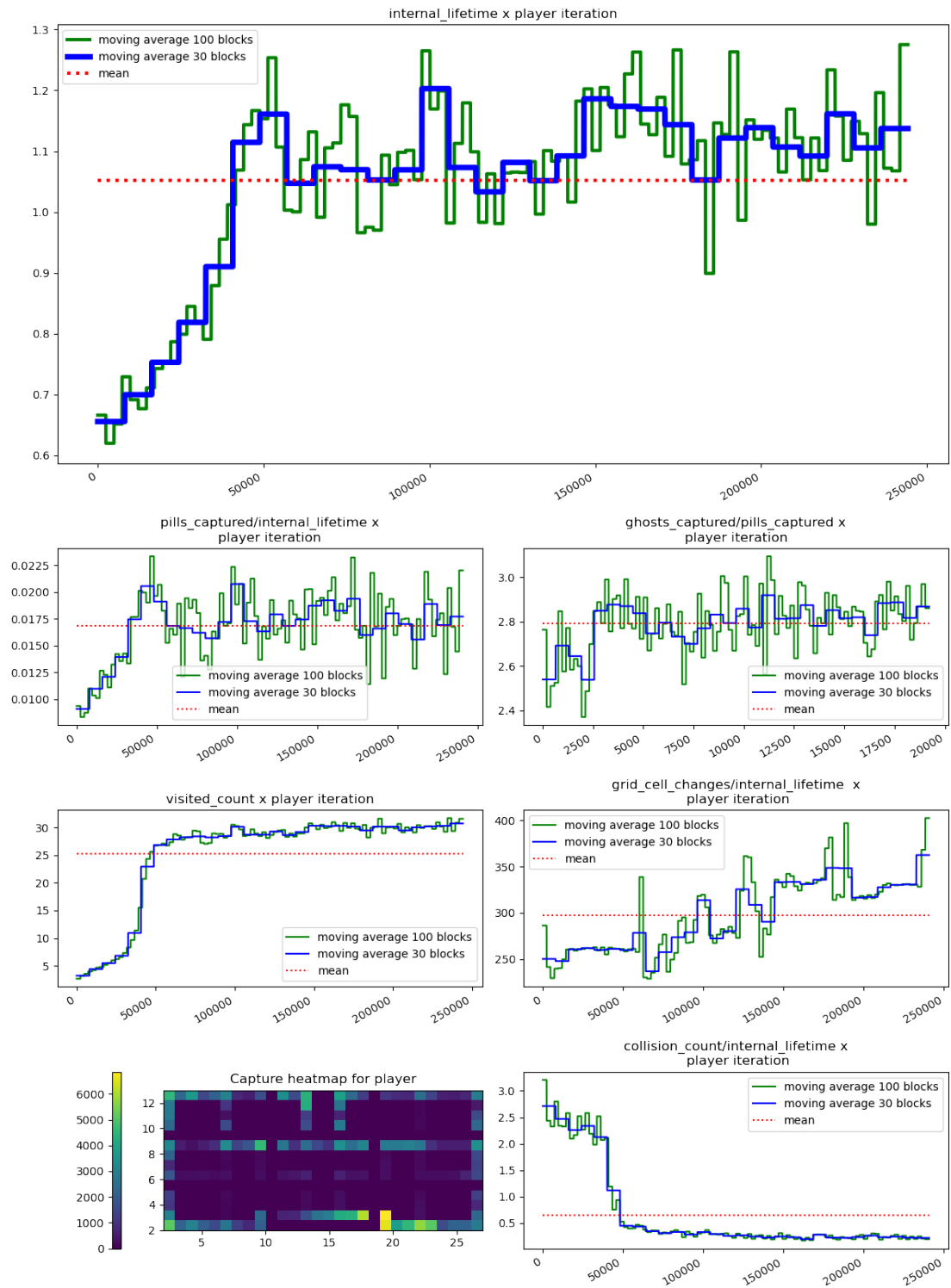
Fonte: próprio autor.

Figura 20: *b1_path_grading_hack* e *movement_captures_hack_26* (1677445825)



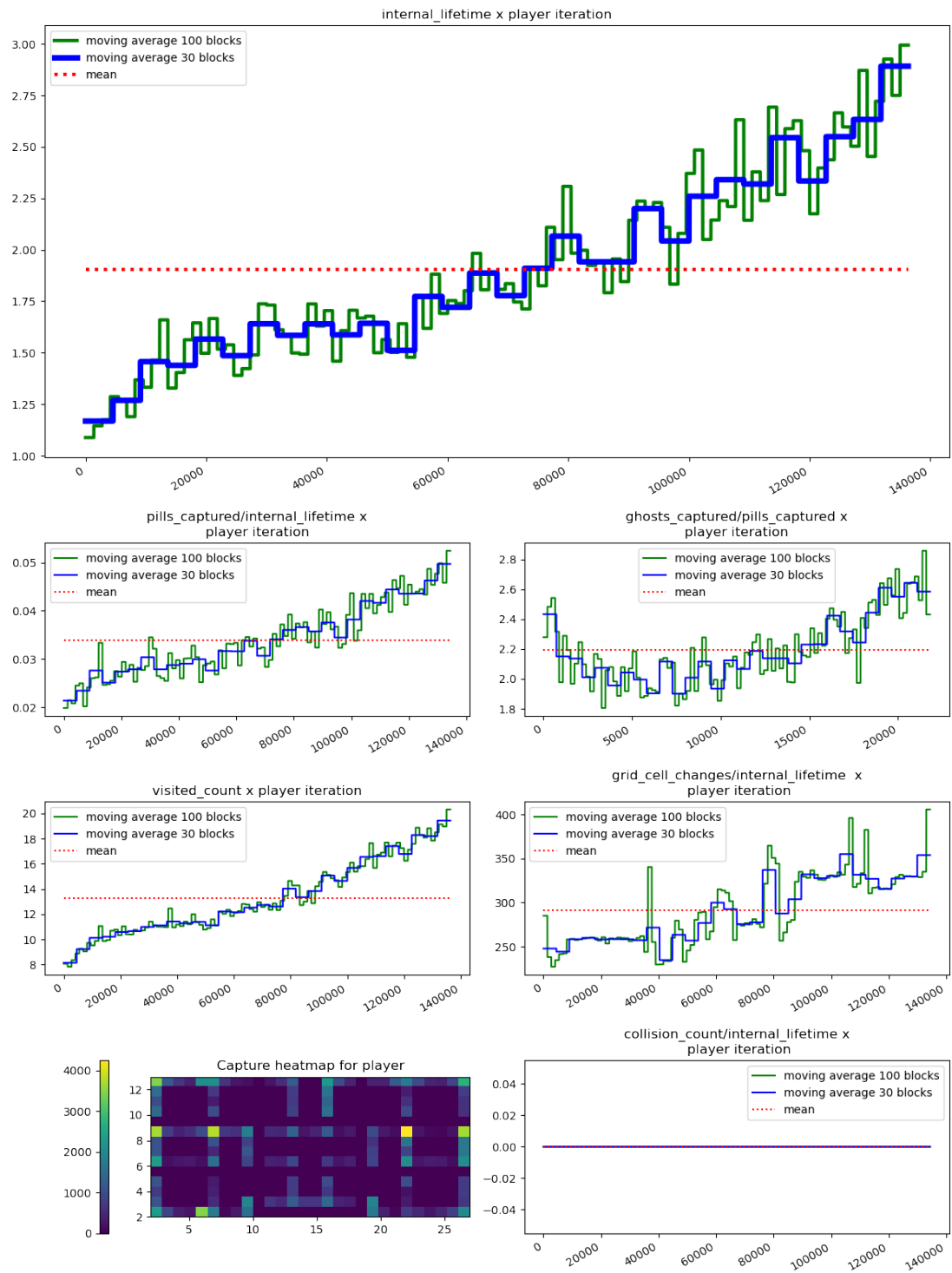
Fonte: próprio autor.

Figura 21: nb4 e *movement_captures_hack_26* (1677445848)

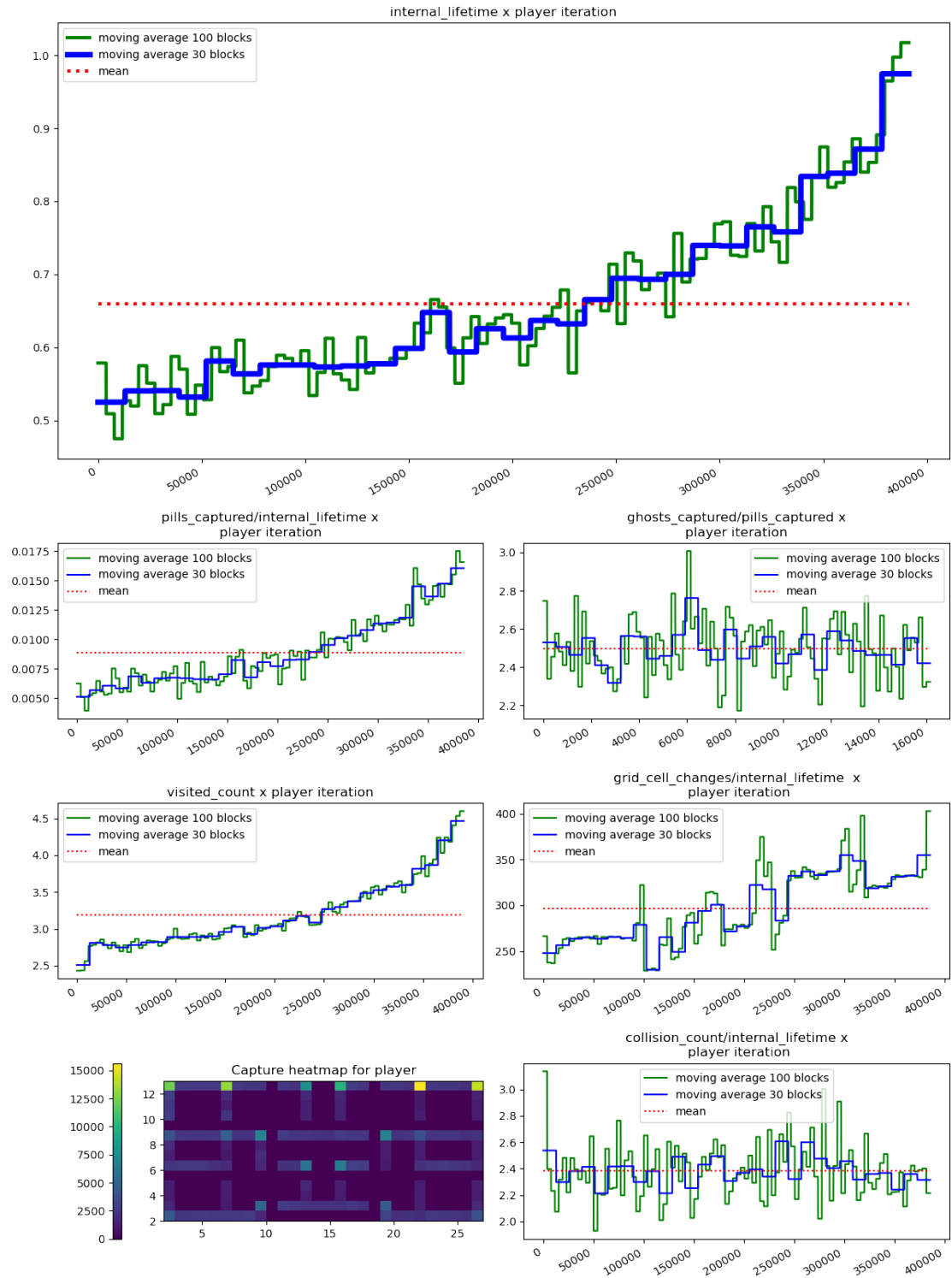


Fonte: próprio autor.

Figura 22: *nb4_path_grading* e *movement_captures_hack_26* (1677445867)

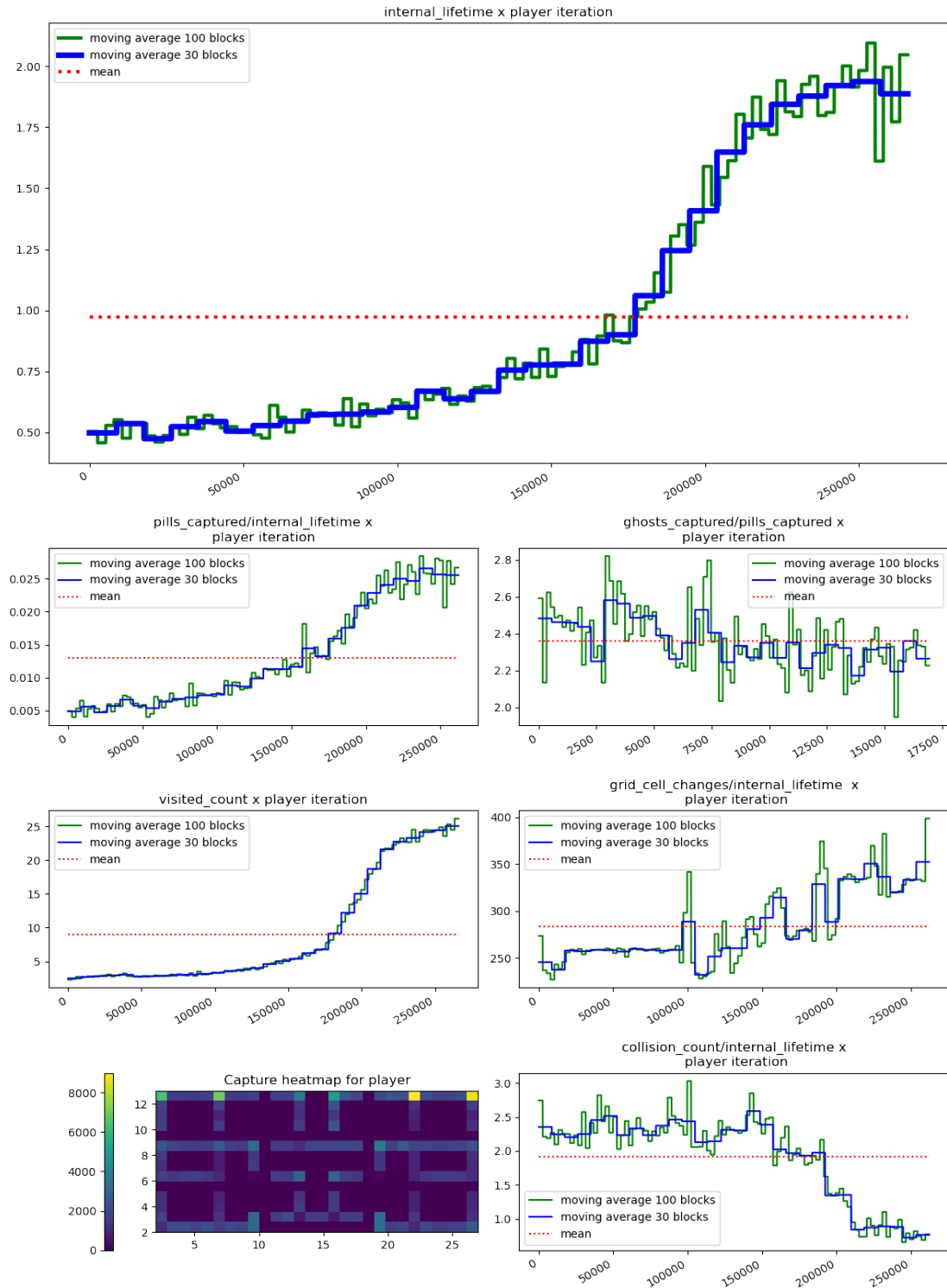


Fonte: próprio autor.

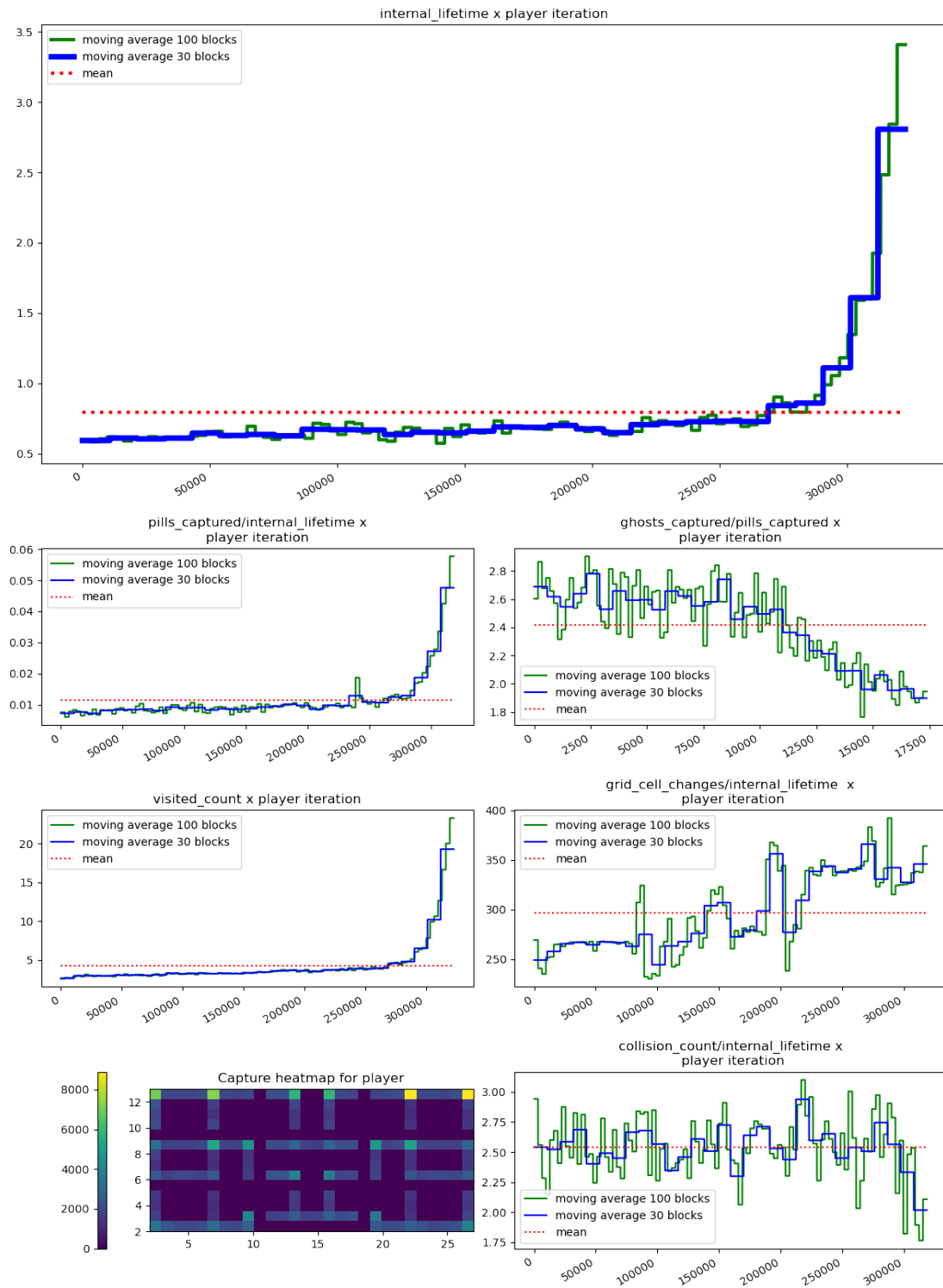
Figura 23: b1 e *lifetime* (1677445899)

Fonte: próprio autor.

Figura 24: *bl_path_grading_hack* e lifetime (1677445913)

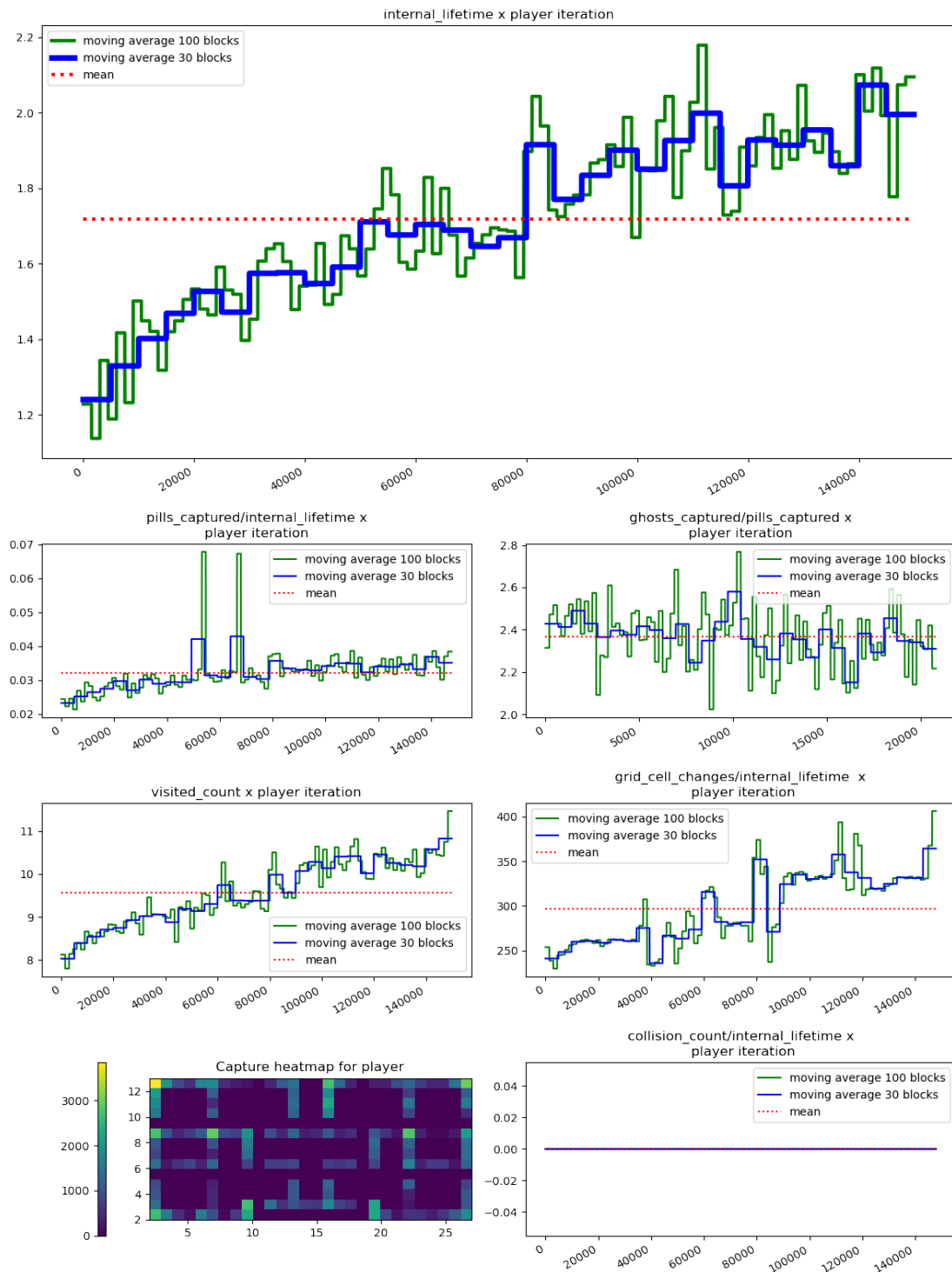


Fonte: próprio autor.

Figura 25: nb4 e *lifetime* (1677445930)

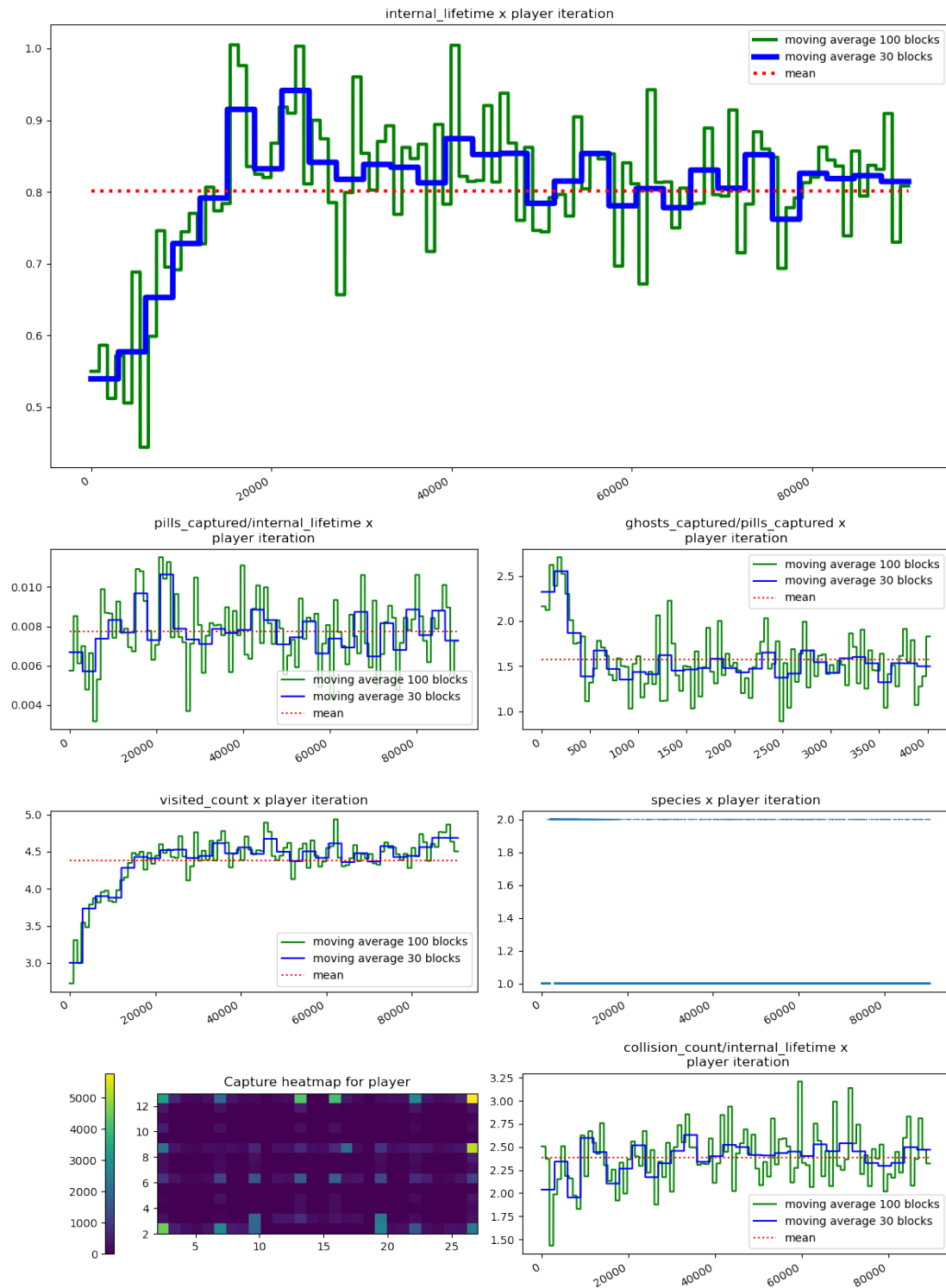
Fonte: próprio autor.

Figura 26: *nb4_path_grading e lifetime* (1677445953)



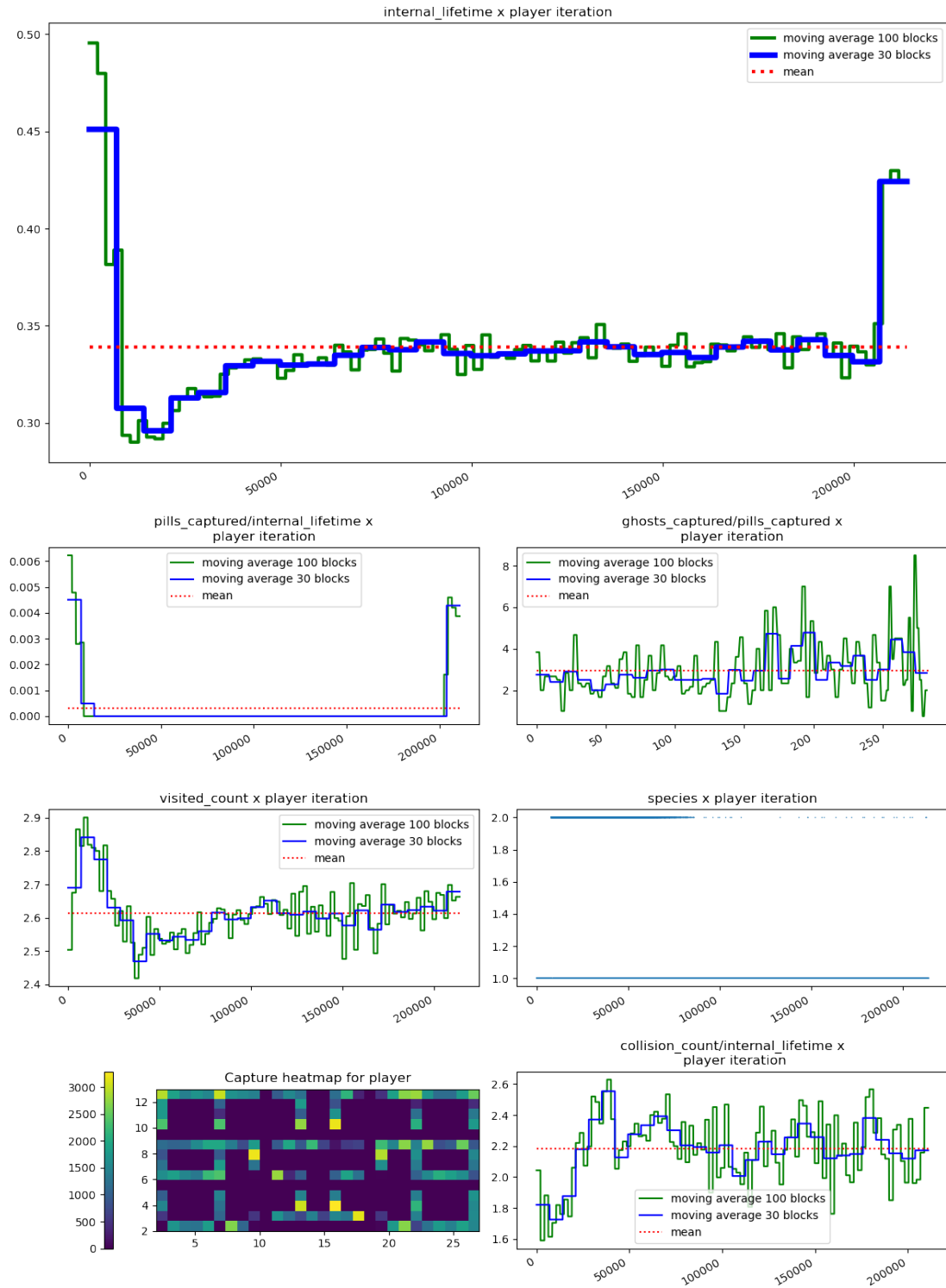
Fonte: próprio autor.

Figura 27: b1 e movement_captures_hack_26 – NEAT(1677959359)



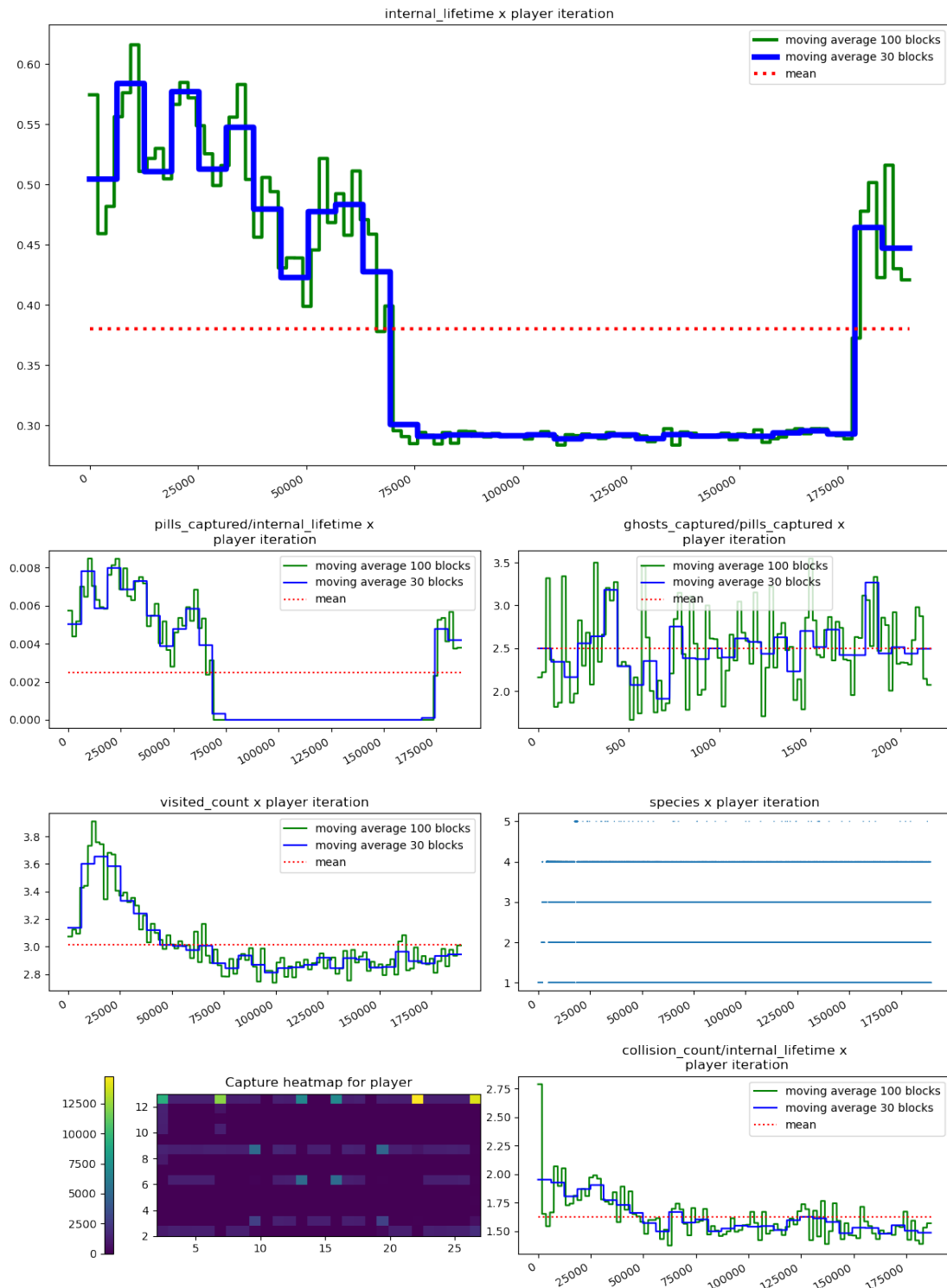
Fonte: próprio autor.

Figura 28: *b1_path_grading_hack* e *movement_captures_hack_26* – NEAT(1677959407)



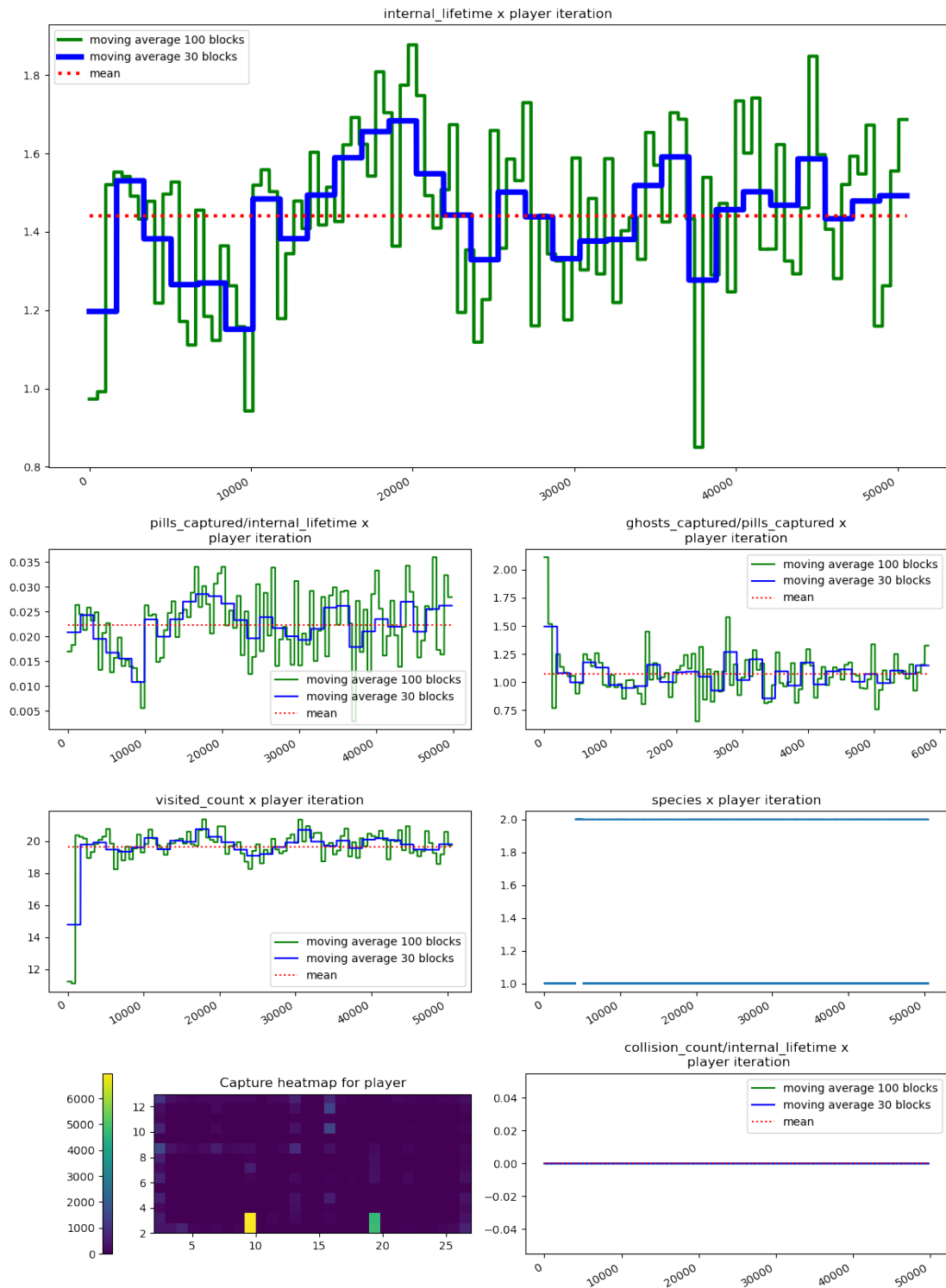
Fonte: próprio autor.

Figura 29: nb4 e *movement_captures_hack_26* – NEAT(1677959485)

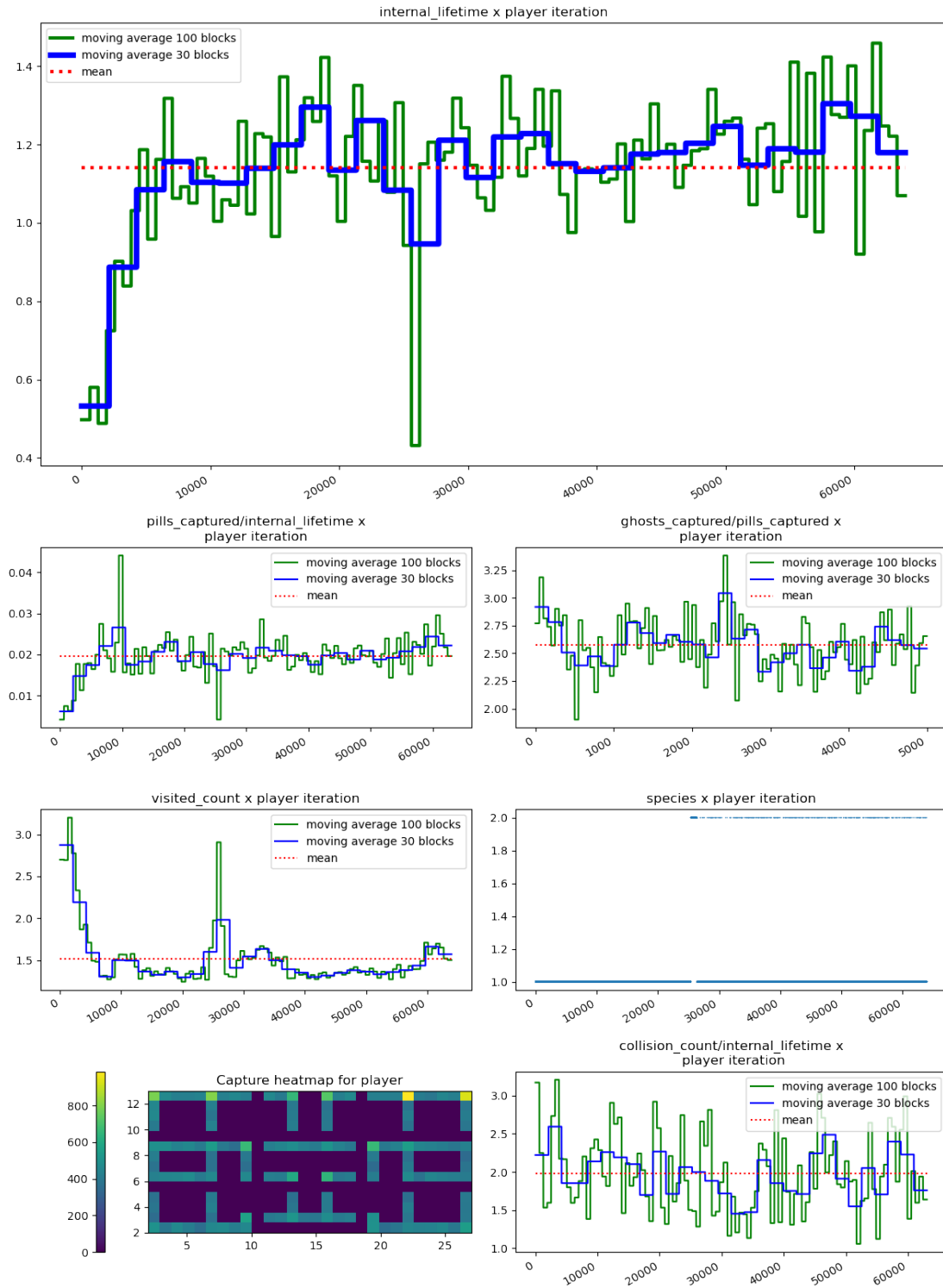


Fonte: próprio autor.

Figura 30: *nb4_path_grading e movement_captures_hack_26* – NEAT(1677959539)

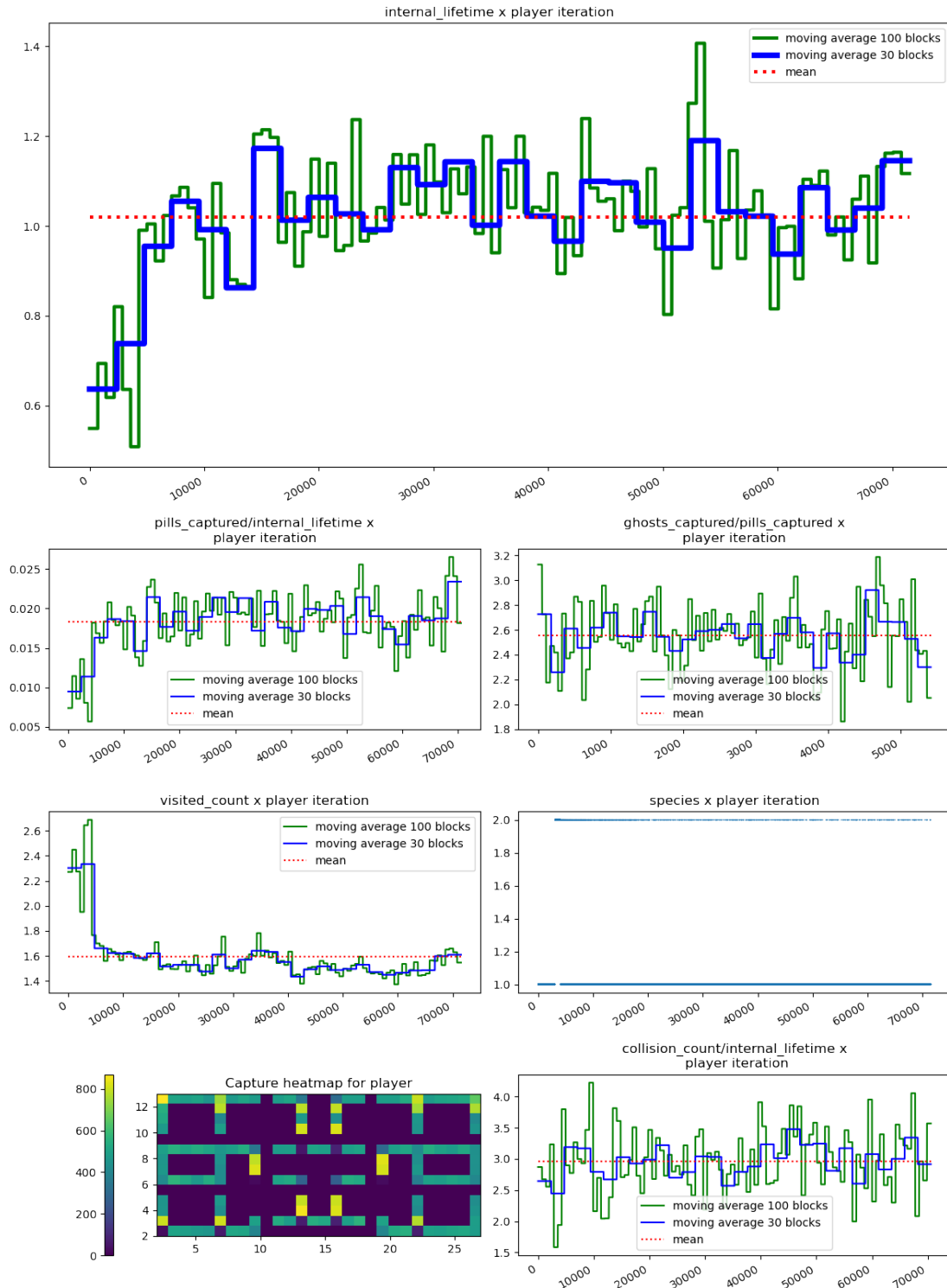


Fonte: próprio autor.

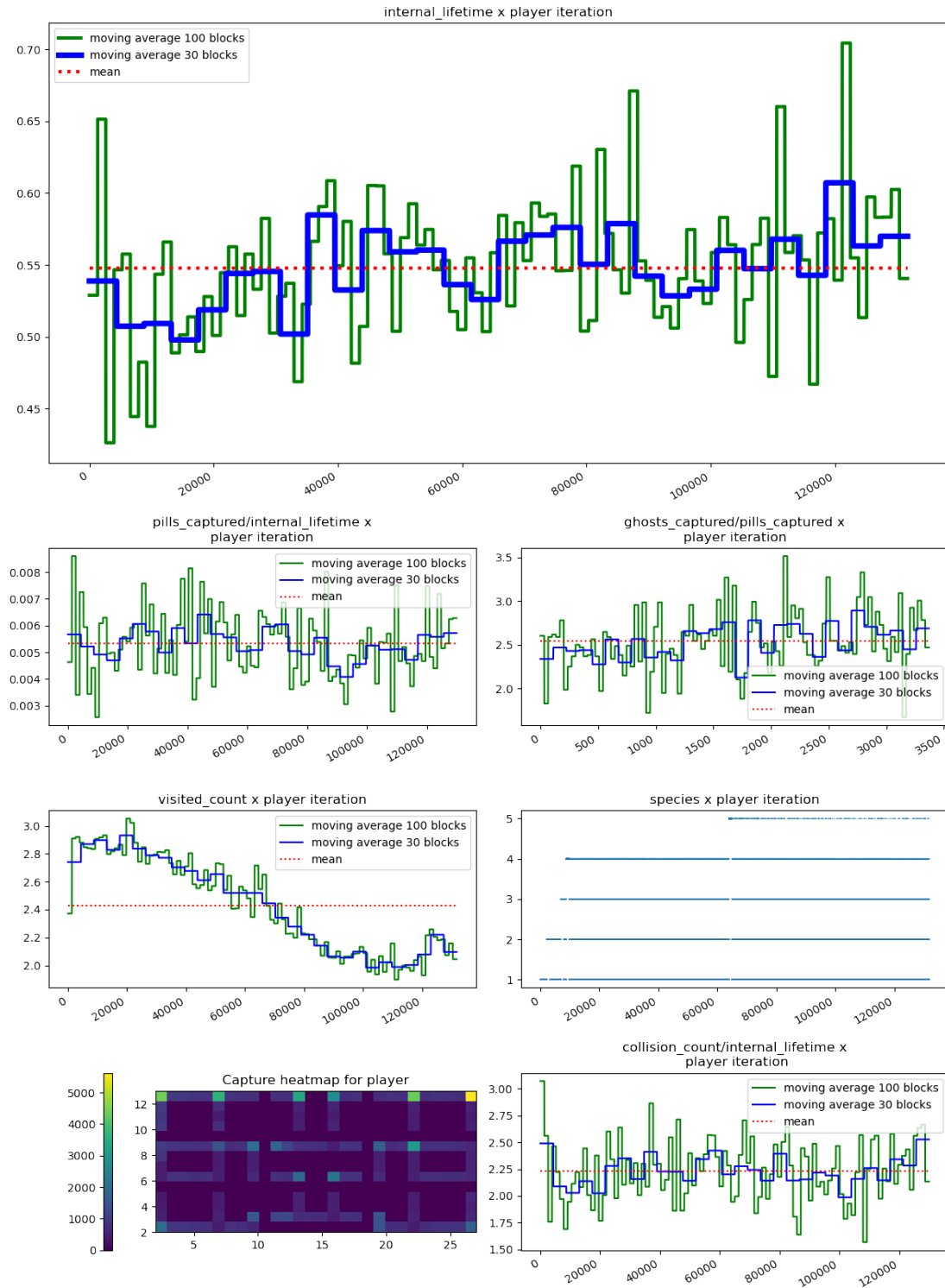
Figura 31: b1 e *lifetime* – NEAT(1677959384)

Fonte: próprio autor.

Figura 32: *bl_path_grading_hack* e *lifetime* – NEAT(1677959432)

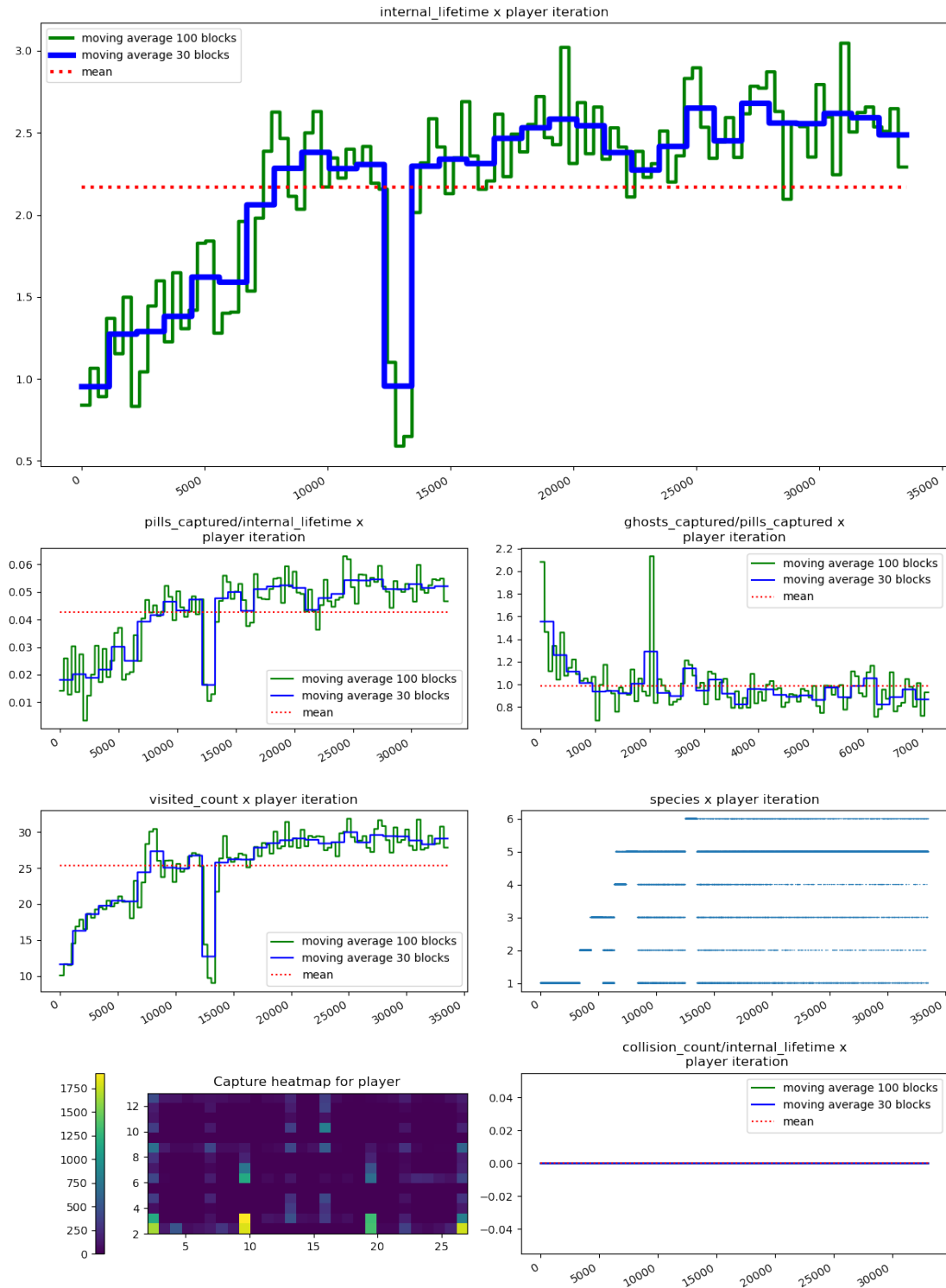


Fonte: próprio autor.

Figura 33: nb4 e *lifetime* – NEAT(1677959510)

Fonte: próprio autor.

Figura 34: *nb4_path_grading* e *lifetime* – NEAT(1677959569)



Fonte: próprio autor.

O ensaio 1677445806 utilizou o modo de operação *b1* e a função *fitness movement_captures_hack_26* e seu relatório está na Figura 19. Nota-se que a população adquiriu boa capacidade de movimentação, apesar de ainda ter indivíduos com um elevado índice de colisões. Existe uma concentração separada do grupo principal na taxa de colisões e há um ponto de concentração no *heatmap* do jogador, o que sugere que aquela posição era problemática.

O tempo de vida mostra uma tendência sutil de crescimento, observada mais nitidamente na contagem de *updates*. Observa-se que esta população ainda estava melhorando a movimentação e o tempo de vida quando o ensaio foi interrompido. A capacidade de capturar pílulas é explicado pela melhor mobilidade e como a eficiência de captura mostra tendência de queda, podemos considerar que não houve aumento da captura de fantasmas.

Quando comparado com seu análogo que usou a função de *fitness lifetime* (1677445899 – Figura 23) observou-se que neste ensaio a população evoluiu a movimentação mais rapidamente e atingiu patamares mais altos de tempo de vida embora a outra mostre sinais fortes de crescimento nessas métricas. Aqui nota-se que a função *fitness movement_captures_hack-26* cumpriu o seu papel de priorizar a movimentação, e como esperado repercutiu no tempo de vida e capacidade de captura, mas a desaceleração no ritmo de crescimento das métricas deixa dúvidas sobre seu impacto na evolução da população.

O ensaio 1677445825 usou o modo de operação *b1_path_grading_hack* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 20. A população deste ensaio adquiriu boa movimentação e o fez rapidamente. Este ensaio ficou preso em um máximo local. O *heatmap* de capturas do jogador e dos fantasmas mostra uma concentração no quadrante superior direito, o que indica que alguma padronização de movimentação, o que é esperado para esse modo de operação.

Quando comparado ao seu análogo que usou a função de *fitness lifetime* (1677445913 – Figura 24) nota-se que neste ensaio a população adquiriu a capacidade de movimentação muito mais rapidamente, mas no outro a população mostrou níveis mais altos de tempo de vida e captura o que novamente sugere que a função de *fitness movement_captures_hack_26* pode atrapalhar a evolução da população a longo prazo.

O ensaio 1677445848 usou o modo de operação *nb4* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 21. A população deste ensaio adquiriu boa capacidade de movimentação e rapidamente. A análise do *heatmap* mostra que tinham dificuldade em sair do corredor mais curto do jogo.

Quando comparado ao seu análogo que usou a função de *fitness lifetime* (1677445930 – Figura 25) nota-se que neste ensaio a população evolui a movimentação mais rapidamente e a um patamar mais alto, mas a outra população mostra sinais fortes de crescimento no fim do ensaio e já exibia melhor tempo e vida o que novamente sugere que a função de *fitness movement_captures_hack_26* pode afetar negativamente a evolução a longo prazo.

O ensaio 1677445867 usou o modo de operação *nb4_path_grading* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 22. A população deste ensaio teve bom tempo de vida, a um nível que indica que desenvolveu a capacidade de evasão, e desenvolveu boa movimentação. O tempo de vida, a eficiência de captura e a movimentação mostravam sinais de crescimento quando o ensaio foi interrompido.

Quando comparado ao seu análogo que usou a função de *fitness lifetime* (1677445953 – Figura 26) nota-se que a população desse ensaio evoluiu mais rapidamente e para patamares mais altos de movimentação e tempo de vida. Nota-se que neste caso a função de *fitness movement_captures_hack_26* funcionou bem, sem impactar negativamente a evolução da população e como este modo de operação não precisa evoluir a movimentação, infere-se que ela impacta a diversidade por priorizar a movimentação.

O ensaio 1677445899 usou o modo de operação *b1* e a função de *fitness lifetime* e seu relatório está na Figura 23. A população deste ensaio estava com a capacidade de movimentação e o tempo de vida em pleno desenvolvimento quando o ensaio foi finalizado. Alguns indivíduos ainda apresentavam alta taxa de colisão, o que é esperado pela forma como o modo *b1* opera.

O ensaio 1677445913 usou o modo de operação *b1_path_grading* e a função de *fitness lifetime* e seu relatório está na Figura 24. A população deste ensaio evoluiu a movimentação e o tempo de vida a níveis comparáveis aos melhores observados, mas mostra sinais de estagnação. Devido ao nível de tempo de vida que alcançaram, acredita-se que desenvolveram a capacidade de evasão.

O ensaio 1677445930 usou o modo de operação *nb4* e a função de *fitness lifetime* e seu relatório está na Figura 25. A população deste ensaio atingiu patamares de tempo de vida e movimentação semelhantes aos melhores vistos nesta bateria e que ainda mostravam forte crescimento quando o ensaio foi finalizado. Esta população só começou a mostrar melhora com quase 300.000 iterações, o que era esperado, pois esse algoritmo tem que aprender todas as regras da movimentação. Devido ao nível de tempo de vida que alcançaram, acredita-se que desenvolveram a capacidade de evasão. Nesse ensaio é interessante notar a alta correlação

entre movimentação, captura de pílulas e tempo, e a correlação inversa com a taxa de colisões.

O ensaio 1677445953 usou o modo de operação *nb4_path_grading* e a função de *fitness lifetime* e seu relatório está na Figura 26. A população deste ensaio atingiu bom nível de tempo de vida e uma movimentação média e mostrava um tímido crescimento quando o ensaio foi interrompido. Devido ao nível de tempo de vida que alcançaram, acredita-se que desenvolveram a capacidade de evasão.

O ensaio 1677959359 usou o sistema NEAT, modo de operação *b1* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 27. A população deste ensaio apresentou uma melhora inicial na movimentação e estagnou. Alguns indivíduos ainda apresentavam alta taxa de colisão, o que é esperado pela forma como o modo *b1* opera.

A população deste ensaio não especiou muito e prevaleceu a topologia inicial e notou-se durante a execução que aprenderam a evadir sem aprender a movimentar-se propriamente.

Quando comparada à sua análoga que usou a função de *fitness lifetime* (1677959384 – Figura 31) a população deste ensaio desenvolveu alguma movimentação e a do outro ensaio teve o tempo de vida médio melhor, o que são as respectivas prioridades de suas funções de *fitness*. Ambas as populações estavam estagnadas quando o ensaio foi finalizado.

O ensaio 1677959407 usou o sistema NEAT, o modo de operação *b1_path_grading_hack* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 28. As métricas dessa população pioraram em quase toda a evolução do algoritmo, mas nota-se uma brusca recuperação pouco antes de o ensaio ser interrompido. Observou-se que adição de capacidades não funcionou corretamente e a população na sua maior parte exibiu a mesma topologia inicial. Foi descoberta uma segunda espécie praticamente extinta. Observou-se do ensaio que adquiriram a capacidade de evadir sem aprender a movimentar-se propriamente.

Quando comparada à sua análoga que usou a função de *fitness lifetime* (1677959432 – Figura 32) a população do outro ensaio tem melhor tempo de vida, mas movimentação pior, o que são as prioridades de suas respectivas funções de *fitness*. No outro ensaio foi observado uma maior variação na topologia e uma segunda espécie está presente até o fim do ensaio, o que sugere que a otimização do sistema de especiação deve levar a função *fitness* em consideração.

O ensaio 1677959485 usou o sistema NEAT, modo de operação *nb4* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 29. A população deste ensaio estagnou e mostrou uma piora nas métricas e sinais de estagnação até iniciar uma

brusca recuperação antes da finalização do ensaio. Observou-se que nesse ensaio a especiação funcionou corretamente e existem 4 espécies com presenças igualmente distribuídas.

Quando comparado ao seu análogo que usou a função de *fitness lifetime* (1677959510 – Figura 33) a população do outro exhibe melhor média de tempo de vida e mostra um tímido crescimento nessa métrica que é constante até o fim do ensaio enquanto esta tem melhor movimentação, mas ambas são ruins nessa métrica. Especiaram de forma parecida, mas a outra população ainda possui uma quinta espécie.

O ensaio 1677959539 usou o sistema NEAT, modo de operação *nb4_path_grading* e a função de *fitness movement_captures_hack_26* e seu relatório está na Figura 30. A população deste ensaio mostrou uma melhora inicial em movimentação e bom tempo de vida antes de estagnar. Observou-se durante o ensaio que alguns indivíduos adquiriram a capacidade de evasão. A população especiou e mostra prevalência da segunda espécie desde a sua descoberta até o fim do ensaio.

Quando comparada à sua análoga que usou a função de *fitness lifetime* (1677959569 – Figura 34) a população deste ensaio teve alguma melhora em movimentação antes de estagnar, o do outro atingiu melhor movimentação e tempo de vida, mas também exhibe sinais de estagnação. A outra população especiou de forma bastante diferente, sugerindo novamente que a função de *fitness* deve ser considerada para otimizar a especiação.

O ensaio 1677959384 usou o sistema NEAT, modo de operação *b1* e a função de *fitness lifetime* e seu relatório está na Figura 31. A população deste ensaio estagnou após uma tímida melhora no tempo de vida. Houve uma única especiação que mostra uma presença enfraquecida desde sua descoberta, mas não mostra sinais de extinção.

O ensaio 1677959432 usou o sistema NEAT modo de operação *b1_path_grading_hack* e a função de *fitness lifetime* e seu relatório está na Figura 32. A população deste ensaio estava estagnada. Houve uma especiação, mas a espécie inicial permaneceu dominante até o fim do ensaio, embora não possa ser observado indícios de extinção da segunda espécie.

O ensaio 1677959510 usou o sistema NEAT, modo de operação *nb4* e a função de *fitness lifetime* e seu relatório está na Figura 33. A mostra um tímido crescimento do tempo de vida constante durante todo ensaio. A movimentação piorou inicialmente e mostra uma tímida recuperação ao final do ensaio. Ocorreram algumas especiações e existem 4 espécies igualmente distribuídas ao fim do ensaio e uma com algum sinal de enfraquecimento. Neste ensaio a especiação funcionou bem e talvez a população mostraria melhora caso o ensaio não tivesse sido interrompido.

O ensaio 1677959569 usou o sistema NEAT, modo de operação *nb4_path_grading* e a função de *fitness lifetime* e seu relatório está na Figura 34. Este ensaio foi o mais bem-sucedido com o sistema NEAT e mostrou desempenho melhor que o seu análogo de topologia fixa, atingindo resultados comparáveis aos melhores da bateria. Essa população especiou bastante e existiam 6 espécies ao fim do ensaio, com a prevalência da número 5 desde a sua descoberta. Nota-se diminuição da presença das espécies 2 e 4. Nota-se uma correlação bem forte entre movimentação, captura de pílulas e fantasmas. Acredita-se que essa população poderia melhorar mais caso o ensaio não tivesse sido interrompido.

No heatmap é possível observar uma concentração na parte inferior condizente com o que foi observado durante o ensaio de que essa população tinha uma preferência pelo corredor inferior central e os dois corredores curtos de acesso.

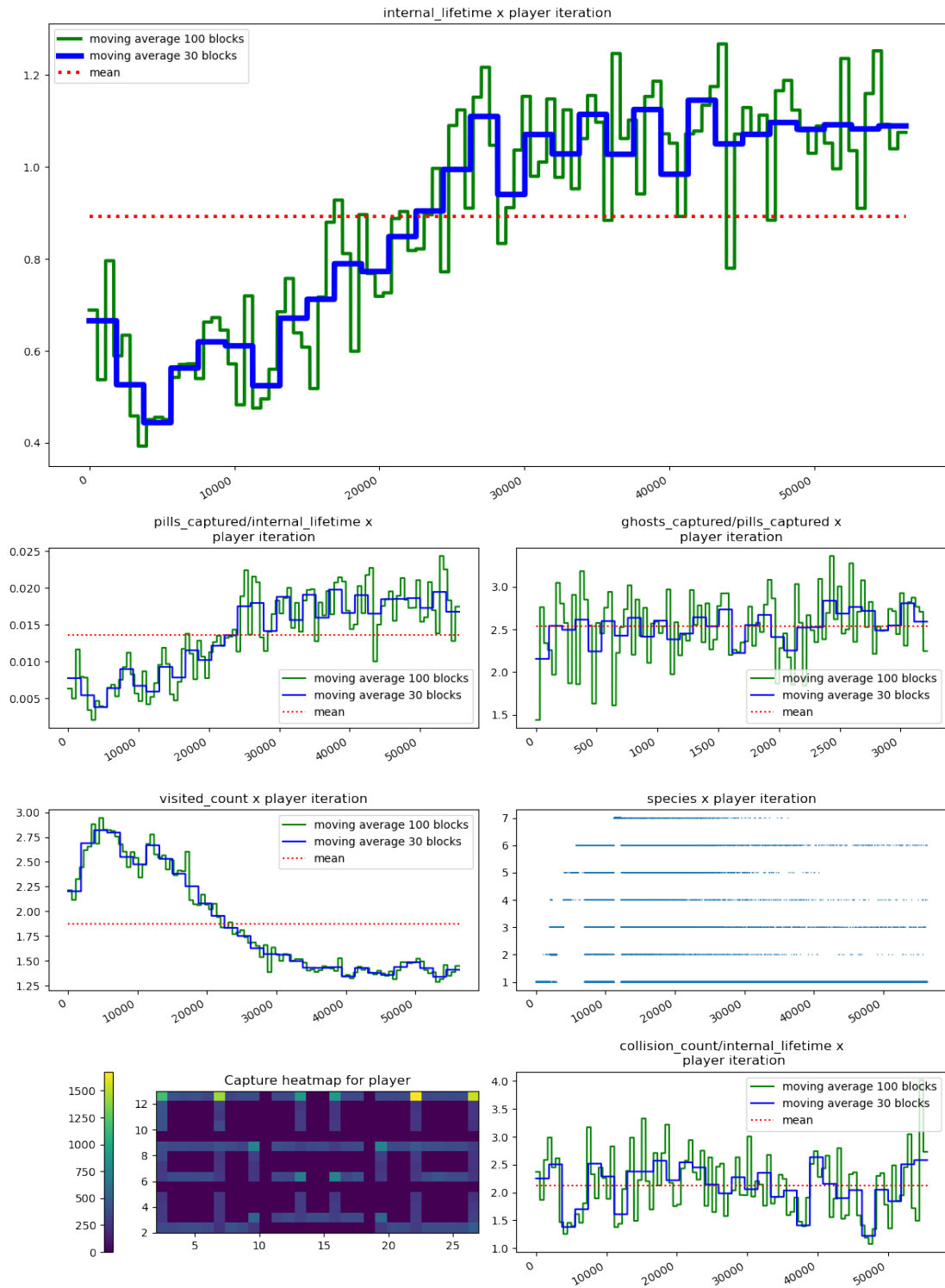
Analisando o resultado dos ensaios NEAT e contrastando as diferenças dos modos de operação, notou-se que no modo *nb1_path_grading* os indivíduos não precisam aprender a movimentação e só precisam de uma conexão para ser possível aprenderem a evasão e esse foi o único modo de operação que obteve bons resultados. Os indivíduos dos modos de operação *b1* e *b1_path_grading_hack* precisam de ao menos uma conexão para aprenderem a movimentação, enquanto os indivíduos do modo *nb4* precisam de ao menos 4 conexões para que a capacidade de movimentação plena seja possível.

Os ensaios NEAT foram configurados que a população fosse iniciada com topologia mínima, ou seja, os neurônios de saída e uma conexão e acredita-se que isso possa ter impedido que o *fitness* aumentasse gradualmente, já que a movimentação é necessária para que as outras capacidades apareçam, como foi observado na análise de *baseline* e por isso resolveu-se realizar ensaios com os modos *b1*, *b1_path_grading_hack* e *nb4* usando o número mínimo de conexões que possibilite que a movimentação plena apareça para criar a população inicial. Acredita-se que esse problema foi agravado por iniciar os jogadores sem direção, pois isso dificulta que indivíduos que só tem movimentação parcial sejam bem-sucedidos.

Nos ensaios anteriores observou-se que a população de *b1* e *b1_path_grading_hack* usavam a sua conexão para evadir, então optou-se por usar duas conexões mínimas. O limite de especiação foi diminuído, já que foi observado que *b1_path_grading_hack* e *b1* não especiam muito. Os resultados gráficos desses ensaios podem ser vistos nas Figuras de 35 a 37. O conjunto desses ensaios será referido como ensaios bateria 3 *take_2*. Os ensaios são:

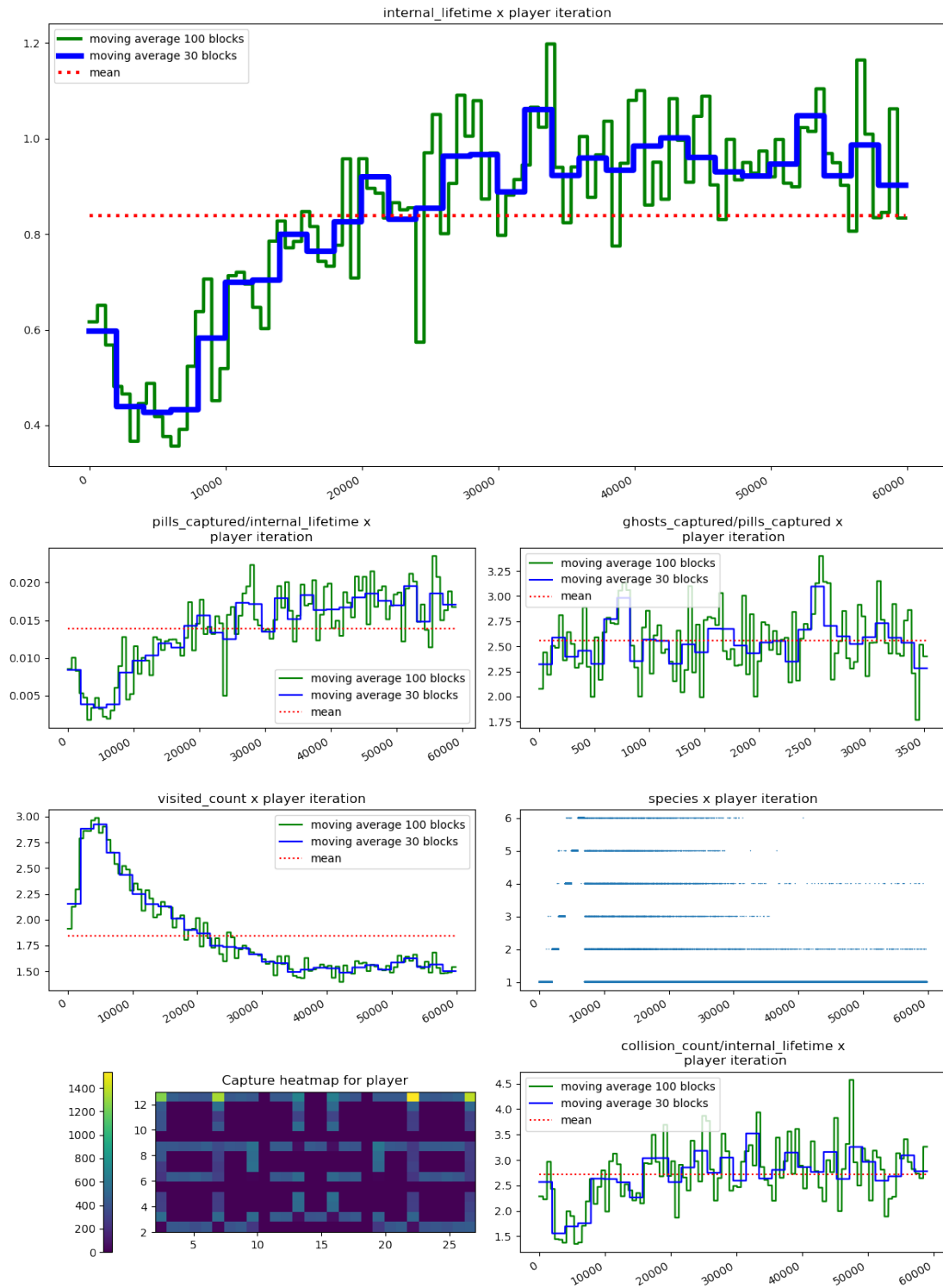
- 1678204540 *b1*, *lifetime* e duas conexões iniciais, Figura 35.
- 1678204555 *b1_path_grading_hack*, *lifetime* e duas conexões iniciais, Figura 36.
- 1678205691 *nb4*, *lifetime* e quatro conexões iniciais, Figura 37.

Figura 35: b1, *lifetime* e duas conexões iniciais (1678204540)



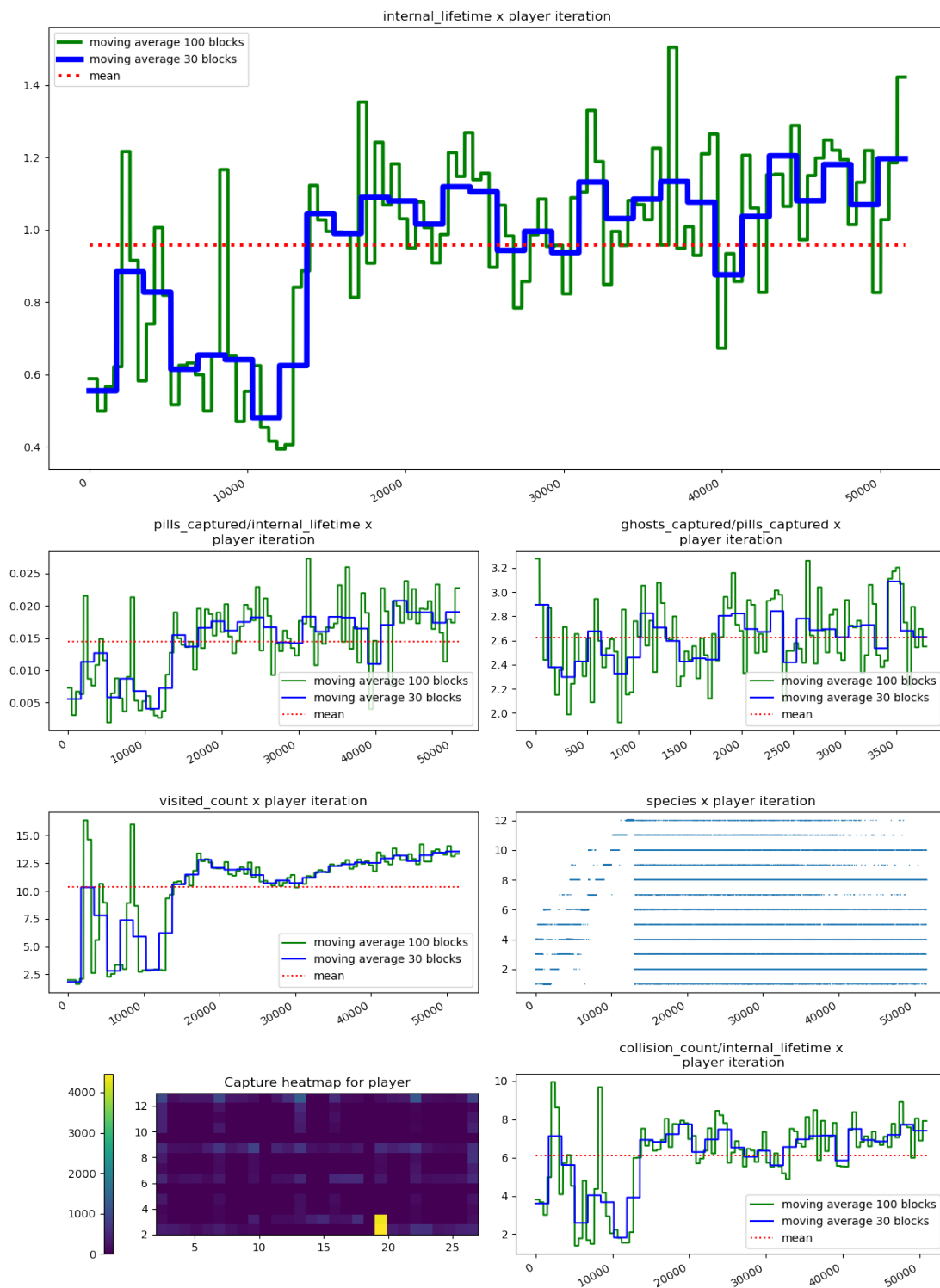
Fonte: próprio autor.

Figura 36: *b1_path_grading_hack*, *lifetime* e duas conexões iniciais (1678204555)



Fonte: próprio autor.

Figura 37: nb4, *lifetime* e quatro conexões iniciais (1678205691)



Fonte: próprio autor.

Comparando o ensaio 1678204540 (Figura 35) com seu análogo dos testes iniciais (1677959384, Figura 31), observamos que as populações atingiram patamares semelhantes, embora tenham progressões diferentes. Podemos observar que com a nova configuração, ocorreram mais especiações, mas que essas mostram sinais de enfraquecimento, sugerindo que as novas espécies não conseguem manter competitividade. Com a nova configuração resolveu-se parte do problema da especiação, mas foi insuficiente.

Comparando o ensaio 1678204555 (Figura 36) com seu análogo dos testes iniciais (1677959432, Figura 32), observamos que as progressões diferiram, mas sem muita alteração nos níveis alcançados. A população do teste inicial mostrou um tempo de vida ligeiramente maior. A população do teste com as novas configurações especiou 6 vezes, mas as novas espécies mostram sinais de enfraquecimento, sugerindo que tem dificuldade em manter a competitividade. A nova configuração resolveu parte do problema da especiação, mas foi insuficiente.

Os indivíduos dos ensaios da nova configuração (*take_2*), nos modos *b1* e *b1_path_grading_hack* não adquiriam a capacidade de evasão.

Comparando o ensaio 1678205691 (Figura 37) com seu análogo dos testes iniciais (1677959510, Figura 33), observamos que a população do segundo ensaio conseguiu níveis melhores de movimentação e tempo de vida. No relatório do novo ensaio observa-se que a população especiou mais e existem espécies competitivas. Durante a realização do ensaio observou-se que alguns indivíduos adquiriram movimentação parcial e acredita-se que a nova configuração pode ter acelerado o processo evolutivo, embora não seja possível avaliar o efeito que causa na evolução a longo prazo.

Analisando esses resultados de forma geral, concluiu-se que aumentar a contagem de conexões iniciais pode acelerar o processo evolutivo e embora tenha sido observado melhora na especiação, as novas espécies não se mantiveram competitivas e pode ter sido causado pela alteração nos hiperparâmetros de especiação.

Para observar melhor o efeito do limite de especiação, optou-se por realizar mais ensaios, com esse ainda menor. Para resolver o problema da progressividade do *fitness* sem usar mais conexões iniciais, a população foi configurada para iniciar em uma orientação válida. Nesses ensaios observou-se que a alteração impediu a progressão da população, pois o nicho cresceu mais rapidamente que as iterações e o ensaio permaneceu em modo nicho, não voltando a operação com a população geral. Esse problema poderia ter sido resolvido com a diminuição do tamanho de nicho, o que em si pode levar a perda de competitividade de novas

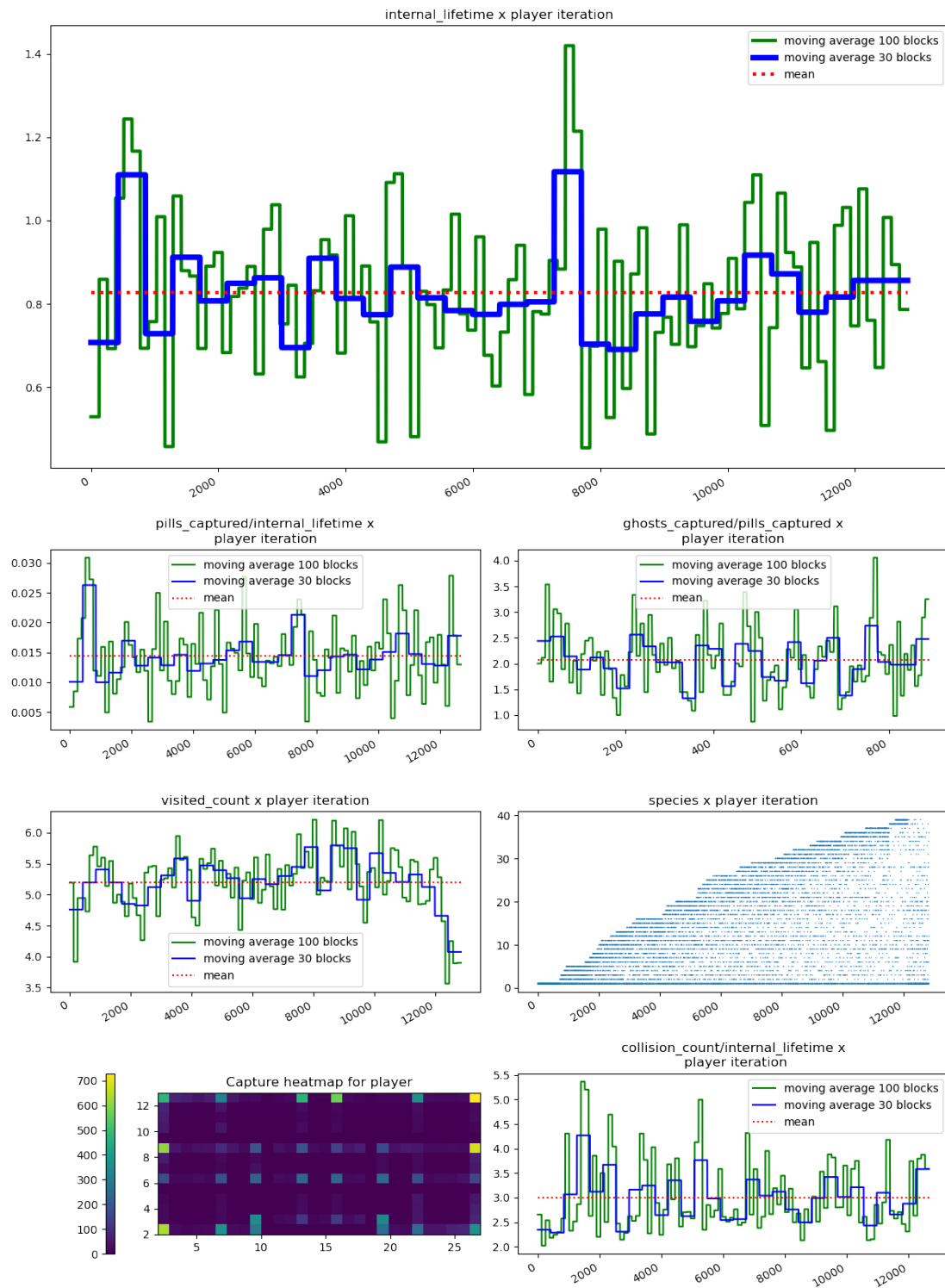
espécies, e como a taxa de especiação pode variar ao longo do ensaio, pode ser dispendioso encontrar a configuração ideal.

Para resolver o problema, o sistema de nicho foi alterado para alternar aleatoriamente entre a população geral e o nicho, com chances iguais para cada. A fila do nicho passou a ser embaralhada a cada adição de espécie e caso o tamanho de nicho ultrapasse um limite (configurável) o sistema elimina a mesma quantidade adicionada. Com isso, o tamanho de nicho é regulado dinamicamente pelo sistema e a população geral pode continuar evoluindo paralelamente. Esse novo modo pode ser habilitado por configuração.

O novo modo de nicho foi testado bateria 3 *take_3*, nos ensaios:

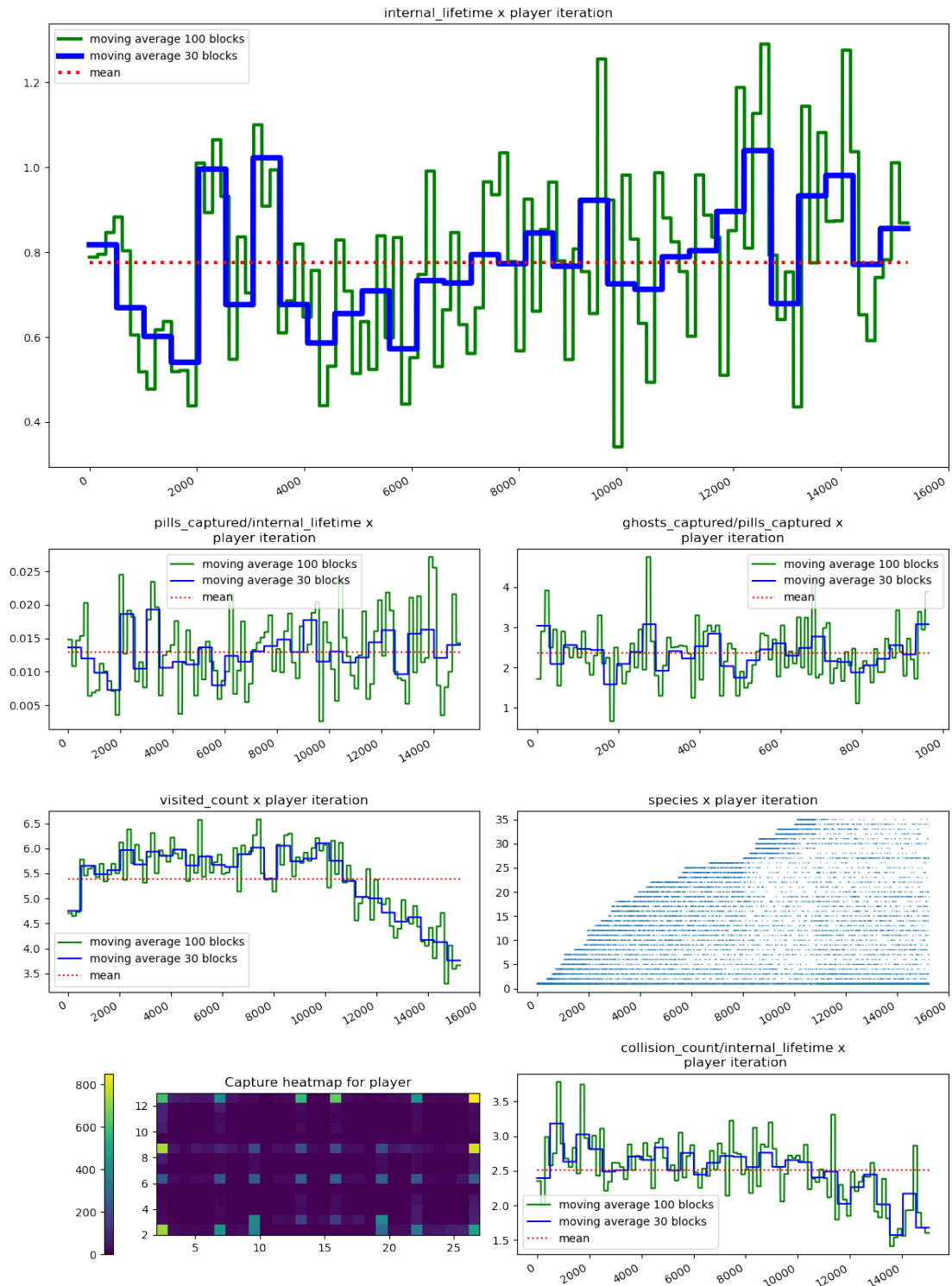
- 1678278297 b1, *lifetime*, Figura 38.
- 1678278313 *b1_path_grading_hack*, *lifetime*, Figura 39.
- 1678278779 nb4, *lifetime*, Figura 40.

Analisando esses ensaios se notou que o novo modo de operação do nicho atingiu o efeito esperado e com a nova configuração ocorreram muitas especiações, talvez até excessivamente. Note que nesse novo sistema, como o nicho não é mais necessariamente aplicado de forma sequencial, por espécie e por isso, a prevalência de novas espécies pode não se mostrar da mesma forma. Pode ser observado que existem espécies que tiveram a sua presença diminuída e conseguiram recuperar espaço.

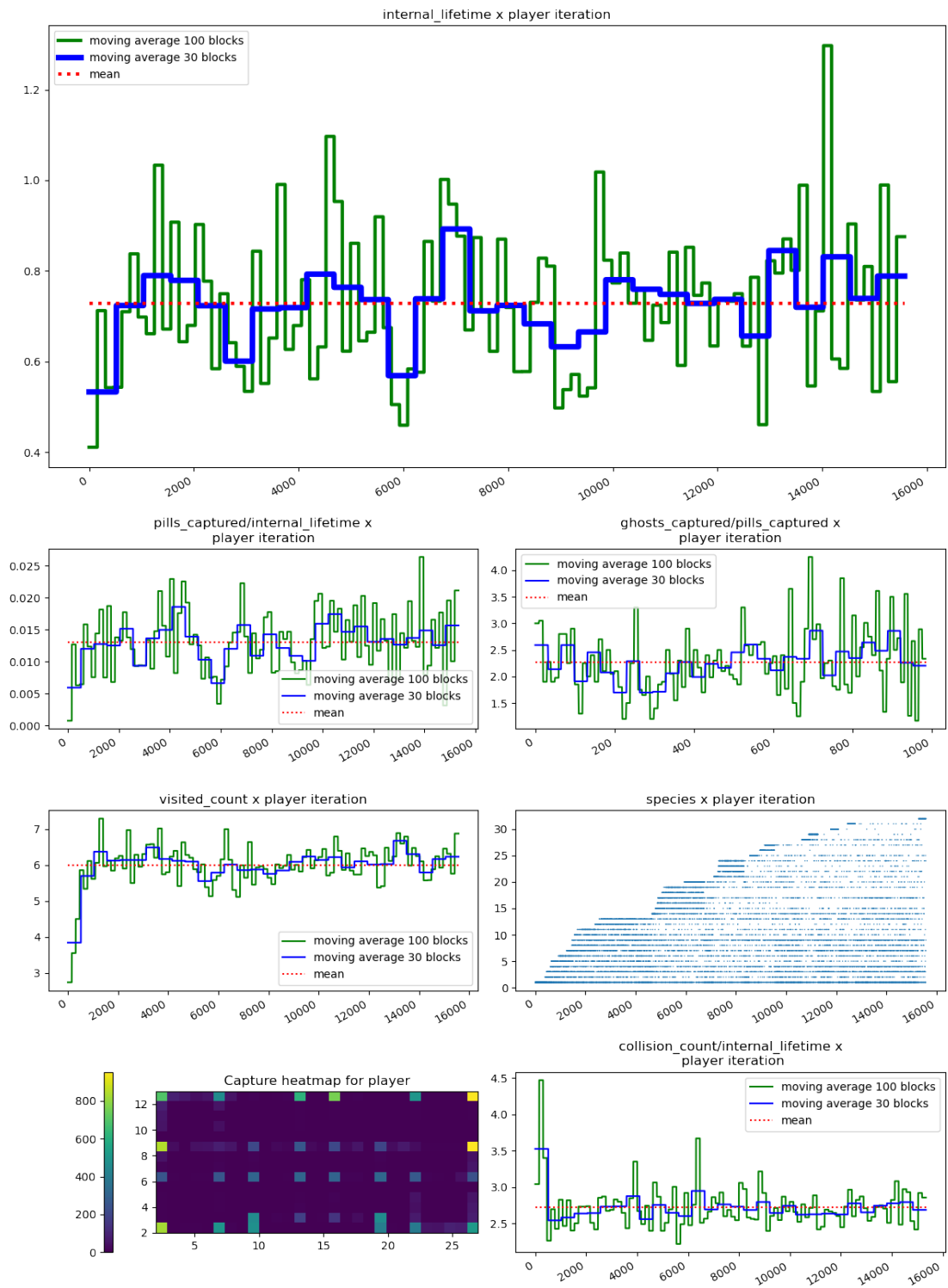
Figura 38: b1, *lifetime* – nova especiação (1678278297)

Fonte: próprio autor.

Figura 39: *bl_path_grading_hack, lifetime* – nova especiação (1678278313)



Fonte: próprio autor.

Figura 40: nb4, *lifetime* – nova especiação (1678278779)

Fonte: próprio autor.

4 CONCLUSÕES

4.1 Conclusões baseline (bateria 1)

Esta bateria de testes tinha como objetivo desenvolver e avaliar a metodologia de análise. Acredita-se que os relatórios produzidos são suficientes para analisar o comportamento conferido pelo modelo aos indivíduos e foi possível observar mudanças gráficas e estatísticas com a presença de diferentes capacidades na população.

Notou-se importante para realizar comparações de forma efetiva que a progressão da adição de capacidades aos algoritmos especialistas seja feita de forma delimitada, evitando adicionar múltiplas capacidades simultaneamente.

Analisar a população dos algoritmos especialistas mostrou-se útil para verificar como a progressão de capacidades afeta as métricas de desempenho, entender como estas estão relacionadas e visualizar caminhos para que elas sejam adquiridas por um modelo neuro-evolutivo, além de auxiliar a avaliar possíveis métricas para serem utilizadas para a função fitness.

Implementar esses algoritmos auxiliou no entendimento do problema e da forma como pode ocorrer a progressão das capacidades em um modelo neuro-evolutivo. Realizar essa implementação pode não ser uma tarefa trivial em todas as aplicações e talvez tenha que ser omitida, embora não tenha sido testado dessa forma. Uma consequência inesperada da elaboração desses algoritmos foi que o seu desenvolvimento inspirou o modo de operação da terceira forma.

Os algoritmos especialistas poderiam ter sido utilizados para analisar as funções de *fitness* diretamente, analisando como operam em uma população com comportamento fixo.

A análise dos resultados dessa etapa serviu para encontrar erros e limitações da infraestrutura e acredita-se que a análise deva ser feita desde as etapas iniciais do desenvolvimento.

4.2 Conclusões Neuro-evolutivo de topologia fixa (bateria 2)

A execução desta bateria exigiu a maior parcela do tempo dedicado ao trabalho. Note que ela acumulou a tarefa de seleção dos ensaios da bateria 3 entre outras responsabilidades, como o desenvolvimento e validação dos modos de operação e de toda infraestrutura do GA.

Devido à quantidade de testes realizados nessa bateria, observou-se que organizar, executar, consolidar e encontrar resultados demanda organização e tempo, além de ser propício a erros e por isso é desejável que seja autonomizável.

4.3 Conclusões NEAT e Neuro-evolutivo de topologia fixa (bateria 3)

A utilização de um algoritmo NEAT elimina a necessidade de ensaios para escolha de topologia, mas com pode ser observado pelos resultados, existem hiperparâmetros, como os de nicho e especiação, que precisam ser otimizados de forma que ensaios preliminares ainda são necessários para alcançar melhores resultados. Não conseguiu-se explorar o potencial do sistema NEAT já que foi executada uma quantidade limitada de ensaios.

Após a realização desta bateria, acredita-se que os resultados obtidos sobre os modos de operação e função de fitness quando usados em neuro-evolução de topologia fixa não são diretamente aplicáveis a um sistema NEAT e talvez também a função de ativação. Acredita-se que os modos B2 e B3 teriam vantagens ao serem usados nesse tipo de sistema.

Essa etapa mostrou a importância de modificar o código e configurações da implementação com prudência e organização. Mudanças que parecem mínimas e triviais podem ter efeitos colaterais aparentemente desassociados. As modificações na configuração dos fantasmas acabaram criando a necessidade de revisar e modificar toda a implementação dos agentes, o que acabou acontecendo simultaneamente a revisão do sistema de métricas que também interferiu nestes agentes, o que escalou o problema.

Este tipo de sistema é sensível a mudanças e por mais de uma vez o sistema parou de funcionar e diversas modificações tiveram que ser analisadas para encontrar a alteração responsável. Recomenda-se que as modificações sejam incrementais, que seja utilizado um sistema de versionamento e que os testes sejam conduzidos sempre em versões limpas e se armazene a versão do código utilizada no teste, de forma que regressões sejam resolvidas mais facilmente.

Teria sido interessante adicionar alguma métrica para a variedade genética, o que teria auxiliado a realizar análises da população como um todo e auxiliado a avaliar diferentes funções de *fitness* e a adaptabilidade.

Embora tenha sido possível inferir sobre o funcionamento da especiação nos sistemas NEAT a partir dos dados que já haviam sido coletados, para essa finalidade a criação de eventos específicos da especiação, como a descoberta de inovações e de espécies e a sua eventual extinção teriam sido úteis nas análises e para otimizar os hiperparâmetros.

4.4 Considerações Finais

Inicialmente acreditava-se que a análise seria útil como ferramenta para auxiliar o entendimento do comportamento do algoritmo, da população e a sua progressão e servir para otimizar hiperparâmetros e, por fim, verificar a adequabilidade de uma população para o problema a que ela se propõe resolver. Essa concepção veio como resposta a alguns problemas que foram encontrados durante a realização do trabalho precursor a este, o [Extinção 3](#), que após finalizado mostrou que implementar a infraestrutura de código e coletar os resultados é somente um dos problemas para a aplicação desse tipo de algoritmo, já que entender os resultados obtidos e qualificá-los mostrou-se um desafio o que foi inesperado.

Acredita-se que as técnicas de visualização e análise como empregadas neste trabalho auxiliam no processo como foi proposto, mas mostraram-se úteis também para identificar erros na infraestrutura e limitações das métricas escolhidas e errou-se ao não ter implementado o sistema de análise logo no início e desenvolvê-lo em paralelo ao restante da infraestrutura, muitos erros e limitações teriam sido identificados mais cedo se as análises estivessem disponíveis.

Foram treinadas boas populações e embora não se tenha obtido nenhuma população que mostrasse capacidades avançadas, foi possível observar agentes que se moviam bem, fugiam dos fantasmas, capturavam pílulas e ficavam parados aguardando os fantasmas. Essa limitação foi prevista na análise do uso do tempo de vida como fitness e se criou uma alternativa baseada em uma combinação de movimentação e captura, mas que se acredita tenha impactado negativamente o desenvolvimento da população a longo prazo e resultados melhores não foram obtidos.

Algumas populações, particularmente dos modos nb4 e nb4_path_grading mostraram bastante potencial e se comportam de maneira natural, sem terem desenvolvido vícios de movimentação e acredita-se que com mais ensaios, otimizações e uma função de fitness que crie pressão seletiva para a captura de fantasmas e não prejudique a evolução da população, estas seriam aplicáveis a um jogo.

Com a finalização deste trabalho, avançou-se na parte da análise e o obstáculo mudou de lugar. A maior agilidade em entender os resultados e propor novos ensaios evidenciou problemas relacionados a escalabilidade. Os testes realizados foram minimizados por limitação dos recursos de *hardware* disponíveis e da infraestrutura de *software*, que foi inicialmente concebida para rodar em uma estação de trabalho e nessa configuração um ensaio pode demorar alguns dias. O volume de dados produzidos foi muito maior de que se esperava

e a existência de um método robusto para armazenar e encontrar análises, relatórios e configurações teria auxiliado todo o desenvolvimento.

4.5 Melhorias

Durante a realização deste trabalho diversas melhorias foram idealizadas e não puderam ser implementadas. Essa lista contém ideias em diferentes níveis de desenvolvimento e estão aqui para referência futura e provavelmente não possuem mérito. Elas são:

- Implementar o alinhamento funcional dos genes antes do cruzamento, este alinhamento teria que ser propagado para as todas as camadas, de forma sequencial. A intenção é eliminar o problema *Competing Conventions*. Não se sabe se isto é possível ou vantajoso.

- Aplicar diferentes funções de *fitness* para os algoritmos especialistas. A finalidade seria verificar como elas se comportam com diferentes capacidades e como o *fitness* progride.

- Implementar alguma forma de treinamento para os indivíduos, este poderia ocorrer antes, durante ou após a neuro-evolução, ou até mesmo de forma intercalada. Os dados para o conjunto de treinamento seriam coletados de sessões de jogo com um jogador humano, um algoritmo especialista, ou até mesmo de um modelo neuro-evolutivo que exiba propriedades que se queira conferir à população. Talvez seja interessante explorar os casos em que o conjunto de treinamento apresente decisões diferentes para a mesma entrada. O objetivo deste treinamento seria acelerar o processo de aprendizado, eliminar vícios de comportamento e ainda manter a variabilidade e adaptabilidade ao aplicar as redes treinadas a um GA. Acredita-se que este tipo de sistema seria mais simples de ser aplicado a um jogo digital, mas não foi investigado. Talvez não seja trivial de ser implementado com NEAT.

- Implementar um sistema neuro-evolutivo que calcule o *fitness* baseado na similaridade da resposta do indivíduo a um conjunto de treinamento. Os dados para o conjunto de treinamento seriam coletados como na melhoria mencionada acima, mantendo a ressalva sobre ambiguidades. As vantagens que esse sistema conferiria as mesmas do mecanismo acima com a vantagem de poder ser usado com NEAT e/ou combinado com o treinamento acima.

- Monitorar a diversidade genética.

- Salvar informações de criação e extinção de espécies para o NEAT.

- Salvar informações sobre adição de inovações para o NEAT.

- Elaborar algum método para testar a adaptabilidade da população.
- Monitorar os genes, fazer análises de clusterização e procurar por correlação entre o resultado da clusterização e o *fitness*.
- O sistema construído tem duas implementações separadas e independentes de ANNs, caso se queira usá-lo é interessante unificar essas implementações.
- Explorar a utilização mais aprofundada dos dados no desenvolvimento da função *fitness*. Isso foi feito somente de forma superficial neste trabalho.
- Realizar ensaios com coevolução de jogadores e fantasmas.
- Testar o desempenho de uma rede neural como preditora do *fitness*. Estender esse sistema para recomendar indivíduos.
- Implementar uma arquitetura cliente-servidor. O servidor seria responsável por criar indivíduos e coletar eventos. Com essa modificação seria possível ter mais de um cliente executando sobre a mesma população.
- Implementar clientes não gráficos.
- Implementar um sistema de visualização para a topologia usada pelo NEAT.
- Adicionar ao NEAT um sistema de variação dinâmica do valor limite para especiação(*autoplayer_neat_specie_threshold*). Poder-se-ia usar alguma regra baseada na quantidade de indivíduos que uma espécie conseguiu colocar na população geral durante seu nicho e a partir dela variar o tamanho do nicho e/ou *specie threshold*, ou ainda utilizar o intervalo(em iterações) entre especiações.
- Basear o sistema de nicho exclusivamente em variações das diferenças trazidas pela espécie, ou seja, salvar qual foram as inovações de um indivíduo e que levaram a especiação e criar o nicho randomizando valores somente para essas inovações.
- Atrelar o sistema de nichos do NEAT a criação de inovações em detrimento da especiação, pode ser combinado com o modelo de nicho acima.
- Priorizar a adição, em um sistema NEAT, de conexões para os neurônios de saída até que todos estejam conectados.

REFERÊNCIAS

BUCKLAND, M.; LAMOTHE, A. **AI Techniques for game programming**. Cincinnati: Premier Press, 2002.

MIKKULAINEN, R. Creating Intelligent Agents in Games. **The Bridge**, Winter 2006, p. 5-13, 2006.

STANLEY, K.O. et al. Evolving Neural Networks through Augmenting Topologies, **Evolutionary Computation**, v.10 p.99-127, jun, 2002.

PRADA, G. C. **Algoritmo Genético aplicado ao comportamento de uma população de NPCs em um jogo digital**. 2019. 36f. Trabalho de Graduação – Faculdade de Tecnologia de Ourinhos, Ourinhos, 2019

PRADA, G. C. **Extinção 3**. Disponível em: <https://github.com/guiprada/Extincao-3>. Acesso em: 13 Set. 2022.

PRADA, G. C. **Extinção 5**. Disponível em: <https://github.com/guiprada/Extincao-5>. Acesso em: 13 Set. 2022.